

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

1. ESLINT WARNINGS & LINT REPORT

```
> axiom@0.1.0 lint
> eslint
```

```
C:\projects\axiom\src\app\dashboard\actions.ts
```

```
25:3 warning 'createIncomeStream' is defined but never used @typescript-eslint/no-unused-vars
```

```
C:\projects\axiom\src\app\dashboard\page.tsx
```

```
41:10 warning 'formatCurrency' is defined but never used @typescript-eslint/no-unused-vars
```

```
67:10 warning 'kairosInsight' is assigned a value but never used @typescript-eslint/no-unused-vars
```

```
273:34 warning 'i' is defined but never used @typescript-eslint/no-unused-vars
```

```
C:\projects\axiom\src\components\dashboard\DayZeroOnboarding.tsx
```

```
17:8 warning 'IncomeType' is defined but never used @typescript-eslint/no-unused-vars
```

```
18:8 warning 'Cadence' is defined but never used @typescript-eslint/no-unused-vars
```

```
58:3 warning 'onComplete' is defined but never used @typescript-eslint/no-unused-vars
```

```
C:\projects\axiom\src\components\ui\animated-number.tsx
```

```
3:21 warning 'useState' is defined but never used @typescript-eslint/no-unused-vars
```

```
C:\projects\axiom\src\lib\ai\kairos.test.ts
```

```
3:10 warning 'generateSystemInsights' is defined but never used @typescript-eslint/no-unused-vars
```

```
C:\projects\axiom\src\lib\analytics\engine.ts
```

```
334:3 warning 'baseline' is assigned a value but never used @typescript-eslint/no-unused-vars
```

```
C:\projects\axiom\src\lib\analytics\types.ts
```

```
3:10 warning 'Account' is defined but never used @typescript-eslint/no-unused-vars
```

```
4:10 warning 'Liability' is defined but never used @typescript-eslint/no-unused-vars
```

```
5:10 warning 'IncomeStream' is defined but never used @typescript-eslint/no-unused-vars
```

```
6:10 warning 'FinancialGoal' is defined but never used @typescript-eslint/no-unused-vars
```

```
7:10 warning 'StrategicObjective' is defined but never used @typescript-eslint/no-unused-vars
```

```
C:\projects\axiom\src\lib\db\accounts.ts
```

```
2:32 warning 'AccountType' is defined but never used @typescript-eslint/no-unused-vars
```

```
C:\projects\axiom\src\lib\db\telemetry.ts
```

```
16:14 warning '_e' is defined but never used @typescript-eslint/no-unused-vars
```

```
â?? 17 problems (0 errors, 17 warnings)
```

2. TYPESCRIPT TYPE CHECK REPORT

No type errors found.

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

3. CRITICAL ARCHITECTURE CODE

FILE: package.json

```
{
  "name": "axiom",
  "version": "0.1.0",
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "lint": "eslint",
    "test": "node --import tsx --test src/lib/**/*.test.ts"
  },
  "dependencies": {
    "@supabase/auth-ui-react": "^0.4.7",
    "@supabase/auth-ui-shared": "^0.1.8",
    "@supabase/ssr": "^0.10.3",
    "@supabase/supabase-js": "^2.105.4",
    "framer-motion": "^12.40.0",
    "lenis": "^1.3.23",
    "lucide-react": "^1.17.0",
    "next": "16.2.6",
    "react": "19.2.4",
    "react-dom": "19.2.4",
    "recharts": "^3.8.1"
  },
  "devDependencies": {
    "@tailwindcss/postcss": "^4",
    "@types/d3-array": "^3.2.2",
    "@types/node": "^20",
    "@types/react": "^19",
    "@types/react-dom": "^19",
    "babel-plugin-react-compiler": "1.0.0",
    "eslint": "^9",
    "eslint-config-next": "16.2.6",
    "tailwindcss": "^4",
    "tsx": "^4.21.0",
    "typescript": "^5"
  }
}
```

FILE: tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2017",
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
"lib": ["dom", "dom.iterable", "esnext"],
"allowJs": true,
"skipLibCheck": true,
"strict": true,
"noEmit": true,
"esModuleInterop": true,
"module": "esnext",
"moduleResolution": "bundler",
"resolveJsonModule": true,
"isolatedModules": true,
"jsx": "react-jsx",
"incremental": true,
"plugins": [
  {
    "name": "next",
  },
],
"paths": {
  "@/*": [".src/*"],
},
},
"include": [
  "next-env.d.ts",
  "**/*.ts",
  "**/*.tsx",
  ".next/types/**/*.ts",
  ".next/dev/types/**/*.ts",
  "**/*.mts",
],
"exclude": ["node_modules", "mobile"],
}
```

FILE: src/app/dashboard/actions.ts

```
"use server";

import { revalidatePath } from "next/cache";
import { createClient } from "@/lib/supabase/server";
import {
  getDeployments,
  createDeployment,
  updateDeployment,
  deleteDeployment,
} from "@/lib/db/deployments";
import {
  getAccounts,
  createAccount,
  updateAccount,
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
    deleteAccount,
} from "@lib/db/accounts";
import {
    getLiabilities,
    createLiability,
    updateLiability,
    deleteLiability,
} from "@lib/db/liabilities";
import {
    getIncomeStreams,
    createIncomeStream,
    createIncomeStreams,
    updateIncomeStream,
    deleteIncomeStream,
} from "@lib/db/income";
import { getGoals, createGoal, updateGoal, deleteGoal } from "@lib/db/goals";
import {
    getStrategicObjectives,
    createStrategicObjective,
    updateStrategicObjective,
    deleteStrategicObjective,
} from "@lib/db/objectives";
import {
    getOperationalBaseline,
    createOperationalBaseline,
    updateOperationalBaseline,
    deleteOperationalBaseline,
} from "@lib/db/baseline";
import { getUserSettings, updateLiquidity } from "@lib/db/settings";
import { getInsights, saveInsight } from "@lib/db/insights";
import { getDeletedLedgerRecords } from "@lib/db/audit";
import { generateKairosAIInsight, type KairosInsight } from "@lib/ai/kairos";
import { generateSummary, type AnalyticsSummary } from "@lib/analytics";
import type {
    Deployment,
    Account,
    Liability,
    IncomeStream,
    FinancialGoal,
    StrategicObjective,
    OperationalBaseline,
    BaselineCadence,
} from "@lib/analytics/types";
import type { DeploymentAdvancedContextInput } from "@lib/finance/deploymentContext";

export interface DashboardSnapshot {
    authenticated: boolean;
    deployments: Deployment[];
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
accounts: Account[];
liabilities: Liability[];
incomeStreams: IncomeStream[];
goals: FinancialGoal[];
objectives: StrategicObjective[];
baseline: OperationalBaseline[];
analytics: AnalyticsSummary | null;
liquidity: number;
kairosInsight: KairosInsight | null;
}
```

```
export interface CreateDeploymentActionInput {
  title: string;
  amount: number;
  category: string;
  advancedContext?: DeploymentAdvancedContextInput;
  accountId?: string;
}
```

```
export interface UpdateDeploymentActionInput {
  title: string;
  amount: number;
  category: string;
  accountId?: string;
}
```

```
export interface CreateAccountActionInput {
  account_name: string;
  account_type: string;
  current_balance: number;
  institution?: string;
}
```

```
export interface CreateLiabilityActionInput {
  liability_name: string;
  liability_type: string;
  outstanding_balance: number;
  interest_rate?: number;
  minimum_payment?: number;
  due_date?: string | null;
  institution?: string;
  is_paid_in_cadences?: boolean;
  cadence?: string | null;
  cadence_day_date?: string | null;
  cadence_amount?: number | null;
}
```

```
export interface CreateIncomeActionInput {
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
income_name: string;
income_type: string;
amount: number;
cadence: string;
execution_day?: number | null;
is_recurring?: boolean;
source?: string;
start_date?: string;
end_date?: string | null;
```

```
}
```

```
export interface CreateGoalActionInput {
```

```
  goal_name: string;
  goal_type: string;
  target_amount: number;
  current_progress?: number;
  target_date?: string | null;
  priority: string;
  status: string;
  notes?: string;
```

```
}
```

```
export interface CreateObjectiveActionInput {
```

```
  objective_name: string;
  objective_type: string;
  target_amount: number;
  current_amount?: number;
  target_date?: string | null;
  priority_level: string;
  status: string;
  notes?: string;
```

```
}
```

```
function unauthenticatedSnapshot(): DashboardSnapshot {
```

```
  return {
    authenticated: false,
    deployments: [],
    accounts: [],
    liabilities: [],
    incomeStreams: [],
    goals: [],
    objectives: [],
    baseline: [],
    analytics: null,
    liquidity: 0,
    kairosInsight: null,
```

```
  };
```

```
}
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
const requireAuth = async () => {
  const supabase = await createClient();
  const {
    data: { user },
    error,
  } = await supabase.auth.getUser();
  if (error || !user) throw new Error("Unauthorized mutation attempt.");
  return { userId: user.id, supabase };
};

function normalizeSavedInsight(row: Record<string, unknown>): KairosInsight {
  const metadata =
    row.metadata && typeof row.metadata === "object"
      ? (row.metadata as Record<string, unknown>)
      : {};

  return {
    type: row.type as KairosInsight["type"],
    severity: (metadata.severity as KairosInsight["severity"]) || "observation",
    category: row.category as KairosInsight["category"],
    message: String(row.message || ""),
    supportingSignals: Array.isArray(metadata.supporting_signals)
      ? (metadata.supporting_signals as string[])
      : metadata.supporting_signal
        ? [String(metadata.supporting_signal)]
        : [],
    timestamp: String(
      metadata.timestamp || row.created_at || new Date().toISOString(),
    ),
    runway: metadata.runway !== undefined ? Number(metadata.runway) : null,
    capitalEfficiency:
      metadata.capital_efficiency !== undefined
        ? Number(metadata.capital_efficiency)
        : 100,
    isSilent: Boolean(metadata.is_silent),
    metadataQuality:
      metadata.metadata_quality && typeof metadata.metadata_quality === "object"
        ? (metadata.metadata_quality as KairosInsight["metadataQuality"])
        : undefined,
    is_new_signal: false,
  };
}

async function buildDashboardSnapshot(
  options: { forceInsightEvaluation?: boolean } = {},
): Promise<DashboardSnapshot> {
  const supabase = await createClient();
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
const {
  data: { user },
} = await supabase.auth.getUser();

if (!user) return unauthenticatedSnapshot();

console.log("ACTIVE SESSION USER ID:", user.id);

const [
  deploymentsData,
  accountsData,
  liabilitiesData,
  incomeStreamsData,
  goalsData,
  objectivesData,
  baselineData,
  settings,
] = await Promise.all([
  getDeployments(supabase),
  getAccounts(supabase),
  getLiabilities(supabase),
  getIncomeStreams(supabase),
  getGoals(supabase),
  getStrategicObjectives(supabase),
  getOperationalBaseline(supabase),
  getUserSettings(supabase, user.id),
]);

const deployments = (deploymentsData || []) as Deployment[];
const accounts = (accountsData || []) as Account[];
const liabilities = (liabilitiesData || []) as Liability[];
const incomeStreams = (incomeStreamsData || []) as IncomeStream[];
const goals = (goalsData || []) as FinancialGoal[];
const objectives = (objectivesData || []) as StrategicObjective[];
const baseline = (baselineData || []) as OperationalBaseline[];
const liquidity = Number(settings.total_liquidity);
const analytics = generateSummary(
  deployments,
  liquidity,
  accounts,
  liabilities,
  incomeStreams,
  goals,
  objectives,
  baseline,
);

const savedInsights = await getInsights(supabase, 1);
const previousInsight =
```


AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
    savedInsights && savedInsights.length > 0
      ? normalizeSavedInsight(savedInsights[0] as Record<string, unknown>)
      : null;

let kairosInsight: KairosInsight | null = null;

if (!options.forceInsightEvaluation && previousInsight) {
  // OPTIMIZATION SHORTCUT: Reuse cached strategic signal
  kairosInsight = previousInsight;
} else {
  // FRESH EVALUATION: Process authoritative telemetry
  kairosInsight = await generateKairosAIInsight({
    deployments,
    liquidity,
    previousInsight,
    objectives,
    accounts,
    liabilities,
    incomeStreams,
    goals,
    baseline,
  });

  // Only save if it's actually a material change
  if (kairosInsight.is_new_signal) {
    await saveInsight(supabase, kairosInsight, user.id);
  }
}

return {
  authenticated: true,
  deployments,
  accounts,
  liabilities,
  incomeStreams,
  goals,
  objectives,
  baseline,
  analytics,
  liquidity,
  kairosInsight,
};
}

export async function getDashboardSnapshotAction() {
  return buildDashboardSnapshot();
}
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
export async function checkOnboardingStatusAction() {
  const { userId, supabase } = await requireAuth();

  const [accounts, baseline] = await Promise.all([
    supabase
      .from("accounts")
      .select("id", { count: "exact", head: true })
      .eq("user_id", userId),
    supabase
      .from("operational_baseline")
      .select("id", { count: "exact", head: true })
      .eq("user_id", userId),
  ]);

  return {
    isOnboarded: (accounts.count ?? 0) > 0 || (baseline.count ?? 0) > 0,
  };
}

export async function createDeploymentAction(
  input: CreateDeploymentActionInput,
) {
  const { userId, supabase } = await requireAuth();

  try {
    // 1. Create the deployment record (The database trigger handles account balance deduction)
    await createDeployment(
      supabase,
      input.title,
      input.amount,
      userId,
      input.category,
      0,
      input.advancedContext,
      input.accountId,
    );

    // 2. Update global liquidity setting if this was a liquidity deployment
    if (input.accountId) {
      const settings = await getUserSettings(supabase, userId);
      const newLiquidity = Number(settings.total_liquidity) - input.amount;
      await updateLiquidity(supabase, userId, newLiquidity);
    }

    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to create deployment:", error);
  }
}
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
        throw error;
    }
}

export async function updateDeploymentAction(
    id: string,
    input: UpdateDeploymentActionInput,
) {
    const { userId, supabase } = await requireAuth();

    try {
        await updateDeployment(supabase, id, userId, {
            title: input.title,
            amount: input.amount,
            category: input.category,
        });
        revalidatePath("/dashboard");
        return buildDashboardSnapshot({ forceInsightEvaluation: true });
    } catch (error) {
        console.error("Failed to update deployment:", error);
        throw error;
    }
}

export async function deleteDeploymentAction(id: string) {
    const { userId, supabase } = await requireAuth();

    try {
        await deleteDeployment(supabase, id, userId);
        revalidatePath("/dashboard");
        return buildDashboardSnapshot({ forceInsightEvaluation: true });
    } catch (error) {
        console.error("Failed to delete deployment:", error);
        throw error;
    }
}

export async function updateLiquidityAction(amount: number) {
    const { userId, supabase } = await requireAuth();

    try {
        await updateLiquidity(supabase, userId, amount);
        revalidatePath("/dashboard");
        return buildDashboardSnapshot({ forceInsightEvaluation: true });
    } catch (error) {
        console.error("Failed to update liquidity:", error);
        throw error;
    }
}
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
}

export async function createAction(input: CreateAccountActionInput) {
  const { userId, supabase } = await requireAuth();

  try {
    await createAccount(supabase, userId, input);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to create account:", error);
    throw error;
  }
}

export async function updateAccountAction(
  id: string,
  input: Partial<CreateAccountActionInput>,
) {
  const { userId, supabase } = await requireAuth();

  try {
    await updateAccount(supabase, id, userId, input);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to update account:", error);
    throw error;
  }
}

export async function deleteAccountAction(id: string) {
  const { userId, supabase } = await requireAuth();

  try {
    await deleteAccount(supabase, id, userId);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to delete account:", error);
    throw error;
  }
}

export async function createLiabilityAction(input: CreateLiabilityActionInput) {
  const { userId, supabase } = await requireAuth();

  try {
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
    await createLiability(supabase, userId, input);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to create liability:", error);
    throw error;
  }
}

export async function updateLiabilityAction(
  id: string,
  input: Partial<CreateLiabilityActionInput>,
) {
  const { userId, supabase } = await requireAuth();

  try {
    await updateLiability(supabase, id, userId, input);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to update liability:", error);
    throw error;
  }
}

export async function deleteLiabilityAction(id: string) {
  const { userId, supabase } = await requireAuth();

  try {
    await deleteLiability(supabase, id, userId);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to delete liability:", error);
    throw error;
  }
}

export async function createIncomeAction(inputs: CreateIncomeActionInput[]) {
  const { userId, supabase } = await requireAuth();

  try {
    await createIncomeStreams(supabase, userId, inputs);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to create income streams:", error);
    throw error;
  }
}
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
    }
  }

export async function updateIncomeAction(
  id: string,
  input: Partial<CreateIncomeActionInput>,
) {
  const { userId, supabase } = await requireAuth();

  try {
    await updateIncomeStream(supabase, id, userId, input);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to update income stream:", error);
    throw error;
  }
}

export async function deleteIncomeAction(id: string) {
  const { userId, supabase } = await requireAuth();

  try {
    await deleteIncomeStream(supabase, id, userId);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to delete income stream:", error);
    throw error;
  }
}

export async function resolvePendingInflowAction(payload: {
  incomeId: string;
  accountId: string;
  amount: number;
}) {
  const { userId, supabase } = await requireAuth();

  try {
    // 1. Get current account balance
    const { data: account, error: accError } = await supabase
      .from("accounts")
      .select("current_balance")
      .eq("id", payload.accountId)
      .eq("user_id", userId)
      .single();
  }
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
if (accError || !account) throw new Error("Target account not found.");

// 2. Update the target account balance
const newBalance = Number(account.current_balance) + payload.amount;
await updateAccount(supabase, payload.accountId, userId, {
  current_balance: newBalance,
});

// 3. Update the income stream last_executed_at
await updateIncomeStream(supabase, payload.incomeId, userId, {
  last_executed_at: new Date().toISOString(),
});

// 4. Update global liquidity if this account is part of it
const settings = await getUserSettings(supabase, userId);
const newLiquidity = Number(settings.total_liquidity) + payload.amount;
await updateLiquidity(supabase, userId, newLiquidity);

revalidatePath("/dashboard");
return buildDashboardSnapshot({ forceInsightEvaluation: true });
} catch (error) {
  console.error("Failed to resolve pending income:", error);
  throw error;
}
}

export async function resolvePendingLiabilityAction(payload: {
  liabilityId: string;
  accountId: string;
  amount: number;
}) {
  const { userId, supabase } = await requireAuth();

  try {
    // 1. Get current account and liability state
    const [accountRes, liabRes] = await Promise.all([
      supabase
        .from("accounts")
        .select("current_balance")
        .eq("id", payload.accountId)
        .eq("user_id", userId)
        .single(),
      supabase
        .from("liabilities")
        .select("outstanding_balance")
        .eq("id", payload.liabilityId)
        .eq("user_id", userId)
        .single(),
    ]
  )
}
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
]);

if (accountRes.error || !accountRes.data)
  throw new Error("Source account not found.");
if (liabRes.error || !liabRes.data)
  throw new Error("Liability record not found.");

// 2. Update balances
const newAccBalance =
  Number(accountRes.data.current_balance) - payload.amount;
const newLiabBalance = Math.max(
  0,
  Number(liabRes.data.outstanding_balance) - payload.amount,
);

await Promise.all([
  updateAccount(supabase, payload.accountId, userId, {
    current_balance: newAccBalance,
  }),
  updateLiability(supabase, payload.liabilityId, userId, {
    outstanding_balance: newLiabBalance,
    last_executed_at: new Date().toISOString(),
  }),
]);

// 3. Update global liquidity
const settings = await getUserSettings(supabase, userId);
const newLiquidity = Number(settings.total_liquidity) - payload.amount;
await updateLiquidity(supabase, userId, newLiquidity);

revalidatePath("/dashboard");
return buildDashboardSnapshot({ forceInsightEvaluation: true });
} catch (error) {
  console.error("Failed to resolve pending liability:", error);
  throw error;
}
}

export async function resolvePendingBaselineAction(payload: {
  baselineId: string;
  accountId: string;
  amount: number;
  title: string;
}) {
  const { userId, supabase } = await requireAuth();

  try {
    // 1. Get current account balance
```


AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
const { data: account, error: accError } = await supabase
  .from("accounts")
  .select("current_balance")
  .eq("id", payload.accountId)
  .eq("user_id", userId)
  .single();

if (accError || !account) throw new Error("Source account not found.");

// 2. Deduct from account
const newBalance = Number(account.current_balance) - payload.amount;
await updateAccount(supabase, payload.accountId, userId, {
  current_balance: newBalance,
});

// 3. Log Deployment (Maintenance Expense)
await createDeployment(
  supabase,
  payload.title,
  payload.amount,
  userId,
  "Maintenance",
  0,
  undefined,
  payload.accountId,
);

// 4. Update the baseline last_executed_at
await updateOperationalBaseline(supabase, payload.baselineId, userId, {
  last_executed_at: new Date().toISOString(),
});

// 5. Update global liquidity
const settings = await getUserSettings(supabase, userId);
const newLiquidity = Number(settings.total_liquidity) - payload.amount;
await updateLiquidity(supabase, userId, newLiquidity);

revalidatePath("/dashboard");
return buildDashboardSnapshot({ forceInsightEvaluation: true });
} catch (error) {
  console.error("Failed to resolve pending baseline:", error);
  throw error;
}
}

export async function createGoalAction(input: CreateGoalActionInput) {
  const { userId, supabase } = await requireAuth();
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
try {
  await createGoal(supabase, userId, input);
  revalidatePath("/dashboard");
  return buildDashboardSnapshot({ forceInsightEvaluation: true });
} catch (error) {
  console.error("Failed to create goal:", error);
  throw error;
}
}

export async function updateGoalAction(
  id: string,
  input: Partial<CreateGoalActionInput>,
) {
  const { userId, supabase } = await requireAuth();

  try {
    await updateGoal(supabase, id, userId, input);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to update goal:", error);
    throw error;
  }
}

export async function deleteGoalAction(id: string) {
  const { userId, supabase } = await requireAuth();

  try {
    await deleteGoal(supabase, id, userId);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to delete goal:", error);
    throw error;
  }
}

export async function createStrategicObjectiveAction(
  input: CreateObjectiveActionInput,
) {
  const { userId, supabase } = await requireAuth();

  try {
    await createStrategicObjective(supabase, userId, input);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  }
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
    } catch (error) {
      console.error("Failed to create strategic objective:", error);
      throw error;
    }
  }

export const createObjectiveAction = createStrategicObjectiveAction;

export async function updateStrategicObjectiveAction(
  id: string,
  input: Partial<CreateObjectiveActionInput>,
) {
  const { userId, supabase } = await requireAuth();

  try {
    await updateStrategicObjective(supabase, id, userId, input);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to update strategic objective:", error);
    throw error;
  }
}

export const updateObjectiveAction = updateStrategicObjectiveAction;

export async function deleteStrategicObjectiveAction(id: string) {
  const { userId, supabase } = await requireAuth();

  try {
    await deleteStrategicObjective(supabase, id, userId);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to delete strategic objective:", error);
    throw error;
  }
}

export const deleteObjectiveAction = deleteStrategicObjectiveAction;

export async function createOperationalBaselineAction(
  data: Omit<
    OperationalBaseline,
    "id" | "user_id" | "created_at" | "updated_at"
  >,
) {
  const { userId, supabase } = await requireAuth();
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
try {
  await createOperationalBaseline(supabase, userId, data);
  revalidatePath("/dashboard");
  return buildDashboardSnapshot({ forceInsightEvaluation: true });
} catch (error) {
  console.error("Failed to create operational baseline:", error);
  throw error;
}
}

export async function updateOperationalBaselineAction(
  id: string,
  updates: Partial<
    Omit<OperationalBaseline, "id" | "user_id" | "created_at" | "updated_at">
  >,
) {
  const { userId, supabase } = await requireAuth();

  try {
    await updateOperationalBaseline(supabase, id, userId, updates);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to update operational baseline:", error);
    throw error;
  }
}

export async function deleteOperationalBaselineAction(id: string) {
  const { userId, supabase } = await requireAuth();

  try {
    await deleteOperationalBaseline(supabase, id, userId);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to delete operational baseline:", error);
    throw error;
  }
}

export async function submitDayZeroBaselineAction(payload: {
  accounts: {
    account_name: string;
    account_type: string;
    current_balance: number;
    institution?: string;
  }
}) {
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
  }[];
  incomes: {
    income_name: string;
    income_type: string;
    amount: number;
    cadence: string;
    execution_day?: number | null;
    is_recurring?: boolean;
    source?: string;
  }[];
  liabilities: {
    liability_name: string;
    liability_type: string;
    outstanding_balance: number;
    interest_rate?: number;
    minimum_payment?: number;
    institution?: string;
    is_paid_in_cadences?: boolean;
    cadence?: string | null;
    cadence_day_date?: string | null;
    cadence_amount?: number | null;
  }[];
  baselines: {
    title?: string;
    amount: number;
    cadence: string;
    execution_day?: number | null;
    is_recurring?: boolean;
    category?: string;
  }[];
}) {
  const { userId, supabase } = await requireAuth();

  try {
    // 1. Insert Accounts
    for (const acc of payload.accounts) {
      await createAccount(supabase, userId, acc);
    }

    // 2. Insert Incomes (if any)
    if (payload.incomes.length > 0) {
      await createIncomeStreams(
        supabase,
        userId,
        payload.incomes.map((inc) => ({
          ...inc,
          is_recurring: inc.is_recurring ?? true,
        })),
      ),
    }
  }
}
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
    );
  }

  // 3. Insert Liabilities (if any)
  for (const liab of payload.liabilities) {
    if (liab.outstanding_balance > 0) {
      await createLiability(supabase, userId, liab);
    }
  }

  // 4. Insert Operational Baselines
  for (const item of payload.baselines) {
    await createOperationalBaseline(supabase, userId, {
      title: item.title || "Core Survival Baseline",
      amount: item.amount,
      category: item.category || "Maintenance",
      cadence: item.cadence as BaselineCadence,
      execution_day: item.execution_day,
      is_recurring: item.is_recurring ?? true,
      baseline_type: "expense",
      is_active: true,
    });
  }

  // 5. Update initial liquidity pool to match sum of all accounts
  const totalLiquidity = payload.accounts.reduce(
    (sum, acc) => sum + acc.current_balance,
    0,
  );
  await updateLiquidity(supabase, userId, totalLiquidity);

  revalidatePath("/dashboard");
  return buildDashboardSnapshot({ forceInsightEvaluation: true });
} catch (error) {
  throw error;
}

export async function fetchHistoricalAuditAction() {
  const { userId, supabase } = await requireAuth();
  return getDeletedLedgerRecords(supabase, userId);
}

export interface AuditLog {
  id: string;
  rule_id: string;
  severity: string;
  evaluation_time_ms: number;
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
    created_at: string;
  }

export interface TelemetrySummary {
  logs: AuditLog[];
  averageLatency: number;
  matchRate: number;
  totalCycles: number;
  ruleHits: Record<string, number>;
}

export async function fetchTelemetryLogsAction(): Promise<TelemetrySummary> {
  const { userId, supabase } = await requireAuth();

  const { data, error } = await supabase
    .from("kairos_insights")
    .select("*")
    .eq("user_id", userId)
    .order("created_at", { ascending: false })
    .limit(50);

  if (error) {
    console.error("[Telemetry] Fetch failed:", error.message);
    return {
      logs: [],
      averageLatency: 0,
      matchRate: 0,
      totalCycles: 0,
      ruleHits: {},
    };
  }

  const logs = (data || []) as AuditLog[];
  const totalLogs = logs.length;

  if (totalLogs === 0) {
    return {
      logs: [],
      averageLatency: 0,
      matchRate: 0,
      totalCycles: 0,
      ruleHits: {},
    };
  }

  const totalLatency = logs.reduce(
    (sum, log) => sum + (log.evaluation_time_ms || 0),
    0,
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
);
const averageLatency = totalLatency / totalLogs;

const matches = logs.filter((log) => log.severity !== "none").length;
const matchRate = (matches / totalLogs) * 100;

const ruleHits: Record<string, number> = {};
logs.forEach((log) => {
  if (log.severity && log.severity !== "none") {
    ruleHits[log.rule_id] = (ruleHits[log.rule_id] || 0) + 1;
  }
});

return {
  logs,
  averageLatency: Math.round(averageLatency * 100) / 100,
  matchRate: Math.round(matchRate * 10) / 10,
  totalCycles: totalLogs,
  ruleHits,
};
}
```

FILE: src/components/dashboard/DayZeroOnboarding.tsx

```
"use client";

import { useState } from "react";
import {
  submitDayZeroBaselineAction,
  type DashboardSnapshot,
} from "@app/dashboard/actions";
import { BrandMark } from "@components/ui/brand-mark";
import { RippleButton } from "@components/ui/multi-type-ripple-buttons";
import { HoverButton } from "@components/ui/hover-glow-button";
import { LuxuryCard } from "@components/ui/luxury-card";
import { motion, AnimatePresence } from "framer-motion";
import { ACCOUNT_TYPES } from "@lib/finance/accounts";
import {
  INCOME_TYPES,
  CADENCES,
  type IncomeType,
  type Cadence,
} from "@lib/finance/income";
import { LIABILITY_TYPES } from "@lib/finance/liabilities";
import { TAXONOMY_CATEGORIES } from "@lib/finance/taxonomy";

interface DayZeroOnboardingProps {
  onComplete: (snapshot: DashboardSnapshot) => void;
```


AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
}
```

```
const BASELINE_CADENCES = [  
  { value: "daily", label: "Daily" },  
  { value: "weekly", label: "Weekly" },  
  { value: "monthly", label: "Monthly" },  
  { value: "yearly", label: "Yearly" },  
];
```

```
const STEPS = [  
  {  
    roman: "I",  
    title: "Accounts",  
    desc: "Every strong financial foundation begins with a clear map of your available capital. Please list the financial accounts that hold your primary source of liquidity (for example, corporate checking accounts or short-term cash reserves)",  
  },  
  {  
    roman: "II",  
    title: "Income Velocity",  
    desc: "Every strong financial foundation relies on a predictable cash flow. Please list your revenue-generating activities so we can calculate how quickly your funds are replenished (for example, recurring subscription fees, product sales, or monthly client retainers).",  
  },  
  {  
    roman: "III",  
    title: "Baseline Expenses",  
    desc: "Every strong financial foundation requires full visibility into its baseline expenses. Please list your fixed operating costs so we can identify and eliminate unnecessary spending (for example, office rent, software subscriptions, or employee salaries).",  
  },  
  {  
    roman: "IV",  
    title: "Financial Commitments",  
    desc: "Every strong financial foundation requires a clear view of what you owe. Please list your current debts and upcoming financial obligations so we can protect your long-term solvency (for example, short-term loans, credit card balances, or vendor invoices).",  
  },  
];
```

```
export default function DayZeroOnboarding({  
  onComplete,  
}: DayZeroOnboardingProps) {  
  const [step, setStep] = useState(1);  
  const [direction, setDirection] = useState(1);  
  const [isLoading, setIsLoading] = useState(false);  
  const [error, setError] = useState<string | null>(null);
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
const [accounts, setAccounts] = useState([
  {
    account_name: "",
    account_type: "mobile_money",
    current_balance: "",
    institution: "",
  },
]);
const [incomes, setIncomes] = useState<
  {
    income_name: string;
    income_type: string;
    amount: string;
    cadence: string;
    execution_day: string;
    is_recurring: boolean;
    source: string;
  }[]
>([]);
const [baselines, setBaselines] = useState([
  {
    title: "",
    amount: "",
    cadence: "monthly",
    execution_day: "1",
    is_recurring: true,
    category: "Maintenance",
  },
]);
const [liabilities, setLiabilities] = useState<
  {
    liability_name: string;
    liability_type: string;
    outstanding_balance: string;
    interest_rate: string;
    institution: string;
    is_paid_in_cadences: boolean;
    cadence: string;
    cadence_day_date: string;
    cadence_amount: string;
  }[]
>([]);

const addAccount = () => {
  if (accounts.length < 3)
    setAccounts([
      ...accounts,
      {

```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
        account_name: "",
        account_type: "checking",
        current_balance: "",
        institution: "",
    },
]);

};

const removeAccount = (index: number) =>
    setAccounts(accounts.filter((_, i) => i !== index));

const addIncome = () => {
    if (incomes.length < 3)
        setIncomes([
            ...incomes,
            {
                income_name: "",
                income_type: "salary",
                amount: "",
                cadence: "monthly",
                execution_day: "1",
                is_recurring: true,
                source: "",
            },
        ]);
};

const removeIncome = (index: number) =>
    setIncomes(incomes.filter((_, i) => i !== index));

const addBaseline = () => {
    if (baselines.length < 3)
        setBaselines([
            ...baselines,
            {
                title: "",
                amount: "",
                cadence: "monthly",
                execution_day: "1",
                is_recurring: true,
                category: "Maintenance",
            },
        ]);
};

const removeBaseline = (index: number) =>
    setBaselines(baselines.filter((_, i) => i !== index));

const addLiability = () => {
    if (liabilities.length < 2)
        setLiabilities([
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
...liabilities,
{
  liability_name: "",
  liability_type: "credit_card",
  outstanding_balance: "",
  interest_rate: "0",
  institution: "",
  is_paid_in_cadences: false,
  cadence: "monthly",
  cadence_day_date: "1",
  cadence_amount: "",
},
]);
};

const removeLiability = (index: number) =>
  setLiabilities(liabilities.filter((_, i) => i !== index));

const isStepValid = () => {
  if (step === 1)
    return (
      accounts.length > 0 &&
      accounts.every((a) => a.account_name.trim() && a.current_balance !== "")
    );
  if (step === 2)
    return (
      incomes.length === 0 ||
      incomes.every((i) => i.income_name.trim() && i.amount !== "")
    );
  if (step === 3)
    return (
      baselines.length > 0 &&
      baselines.every((b) => b.title.trim() && b.amount !== "")
    );
  if (step === 4)
    return (
      liabilities.length === 0 ||
      liabilities.every(
        (l) => l.liability_name.trim() && l.outstanding_balance !== "",
      )
    );
  return false;
};

const handleNext = () => {
  if (step < 4) {
    setDirection(1);
    setStep(step + 1);
    setError(null);
  }
}
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
    }
  };
  const handlePrev = () => {
    if (step > 1) {
      setDirection(-1);
      setStep(step - 1);
      setError(null);
    }
  };

  async function handleSubmit(e: React.FormEvent) {
    e.preventDefault();
    if (step < 4) {
      handleNext();
      return;
    }
    setIsLoading(true);
    setError(null);
    try {
      await submitDayZeroBaselineAction({
        accounts: accounts.map((a) => ({
          account_name: a.account_name,
          account_type: a.account_type,
          current_balance: Number(a.current_balance),
          institution: a.institution || undefined,
        })),
        incomes: incomes.map((i) => ({
          income_name: i.income_name,
          income_type: i.income_type,
          amount: Number(i.amount),
          cadence: i.is_recurring ? i.cadence : "irregular",
          execution_day:
            i.is_recurring &&
            (i.cadence === "monthly" ||
             i.cadence === "weekly" ||
             i.cadence === "biweekly")
              ? Number(i.execution_day)
              : null,
          is_recurring: i.is_recurring,
          source: i.source || undefined,
        })),
        liabilities: liabilities
          .filter((l) => Number(l.outstanding_balance) > 0)
          .map((l) => ({
            liability_name: l.liability_name,
            liability_type: l.liability_type,
            outstanding_balance: Number(l.outstanding_balance),
            interest_rate: Number(l.interest_rate) || 0,
          })),
      });
    } catch (error) {
      console.error("handleSubmit error:", error);
      setError(error.message);
      setIsLoading(false);
    }
  }
}
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```

        institution: l.institution || undefined,
        is_paid_in_cadences: l.is_paid_in_cadences,
        cadence: l.is_paid_in_cadences ? l.cadence : null,
        cadence_day_date: l.is_paid_in_cadences
          ? String(l.cadence_day_date)
          : null,
        cadence_amount: l.is_paid_in_cadences
          ? Number(l.cadence_amount)
          : null,
      })),
    baselines: baselines.map((b) => ({
      title: b.title,
      amount: Number(b.amount),
      cadence: b.is_recurring ? b.cadence : "monthly",
      execution_day:
        b.is_recurring &&
        (b.cadence === "monthly" ||
         b.cadence === "weekly" ||
         b.cadence === "biweekly")
          ? Number(b.execution_day)
          : null,
      is_recurring: b.is_recurring,
      category: b.category,
    })),
  });
window.location.reload();
} catch (err: unknown) {
  setError(
    err instanceof Error
      ? err.message
      : "An unexpected error occurred. Please verify your inputs.",
  );
} finally {
  setIsLoading(false);
}
}

const activeStep = STEPS[step - 1];
const backgroundX = -(step - 1) * 5;

return (
  <div className="fixed inset-0 bg-black z-100 flex flex-col selection:bg-white selection:text-black
overflow-hidden h-screen w-full font-sans text-white">
    <div className="absolute inset-0 z-0 pointer-events-none overflow-hidden">
      <motion.div
        animate={{ x: `${backgroundX}%` }}
        transition={{ duration: 1.2, ease: [0.215, 0.61, 0.355, 1] }}
        className="absolute inset-0 w-[150%] h-full"

```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
>
  <div className="absolute top-[-10%] left-[-10%] w-[40%] h-[40%] bg-white/2 rounded-full blur-[120px]"
/>
  <div className="absolute bottom-[-10%] right-[30%] w-[40%] h-[40%] bg-white/2 rounded-full
blur-[120px]" />
    <div
      className="absolute inset-0 opacity-15 mix-blend-screen grayscale"
      style={{
        backgroundImage: "url('/images/tactical-schematic.png')",
        backgroundSize: "cover",
        backgroundPosition: "left center",
      }}
    />
  </motion.div>
</div>

<nav className="absolute top-0 left-0 w-full z-50 px-12 py-10 flex justify-between items-center
mix-blend-difference pointer-events-none">
  <div className="flex items-center gap-6 group cursor-pointer pointer-events-auto">
    <BrandMark className="w-10 h-10" />
    <div className="flex flex-col">
      <span className="font-mono text-[10px] tracking-[0.6em] uppercase font-bold text-white">
        AXIOM
      </span>
      <span className="text-[8px] font-mono tracking-[0.4em] uppercase text-white/40">
        Architecture
      </span>
    </div>
  </div>
  <div className="flex gap-12 items-center pointer-events-auto">
    <div className="hidden md:flex gap-8">
      {STEPS.map((s, idx) => (
        <div
          key={s.roman}
          className="flex flex-col items-center gap-2 text-white"
        >
          <span
            className={`font-mono text-[9px] tracking-widest transition-colors duration-700 ${idx + 1 ===
step ? "opacity-100" : "opacity-10"}`}
          >
            {s.roman}
          </span>
          <motion.div
            animate={{ scaleX: idx + 1 === step ? 1 : 0 }}
            className="h-px w-4 bg-white origin-left"
          />
        </div>
      ))}
    </div>
  </div>
</nav>
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
</div>
<span className="font-mono text-[9px] tracking-[0.4em] uppercase text-white/20">
  v1.0
</span>
</div>
</nav>

<div className="relative z-10 flex-1 flex items-center justify-center px-8 md:px-12 pt-16">
  <div className="max-w-7xl w-full h-155 grid lg:grid-cols-[1.1fr_1.4fr] gap-16 md:gap-24 items-stretch overflow-visible">
    <div className="flex flex-col justify-center h-full border-r border-white/5 pr-12 text-white overflow-hidden">
      <AnimatePresence mode="wait" custom={direction}>
        <motion.div
          key={step}
          custom={direction}
          initial={{ opacity: 0, x: direction * 50 }}
          animate={{ opacity: 1, x: 0 }}
          exit={{ opacity: 0, x: -direction * 50 }}
          transition={{ duration: 0.8, ease: [0.215, 0.61, 0.355, 1] }}
          className="space-y-6"
        >
          <div className="space-y-2">
            <div className="font-cormorant italic text-8xl text-white/5 leading-none select-none">
              {activeStep.roman}
            </div>
            <h1 className="font-cormorant text-6xl text-white leading-none tracking-tight">
              {activeStep.title}
            </h1>
          </div>
          <p className="text-white/40 text-lg font-light leading-relaxed">
            {activeStep.desc}
          </p>
        </motion.div>
      </AnimatePresence>
    </div>

    <LuxuryCard className="h-full flex flex-col border border-white/10 shadow-[0_0_80px_rgba(0,0,0,0.8)] overflow-hidden">
      <div className="flex-1 p-10 md:p-14 overflow-hidden relative rounded-t-[inherit]">
        <form
          onSubmit={handleSubmit}
          className="h-full flex flex-col text-white"
        >
          <AnimatePresence mode="wait" custom={direction}>
            <motion.div
              key={step}
              custom={direction}
```


AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
initial={{ opacity: 0, x: direction * 30 }}
animate={{ opacity: 1, x: 0 }}
exit={{ opacity: 0, x: -direction * 30 }}
transition={{ duration: 0.6, ease: "easeOut" }}
className="flex-1 flex flex-col"
>
{step === 1 && (
  <div className="flex flex-col h-full gap-6">
    <h2 className="font-cormorant text-2xl text-white tracking-wide uppercase">
      Account Name
    </h2>
    <div className="flex-1 space-y-4 overflow-y-auto scrollbar-hide pr-2">
      {accounts.map((acc, idx) => (
        <div
          key={idx}
          className="space-y-3 pb-3 border-b border-white/5 relative shrink-0"
        >
          <div className="grid grid-cols-2 gap-6">
            <input
              type="text"
              placeholder="e.g. M-Pesa, Crypto Wallet"
              value={acc.account_name}
              onChange={(e) => {
                const n = [...accounts];
                n[idx].account_name = e.target.value;
                setAccounts(n);
              }}
              className="bg-transparent border-b border-white/10 py-1 font-cormorant
text-2xl text-white focus:outline-none placeholder:text-white/5"
            />
            <input
              type="text"
              placeholder="Financial Institution (optional)"
              value={acc.institution}
              onChange={(e) => {
                const n = [...accounts];
                n[idx].institution = e.target.value;
                setAccounts(n);
              }}
              className="bg-transparent border-b border-white/10 py-1 text-sm font-light
text-white/60 focus:outline-none placeholder:text-white/5"
            />
          </div>
          <div className="grid grid-cols-2 gap-8">
            <div className="space-y-1">
              <label className="text-[9px] font-mono text-white/20 uppercase
tracking-[0.4em]">
                Account Type
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
</label>
<select
  value={acc.account_type}
  onChange={(e) => {
    const n = [...accounts];
    n[idx].account_type = e.target.value;
    setAccounts(n);
  }}
  className="w-full bg-transparent border-b border-white/10 py-1 text-[10px]
font-mono tracking-widest uppercase text-white/40 focus:outline-none"
>
  {ACCOUNT_TYPES.map((t) => (
    <option
      key={t.value}
      value={t.value}
      className="bg-[#080808]"
    >
      {t.label}
    </option>
  ))}
</select>
</div>
<div className="space-y-1">
  <label className="text-[9px] font-mono text-white/20 uppercase
tracking-[0.4em]">

    Current Liquidity
  </label>
  <input
    type="number"
    placeholder="0.00"
    value={acc.current_balance}
    onChange={(e) => {
      const n = [...accounts];
      n[idx].current_balance = e.target.value;
      setAccounts(n);
    }}
    className="w-full bg-transparent border-b border-white/10 py-1 text-xl
font-light text-white focus:outline-none tabular-nums"
  />
</div>
</div>
{accounts.length > 1 && (
  <button
    type="button"
    onClick={() => removeAccount(idx)}
    className="absolute top-0 right-0 p-2 text-white/10 hover:text-red-400
bg-white/5 rounded-full hover:bg-white/10 transition-all"
  >
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
        ?
      </button>
    })
  </div>
  )))
</div>
<button
  type="button"
  onClick={addAccount}
  className="text-[9px] font-mono tracking-[0.6em] uppercase text-white/30
hover:text-white transition-all py-3 px-6 border border-white/10 rounded-full self-start active:scale-95"
  >
    + Append Source
  </button>
</div>
)})

{step === 2 && (
  <div className="flex flex-col h-full gap-6">
    <h2 className="font-cormorant text-2xl text-white tracking-wide uppercase">
      Income Velocity
    </h2>
    <div className="flex-1 space-y-4 overflow-y-auto scrollbar-hide pr-2">
      {incomes.map((inc, idx) => (
        <div
          key={idx}
          className="space-y-3 pb-3 border-b border-white/5 relative shrink-0"
          >
          <div className="grid grid-cols-2 gap-6">
            <input
              type="text"
              placeholder="e.g. Salary"
              value={inc.income_name}
              onChange={(e) => {
                const n = [...incomes];
                n[idx].income_name = e.target.value;
                setIncomes(n);
              }}
              className="bg-transparent border-b border-white/10 py-1 font-cormorant
text-2xl text-white focus:outline-none placeholder:text-white/5"
            />
            <input
              type="text"
              placeholder="Origin"
              value={inc.source}
              onChange={(e) => {
                const n = [...incomes];
                n[idx].source = e.target.value;
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
        setIncomes(n);
      }}
      className="bg-transparent border-b border-white/10 py-1 text-sm font-light
text-white/60 focus:outline-none placeholder:text-white/5"
    />
  </div>
  <div className="grid grid-cols-2 gap-6">
    <div className="space-y-1">
      <label className="text-[9px] font-mono text-white/20 uppercase
tracking-[0.4em]">

        Type
      </label>
      <select
        value={inc.income_type}
        onChange={(e) => {
          const n = [...incomes];
          n[idx].income_type = e.target.value;
          setIncomes(n);
        }}
        className="w-full bg-transparent border-b border-white/10 py-1 text-[10px]
font-mono tracking-widest uppercase text-white/40 focus:outline-none"
      >
        {INCOME_TYPES.map((t) => (
          <option
            key={t.value}
            value={t.value}
            className="bg-[#080808]"
          >
            {t.label}
          </option>
        ))}
      </select>
    </div>
    <div className="space-y-1">
      <label className="text-[9px] font-mono text-white/20 uppercase
tracking-[0.4em]">

        KES
      </label>
      <input
        type="number"
        placeholder="0.00"
        value={inc.amount}
        onChange={(e) => {
          const n = [...incomes];
          n[idx].amount = e.target.value;
          setIncomes(n);
        }}
        className="w-full bg-transparent border-b border-white/10 py-1 text-xl
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
font-light text-white focus:outline-none tabular-nums"
    />
  </div>
</div>

<div className="pt-2 space-y-3">
  <label className="flex items-center space-x-3 cursor-pointer group">
    <input
      type="checkbox"
      checked={inc.is_recurring}
      onChange={(e) => {
        const n = [...incomes];
        n[idx].is_recurring = e.target.checked;
        setIncomes(n);
      }}
      className="w-4 h-4 rounded border-white/10 bg-transparent checked:bg-white
transition-colors cursor-pointer"
    />
    <span className="text-[10px] font-mono text-white/30 uppercase
tracking-widest group-hover:text-white/60 transition-colors">
      Recurring Income
    </span>
  </label>
  {inc.is_recurring && (
    <motion.div
      initial={{ opacity: 0, y: -5 }}
      animate={{ opacity: 1, y: 0 }}
      className="grid grid-cols-2 gap-6 pt-2 pl-8 border-l border-white/5"
    >
      <div className="space-y-1">
        <label className="text-[9px] font-mono text-white/20 uppercase">
          Frequency
        </label>
        <select
          value={inc.cadence}
          onChange={(e) => {
            const n = [...incomes];
            n[idx].cadence = e.target.value;
            setIncomes(n);
          }}
          className="w-full bg-transparent border-b border-white/10 py-1
text-[10px] font-mono text-white/60 focus:outline-none"
        >
          {CADENCES.map((c) => (
            <option
              key={c.value}
              value={c.value}
              className="bg-black"
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
>
    {c.label}
  </option>
  )})
</select>
</div>
{(inc.cadence === "monthly" ||
inc.cadence === "weekly" ||
inc.cadence === "biweekly") && (
<div className="space-y-1">
  <label className="text-[9px] font-mono text-white/20 uppercase">
    {inc.cadence === "monthly"
    ? "Day"
    : "Day of Week"}
  </label>
  {inc.cadence === "monthly" ? (
    <input
      type="number"
      min="1"
      max="31"
      placeholder="15"
      value={inc.execution_day || ""}
      onChange={(e) => {
        const n = [...incomes];
        n[idx].execution_day =
          e.target.value;
        setIncomes(n);
      }}
      className="w-full bg-transparent border-b border-white/10 py-1
text-[10px] font-mono text-white/60 focus:outline-none"
    />
  ) : (
    <select
      value={inc.execution_day || 1}
      onChange={(e) => {
        const n = [...incomes];
        n[idx].execution_day =
          e.target.value;
        setIncomes(n);
      }}
      className="w-full bg-transparent border-b border-white/10 py-1
text-[10px] font-mono text-white/60 focus:outline-none"
    >
    {[
      { label: "Mon", value: 1 },
      { label: "Tue", value: 2 },
      { label: "Wed", value: 3 },
      { label: "Thu", value: 4 },
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
        { label: "Fri", value: 5 },
        { label: "Sat", value: 6 },
        { label: "Sun", value: 7 },
      ].map((day) => (
        <option
          key={day.value}
          value={day.value}
          className="bg-black"
        >
          {day.label}
        </option>
      )))
    </select>
  )}
</div>
)}
</motion.div>
)}
</div>
<button
  type="button"
  onClick={() => removeIncome(idx)}
  className="absolute top-0 right-0 p-2 text-white/10 hover:text-red-400
bg-white/5 rounded-full hover:bg-white/10 transition-all"
>
  ?
</button>
</div>
)})
</div>
<button
  type="button"
  onClick={addIncome}
  className="text-[9px] font-mono tracking-[0.6em] uppercase text-white/30
hover:text-white transition-all py-3 px-6 border border-white/10 rounded-full self-start active:scale-95"
>
  + Append Income
</button>
</div>
)}

{step === 3 && (
  <div className="flex flex-col h-full gap-6">
    <h2 className="font-cormorant text-2xl text-white tracking-wide uppercase">
      Expense
    </h2>
    <div className="flex-1 space-y-4 overflow-y-auto scrollbar-hide pr-2">
      {baselines.map((base, idx) => (
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
<div
  key={idx}
  className="space-y-3 pb-3 border-b border-white/5 relative shrink-0"
>
  <div className="grid grid-cols-2 gap-6">
    <input
      type="text"
      placeholder="Baseline Name"
      value={base.title}
      onChange={(e) => {
        const n = [...baselines];
        n[idx].title = e.target.value;
        setBaselines(n);
      }}
      className="bg-transparent border-b border-white/10 py-1 font-cormorant
text-2xl text-white focus:outline-none placeholder:text-white/5"
    />
    <div className="space-y-1">
      <label className="text-[9px] font-mono text-white/20 uppercase
tracking-[0.4em]">
        Category
      </label>
      <select
        value={base.category}
        onChange={(e) => {
          const n = [...baselines];
          n[idx].category = e.target.value;
          setBaselines(n);
        }}
        className="bg-transparent border-b border-white/10 py-1 text-[10px]
font-mono tracking-widest uppercase text-white/40 focus:outline-none w-full"
      >
        {TAXONOMY_CATEGORIES.map((cat) => (
          <option
            key={cat.value}
            value={cat.value}
            className="bg-[#080808]"
          >
            {cat.label}
          </option>
        ))}
      </select>
    </div>
  </div>
  <div className="grid grid-cols-2 gap-12">
    <div className="space-y-1">
      <label className="text-[9px] font-mono text-white/20 uppercase
tracking-[0.4em]">
```


AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```

        Amount
      </label>
      <input
        type="number"
        placeholder="0.00"
        value={base.amount}
        onChange={(e) => {
          const n = [...baselines];
          n[idx].amount = e.target.value;
          setBaselines(n);
        }}
        className="w-full bg-transparent border-b border-white/10 py-1 text-xl
font-light text-white focus:outline-none tabular-nums"
      />
    </div>
  </div>

  <div className="pt-2 space-y-3">
    <label className="flex items-center space-x-3 cursor-pointer group">
      <input
        type="checkbox"
        checked={base.is_recurring}
        onChange={(e) => {
          const n = [...baselines];
          n[idx].is_recurring = e.target.checked;
          setBaselines(n);
        }}
        className="w-4 h-4 rounded border-white/10 bg-transparent checked:bg-white
transition-colors cursor-pointer"
      />
      <span className="text-[10px] font-mono text-white/30 uppercase
tracking-widest group-hover:text-white/60 transition-colors">
        Recurring Expense
      </span>
    </label>
    {base.is_recurring && (
      <motion.div
        initial={{ opacity: 0, y: -5 }}
        animate={{ opacity: 1, y: 0 }}
        className="grid grid-cols-2 gap-6 pt-2 pl-8 border-l border-white/5"
      >
        <div className="space-y-1">
          <label className="text-[9px] font-mono text-white/20 uppercase">
            Frequency
          </label>
          <select
            value={base.cadence}
            onChange={(e) => {
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
const n = [...baselines];
n[idx].cadence = e.target.value;
setBaselines(n);
}}

className="w-full bg-transparent border-b border-white/10 py-1
text-[10px] font-mono text-white/60 focus:outline-none"
>
{BASELINE_CADENCES.map((c) => (
  <option
    key={c.value}
    value={c.value}
    className="bg-black"
  >
    {c.label}
  </option>
)}}
</select>
</div>

{base.cadence === "daily" && (
  <div className="flex items-center pt-2">
    <p className="text-[8px] font-mono text-white/20 uppercase
leading-tight">

      Prompted everyday to verify.
    </p>
  </div>
)}

{(base.cadence === "monthly" ||
base.cadence === "weekly" ||
base.cadence === "biweekly") && (
  <div className="space-y-1">
    <label className="text-[9px] font-mono text-white/20 uppercase">
      {base.cadence === "monthly"
        ? "Day"
        : "Day of Week"}
    </label>
    {base.cadence === "monthly" ? (
      <input
        type="number"
        min="1"
        max="31"
        placeholder="15"
        value={base.execution_day || ""}
        onChange={(e) => {
          const n = [...baselines];
          n[idx].execution_day =
            e.target.value;
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
        setBaselines(n);
      }}
      className="w-full bg-transparent border-b border-white/10 py-1
text-[10px] font-mono text-white/60 focus:outline-none"
    />
  ) : (
    <select
      value={base.execution_day || 1}
      onChange={(e) => {
        const n = [...baselines];
        n[idx].execution_day =
          e.target.value;
        setBaselines(n);
      }}
      className="w-full bg-transparent border-b border-white/10 py-1
text-[10px] font-mono text-white/60 focus:outline-none"
    >
      {[
        { label: "Mon", value: 1 },
        { label: "Tue", value: 2 },
        { label: "Wed", value: 3 },
        { label: "Thu", value: 4 },
        { label: "Fri", value: 5 },
        { label: "Sat", value: 6 },
        { label: "Sun", value: 7 },
      ]}.map((day) => (
        <option
          key={day.value}
          value={day.value}
          className="bg-black"
        >
          {day.label}
        </option>
      )))
    </select>
  )}
</div>
)}
</motion.div>
)}
</div>
{baselines.length > 1 && (
  <button
    type="button"
    onClick={() => removeBaseline(idx)}
    className="absolute top-0 right-0 p-2 text-white/10 hover:text-red-400
bg-white/5 rounded-full hover:bg-white/10 transition-all"
  >
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
        ?
      </button>
    })
  </div>
  )))
</div>
<button
  type="button"
  onClick={addBaseline}
  className="text-[9px] font-mono tracking-[0.6em] uppercase text-white/30
hover:text-white transition-all py-3 px-6 border border-white/10 rounded-full self-start active:scale-95"
  >
    + Append Recurring
  </button>
</div>
)})

{step === 4 && (
  <div className="flex flex-col h-full gap-6">
    <h2 className="font-cormorant text-2xl text-white tracking-wide uppercase">
      Financial Commitment
    </h2>
    <div className="flex-1 space-y-4 overflow-y-auto scrollbar-hide pr-2">
      {liabilities.map((liab, idx) => (
        <div
          key={idx}
          className="space-y-3 pb-3 border-b border-white/5 relative shrink-0"
          >
          <div className="grid grid-cols-2 gap-6">
            <input
              type="text"
              placeholder="Obligation Name"
              value={liab.liability_name}
              onChange={(e) => {
                const n = [...liabilities];
                n[idx].liability_name = e.target.value;
                setLiabilities(n);
              }}
              className="bg-transparent border-b border-white/10 py-1 font-cormorant
text-2xl text-white focus:outline-none placeholder:text-white/5"
            />
            <input
              type="text"
              placeholder="Financial Institution"
              value={liab.institution}
              onChange={(e) => {
                const n = [...liabilities];
                n[idx].institution = e.target.value;
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
        setLiabilities(n);
    }}
        className="bg-transparent border-b border-white/10 py-1 text-sm font-light
text-white/60 focus:outline-none placeholder:text-white/5"
    />
</div>
<div className="grid grid-cols-3 gap-6">
    <div className="space-y-1">
        <label className="text-[9px] font-mono text-white/20 uppercase
tracking-[0.4em]">

            Type
        </label>
        <select
            value={liab.liability_type}
            onChange={(e) => {
                const n = [...liabilities];
                n[idx].liability_type = e.target.value;
                setLiabilities(n);
            }}
            className="w-full bg-transparent border-b border-white/10 py-1 text-[11px]
font-mono tracking-widest uppercase text-white/40 focus:outline-none"
        >
            {LIABILITY_TYPES.map((t) => (
                <option
                    key={t.value}
                    value={t.value}
                    className="bg-[#080808]"
                >
                    {t.label}
                </option>
            ))}
        </select>
    </div>
    <div className="space-y-1">
        <label className="text-[9px] font-mono text-white/20 uppercase
tracking-[0.4em]">

            Balance
        </label>
        <input
            type="number"
            placeholder="0"
            value={liab.outstanding_balance}
            onChange={(e) => {
                const n = [...liabilities];
                n[idx].outstanding_balance =
                    e.target.value;
                setLiabilities(n);
            }}
        />
    </div>
</div>
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```

        className="w-full bg-transparent border-b border-white/10 py-1 text-xl
font-light text-white focus:outline-none tabular-nums"
    />
</div>
<div className="space-y-1">
    <label className="text-[9px] font-mono text-white/20 uppercase
tracking-[0.4em]">

        Interest %
    </label>
    <input
        type="number"
        step="0.1"
        placeholder="0"
        value={liab.interest_rate}
        onChange={(e) => {
            const n = [...liabilities];
            n[idx].interest_rate = e.target.value;
            setLiabilities(n);
        }}
        className="w-full bg-transparent border-b border-white/10 py-1 text-xl
font-light text-white focus:outline-none tabular-nums"
    />
</div>
</div>

<div className="pt-2 space-y-3">
    <label className="flex items-center space-x-3 cursor-pointer group">
        <input
            type="checkbox"
            checked={liab.is_paid_in_cadences}
            onChange={(e) => {
                const n = [...liabilities];
                n[idx].is_paid_in_cadences =
                    e.target.checked;
                setLiabilities(n);
            }}
            className="w-4 h-4 rounded border-white/10 bg-transparent checked:bg-white
transition-colors cursor-pointer"
        />

        <span className="text-[10px] font-mono text-white/30 uppercase
tracking-widest group-hover:text-white/60 transition-colors">

            Paid in Cadence
        </span>
    </label>
    {liab.is_paid_in_cadences && (
        <motion.div
            initial={{ opacity: 0, y: -5 }}
            animate={{ opacity: 1, y: 0 }}

```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
      className="grid grid-cols-3 gap-6 pt-2 pl-8 border-l border-white/5"
    >
    <div className="space-y-1">
      <label className="text-[9px] font-mono text-white/20 uppercase">
        Cadence
      </label>
      <select
        value={liab.cadence}
        onChange={(e) => {
          const n = [...liabilities];
          n[idx].cadence = e.target.value;
          setLiabilities(n);
        }}
        className="w-full bg-transparent border-b border-white/10 py-1
text-[10px] font-mono text-white/60 focus:outline-none"
      >
        <option
          value="weekly"
          className="bg-black"
        >
          Weekly
        </option>
        <option
          value="monthly"
          className="bg-black"
        >
          Monthly
        </option>
      </select>
    </div>
    <div className="space-y-1">
      <label className="text-[9px] font-mono text-white/20 uppercase">
        {liab.cadence === "weekly"
          ? "Day of Week"
          : "Day of Month"}
      </label>
      {liab.cadence === "monthly" ? (
        <input
          type="number"
          min="1"
          max="31"
          placeholder="15"
          value={liab.cadence_day_date || ""}
          onChange={(e) => {
            const n = [...liabilities];
            n[idx].cadence_day_date =
              e.target.value;
            setLiabilities(n);
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
    }}
    className="w-full bg-transparent border-b border-white/10 py-1
text-[10px] font-mono text-white/60 focus:outline-none"
  />
) : (
  <select
    value={
      liab.cadence_day_date || "Monday"
    }
    onChange={(e) => {
      const n = [...liabilities];
      n[idx].cadence_day_date =
        e.target.value;
      setLiabilities(n);
    }}
    className="w-full bg-transparent border-b border-white/10 py-1
text-[10px] font-mono text-white/60 focus:outline-none"
    >
    {[
      "Monday",
      "Tuesday",
      "Wednesday",
      "Thursday",
      "Friday",
      "Saturday",
      "Sunday",
    ].map((day) => (
      <option
        key={day}
        value={day}
        className="bg-black"
      >
        {day}
      </option>
    )))
  </select>
)}
</div>
<div className="space-y-1">
  <label className="text-[9px] font-mono text-white/20 uppercase">
    Payment
  </label>
  <input
    type="number"
    placeholder="0"
    value={liab.cadence_amount || ""}
    onChange={(e) => {
      const n = [...liabilities];
```


AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```

        n[idx].cadence_amount =
            e.target.value;
        setLiabilities(n);
    }}

    className="w-full bg-transparent border-b border-white/10 py-1
text-[10px] font-mono text-white/60 focus:outline-none"
    />
  </div>
</motion.div>

  )}
</div>
<button
  type="button"
  onClick={() => removeLiability(idx)}
  className="absolute top-0 right-0 p-2 text-white/10 hover:text-red-400
bg-white/5 rounded-full hover:bg-white/10 transition-all"
  >
    ?
  </button>
</div>
  )}}
</div>
<button
  type="button"
  onClick={addLiability}
  className="text-[11px] font-mono tracking-[0.6em] uppercase text-white/30
hover:text-white transition-all py-4 px-8 border border-white/10 rounded-full self-start active:scale-95"
  >
    + Append Obligation
  </button>
</div>
  )}
</motion.div>
</AnimatePresence>
</form>
</div>

  <div className="h-32 border-t border-white/5 relative z-10 px-12 md:px-16 flex flex-col
justify-center shrink-0 bg-[#080808]/60 backdrop-blur-3xl rounded-b-[inherit]">
    {error && (
      <div className="absolute top-0 left-0 w-full transform -translate-y-full px-12 pb-4">
        <div className="bg-red-500/10 border border-red-500/20 rounded-lg p-3 text-red-400
text-[10px] font-mono tracking-wider flex items-center gap-3">
          <span className="shrink-0 w-1.5 h-1.5 rounded-full bg-red-500 animate-pulse" />
          {error}
        </div>
      </div>
    )}
  </div>
  )}
  )}
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```

<div className="w-full flex items-center justify-between">
  <button
    type="button"
    onClick={handlePrev}
    disabled={step === 1}
    className="text-[11px] font-mono tracking-[0.6em] text-white/20 hover:text-white
transition-all uppercase disabled:opacity-0 py-8 px-12 -ml-12 hover:bg-white/5 rounded-full"
  >
    ? Back
  </button>
  <div onClick={handleSubmit}>
    {step === 4 ? (
      <HoverButton
        glowColor="rgba(255,255,255,0.4)"
        className="px-24 py-10 rounded-full text-[11px]"
        disabled={isLoading || !isStepValid()}
      >
        {isLoading ? "... " : "Archive Setup"}
      </HoverButton>
    ) : (
      <RippleButton
        variant="hoverborder"
        className="px-24 py-10 border-none text-[11px]"
        disabled={isLoading || !isStepValid()}
      >
        Next Phase ?
      </RippleButton>
    )}
  </div>
</div>
</div>
</LuxuryCard>
</div>
</div>
</div>
);
}

```

FILE: src/app/dashboard\AccountSection.tsx

```
"use client";

import { useState, useSyncExternalStore } from "react";
import { ACCOUNT_TYPES, type Account } from "@lib/finance/accounts";
import { createAccountAction, type DashboardSnapshot } from "./actions";
import { formatCurrency } from "@lib/utils/formatters";
import { AccountMap } from "@lib/utils/taxonomy";
import { DistributionPieChart } from "@components/dashboard/MiniCharts";
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
import { ScrollReveal } from "@components/ui/scroll-reveal";

interface AccountSectionProps {
  accounts: Account[];
  onSnapshot: (snapshot: DashboardSnapshot) => void;
}

const emptySubscribe = () => () => {};

export function AccountSection({ accounts, onSnapshot }: AccountSectionProps) {
  const isClient = useSyncExternalStore(
    emptySubscribe,
    () => true,
    () => false,
  );

  const [isAdding, setIsAdding] = useState(false);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const [sourceForm, setSourceForm] = useState({
    account_name: "",
    account_type: "checking",
    current_balance: "",
    institution: "",
  });

  const chartData = accounts.map((a) => ({
    name: a.account_name,
    value: Number(a.current_balance),
  }));

  async function handleSourceSubmit(e: React.FormEvent) {
    e.preventDefault();
    setIsLoading(true);
    setError(null);
    try {
      const snapshot = await createAction({
        account_name: sourceForm.account_name,
        account_type: sourceForm.account_type,
        current_balance: Number(sourceForm.current_balance),
        institution: sourceForm.institution || undefined,
      });
      setSourceForm({
        account_name: "",
        account_type: "checking",
        current_balance: "",
        institution: "",
      });
    } catch (error) {
      setError(error.message);
    }
  }
}
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
    });
    setIsAdding(false);
    onSnapshot(snapshot);
  } catch (err: unknown) {
    setError(err instanceof Error ? err.message : "Failed to create source");
  } finally {
    setIsLoading(false);
  }
}

if (!isClient) return null;

return (
  <div className="space-y-12">
    <div className="flex items-center justify-between">
      <h2 className="font-cormorant text-2xl text-white tracking-wide uppercase">
        Accounts
      </h2>
      <button
        onClick={() => setIsAdding(!isAdding)}
        className="font-mono text-[9px] tracking-[0.4em] uppercase text-white/40 hover:text-white
transition-colors"
      >
        {isAdding ? "? CANCEL" : "+ APPEND ACCOUNT"}
      </button>
    </div>

    {isAdding && (
      <div className="bg-white/2 border border-white/5 p-8 animate-in fade-in slide-in-from-top-4
duration-500">
        <form onSubmit={handleSourceSubmit} className="space-y-8">
          <div className="grid grid-cols-1 md:grid-cols-2 gap-8">
            <div className="space-y-2">
              <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
                Account Name
              </label>
              <input
                type="text"
                required
                placeholder="e.g. M-Pesa, Savings"
                value={sourceForm.account_name}
                onChange={(e) =>
                  setSourceForm({
                    ...sourceForm,
                    account_name: e.target.value,
                  })
                }
              >
                <div className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
focus:outline-none focus:border-white transition-colors"
    />
  </div>
  <div className="space-y-2">
    <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
      Current Liquidity
    </label>
    <input
      type="number"
      required
      placeholder="0.00"
      value={sourceForm.current_balance}
      onChange={(e) =>
        setSourceForm({
          ...sourceForm,
          current_balance: e.target.value,
        })
      }
      className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light
focus:outline-none focus:border-white transition-colors"
    />
  </div>
</div>

<div className="grid grid-cols-1 md:grid-cols-2 gap-8">
  <div className="space-y-2">
    <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
      Account Type
    </label>
    <select
      value={sourceForm.account_type}
      onChange={(e) =>
        setSourceForm({
          ...sourceForm,
          account_type: e.target.value,
        })
      }
      className="w-full bg-transparent border-b border-white/10 py-3 text-white/60 font-mono
text-[10px] tracking-widest uppercase focus:outline-none"
    >
      {ACCOUNT_TYPES.map((t) => (
        <option key={t.value} value={t.value} className="bg-black">
          {t.label}
        </option>
      ))}
    </select>
  </div>
  <div className="space-y-2">
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
<label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
    Financial Institution
</label>

```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```

    </p>
  </div>
) : (
  accounts.map((account, index) => (
    <ScrollReveal key={account.id} delay={index * 0.05} distance={10}>
      <div className="flex items-center justify-between py-6 border-b border-white/5 group
hover:bg-white/1 transition-all px-2">
        <div className="space-y-1">
          <span className="text-[8px] font-mono tracking-widest uppercase text-white/20">
            {AccountMap[account.account_type] || account.account_type}
          </span>
          <h3 className="font-cormorant text-xl text-white transition-transform
group-hover:translate-x-2">
            {account.account_name}
          </h3>
        </div>
        <div className="text-right">
          <p className="font-cormorant text-xl text-white">
            {formatCurrency(account.current_balance)}
          </p>
          {account.institution && (
            <p className="text-[8px] font-mono text-white/10 uppercase tracking-widest mt-1">
              {account.institution}
            </p>
          )}
        </div>
      </div>
    </ScrollReveal>
  ))
)}
</div>
</div>
);
}
```

FILE: src/app/dashboard\BaselineSection.tsx

```
"use client";

import { useState } from "react";
import { TAXONOMY_CATEGORIES } from "@lib/finance/taxonomy";
import {
  createOperationalBaselineAction,
  type DashboardSnapshot,
} from "../actions";
import { OperationalBaseline, BaselineCadence } from "@lib/analytics/types";
import { formatCurrency } from "@lib/utils/formatters";
import { DistributionPieChart } from "@components/dashboard/MiniCharts";
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
import { ScrollReveal } from "@components/ui/scroll-reveal";

interface BaselineSectionProps {
  baseline: OperationalBaseline[];
  onSnapshot: (snapshot: DashboardSnapshot) => void;
}

const CADENCES = [
  { value: "daily", label: "Daily" },
  { value: "weekly", label: "Weekly" },
  { value: "biweekly", label: "Bi-weekly" },
  { value: "monthly", label: "Monthly" },
  { value: "quarterly", label: "Quarterly" },
  { value: "yearly", label: "Yearly" },
];

export function BaselineSection({
  baseline,
  onSnapshot,
}: BaselineSectionProps) {
  const [isAdding, setIsAdding] = useState(false);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const chartData = baseline.map((b) => ({
    name: b.title,
    value: Number(b.amount),
  }));

  const [form, setForm] = useState({
    title: "",
    amount: "",
    category: "Maintenance",
    cadence: "monthly" as BaselineCadence,
    execution_day: 1,
    is_recurring: true,
    baseline_type: "expense" as "expense" | "allocation",
    notes: "",
  });

  async function handleSubmit(e: React.FormEvent) {
    e.preventDefault();
    setIsLoading(true);
    setError(null);
    try {
      const snapshot = await createOperationalBaselineAction({
        title: form.title,
        amount: Number(form.amount),
      });
    }
  }
}
```


AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
category: form.category,
cadence: form.is_recurring ? form.cadence : "monthly", // Default or special irregular? Plan says use
is_recurring logic.
execution_day:
  form.is_recurring &&
  (form.cadence === "monthly" ||
   form.cadence === "weekly" ||
   form.cadence === "biweekly")
  ? Number(form.execution_day)
  : null,
is_recurring: form.is_recurring,
baseline_type: form.baseline_type,
is_active: true,
notes: form.notes || undefined,
});
setForm({
  title: "",
  amount: "",
  category: "Maintenance",
  cadence: "monthly",
  execution_day: 1,
  is_recurring: true,
  baseline_type: "expense",
  notes: "",
});
setIsAdding(false);
onSnapshot(snapshot);
} catch (err: unknown) {
  setError(
    err instanceof Error ? err.message : "Failed to create baseline entry",
  );
} finally {
  setIsLoading(false);
}
}

return (
  <div className="space-y-12">
    <div className="flex items-center justify-between">
      <h2 className="font-cormorant text-2xl text-white tracking-wide uppercase">
        Operational Maintenance
      </h2>
      <button
        onClick={() => setIsAdding(!isAdding)}
        className="font-mono text-[9px] tracking-[0.4em] uppercase text-white/40 hover:text-white
transition-colors"
        >
        {isAdding ? "? CANCEL" : "+ APPEND RECURRING"}
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```

    </button>
  </div>

  {isAdding && (
    <div className="bg-white/2 border border-white/5 p-8 animate-in fade-in slide-in-from-top-4
duration-500">
      <form onSubmit={handleSubmit} className="space-y-8">
        <div className="grid grid-cols-1 md:grid-cols-2 gap-8">
          <div className="space-y-2">
            <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
              Baseline Name
            </label>
            <input
              type="text"
              required
              placeholder="e.g. Rent, Netflix"
              value={form.title}
              onChange={(e) => setForm({ ...form, title: e.target.value })}
              className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light
focus:outline-none focus:border-white transition-colors"
            />
          </div>
          <div className="space-y-2">
            <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
              Amount
            </label>
            <input
              type="number"
              required
              placeholder="0.00"
              value={form.amount}
              onChange={(e) => setForm({ ...form, amount: e.target.value })}
              className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light
focus:outline-none focus:border-white transition-colors"
            />
          </div>
        </div>
      </div>

      <div className="space-y-6">
        <div className="space-y-2">
          <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
            Category
          </label>
          <select
            value={form.category}
            onChange={(e) =>
              setForm({ ...form, category: e.target.value })
            }
          />
        </div>
      </div>
    </div>
  )
}
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
        className="w-full bg-transparent border-b border-white/10 py-3 text-white/60 font-mono
text-[10px] tracking-widest uppercase focus:outline-none"
    >
        {TAXONOMY_CATEGORIES.map((cat) => (
            <option
                key={cat.value}
                value={cat.value}
                className="bg-black"
            >
                {cat.label}
            </option>
        ))}
    </select>
</div>

<div className="space-y-4">
    <label className="flex items-center space-x-3 cursor-pointer group">
        <input
            type="checkbox"
            checked={form.is_recurring}
            onChange={(e) =>
                setForm({ ...form, is_recurring: e.target.checked })
            }
            className="w-4 h-4 rounded border-white/10 bg-transparent checked:bg-white
transition-colors cursor-pointer"
        />
        <span className="text-[10px] font-mono text-white/40 uppercase tracking-widest
group-hover:text-white/60 transition-colors">
            Recurring Expense
        </span>
    </label>

    {form.is_recurring && (
        <div className="space-y-8 pt-4 animate-in fade-in slide-in-from-top-2 border-l border-white/5
pl-6 ml-2">
            <div className="grid grid-cols-1 md:grid-cols-2 gap-8">
                <div className="space-y-2">
                    <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
                        Frequency
                    </label>
                    <select
                        value={form.cadence}
                        onChange={(e) =>
                            setForm({
                                ...form,
                                cadence: e.target.value as BaselineCadence,
                            })
                        }
                    >
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
        className="w-full bg-transparent border-b border-white/10 py-3 text-white/60
font-mono text-[10px] tracking-widest uppercase focus:outline-none"
    >
        {CADENCES.map((c) => (
            <option
                key={c.value}
                value={c.value}
                className="bg-black"
            >
                {c.label}
            </option>
        ))}
    </select>
</div>

{form.cadence === "daily" && (
    <div className="flex items-center pt-4">
        <p className="text-[10px] font-mono text-white/40 uppercase tracking-widest
leading-relaxed">

            <span className="text-white/60">Note:</span> You
            will be prompted everyday to verify this baseline.
        </p>
    </div>
)}

{(form.cadence === "monthly" ||
form.cadence === "weekly" ||
form.cadence === "biweekly") && (
    <div className="space-y-2">
        <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
            {form.cadence === "monthly"
                ? "Day of Month (1-31)"
                : "Day of Week"}
        </label>
        {form.cadence === "monthly" ? (
            <input
                type="number"
                min="1"
                max="31"
                required
                value={form.execution_day || ""}
                onChange={(e) =>
                    setForm({
                        ...form,
                        execution_day: Number(e.target.value),
                    })
                }
                className="w-full bg-transparent border-b border-white/10 py-3 text-white
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
font-light focus:outline-none focus:border-white transition-colors"
    />
  ) : (
    <select
      value={form.execution_day || 1}
      onChange={(e) =>
        setForm({
          ...form,
          execution_day: Number(e.target.value),
        })
      }
      className="w-full bg-transparent border-b border-white/10 py-3 text-white/60
font-mono text-[10px] tracking-widest uppercase focus:outline-none"
    >
      {[
        { label: "Monday", value: 1 },
        { label: "Tuesday", value: 2 },
        { label: "Wednesday", value: 3 },
        { label: "Thursday", value: 4 },
        { label: "Friday", value: 5 },
        { label: "Saturday", value: 6 },
        { label: "Sunday", value: 7 },
      ]}.map((day) => (
        <option
          key={day.value}
          value={day.value}
          className="bg-black"
        >
          {day.label}
        </option>
      )))
    </select>
  )}
</div>
)}
</div>
</div>
)}
</div>
</div>

{error && (
  <p className="text-red-500 font-mono text-[9px] uppercase tracking-widest">
    {error}
  </p>
)}
)}

<button
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
        type="submit"
        disabled={isLoading}
        className="w-full bg-white text-black py-4 font-medium tracking-[0.2em] uppercase text-[10px]
hover:bg-white/90 transition-colors disabled:opacity-50"
      >
        {isLoading ? "SAVING..." : "CONFIRM RECURRING"}
      </button>
    </form>
  </div>
)}

{baseline.length > 0 && (
  <div className="py-6 border-b border-white/5 mb-6">
    <DistributionPieChart data={chartData} />
  </div>
)}

<div className="space-y-2">
  {baseline.length === 0 ? (
    <div className="py-12 text-center opacity-20">
      <p className="text-[10px] font-mono uppercase tracking-[0.5em]">
        No recurring flows active
      </p>
    </div>
  ) : (
    baseline.map((item, index) => (
      <ScrollReveal key={item.id} delay={index * 0.05} distance={10}>
        <div className="flex items-center justify-between py-6 border-b border-white/5 group
hover:bg-white/1 transition-all px-2">
          <div className="space-y-1">
            <div className="flex items-center gap-3">
              <span className="text-[8px] font-mono tracking-widest uppercase text-white/20">
                {item.cadence}
              </span>
              {item.execution_day && (
                <span className="text-[8px] font-mono tracking-widest uppercase text-white/10">
                  ? Day {item.execution_day}
                </span>
              )}
            </div>
          </div>
          <h3 className="font-cormorant text-xl text-white transition-transform
group-hover:translate-x-2">
            {item.title}
          </h3>
        </div>
        <div className="text-right">
          <p className="font-cormorant text-xl text-white">
            {formatCurrency(item.amount)}
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
        </p>
      </div>
    </div>
  </ScrollReveal>
  ))
  })
</div>
</div>
);
}
```

FILE: src/app/dashboard/error.tsx

```
"use client";

import { useEffect } from "react";

export default function DashboardError({
  error,
  reset,
}: {
  error: Error & { digest?: string };
  reset: () => void;
}) {
  useEffect(() => {
    // Log the error to an error reporting service
    console.error("Dashboard Route Error:", error);
  }, [error]);

  return (
    <main className="min-h-[80vh] flex items-center justify-center p-6">
      <div className="w-full max-w-md bg-background border-2 border-red-500/20 rounded-[2.5rem] p-10 shadow-2xl text-center">
        <div className="w-20 h-20 bg-red-500/10 rounded-3xl flex items-center justify-center mx-auto mb-6">
          <svg
            width="40"
            height="40"
            viewBox="0 0 24 24"
            fill="none"
            stroke="currentColor"
            strokeWidth="3"
            className="text-red-500"
          >
            <path d="M10.29 3.86L11.82 18a2 2 0 0 0 1.71 3h16.94a2 2 0 0 0 1.71-3L13.71 3.86a2 2 0 0 0-3.42 0z"
            />
            <line x1="12" y1="9" x2="12" y2="13" />
            <line x1="12" y1="17" x2="12.01" y2="17" />
          </svg>
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
</div>

<h1 className="text-3xl font-black tracking-tighter uppercase mb-4 text-foreground">
  System Interrupted
</h1>

<p className="text-foreground/60 text-sm font-bold uppercase tracking-widest mb-8 leading-relaxed">
  The intelligence engine encountered an unhandled exception.
</p>

<div className="bg-foreground/5 rounded-2xl p-4 mb-8 text-left">
  <p className="text-[10px] font-black text-foreground/40 uppercase tracking-widest mb-2">
    Error Diagnostic
  </p>
  <p className="text-xs font-mono text-red-400 break-all leading-tight">
    {error.message || "Unknown cryptographic failure"}
  </p>
</div>

<button
  onClick={() => reset()}
  className="w-full bg-foreground text-background px-4 py-5 rounded-2xl font-black text-lg
  hover:scale-[1.02] active:scale-[0.98] transition-all shadow-xl shadow-foreground/10 uppercase"
  >
  Attempt Recovery
</button>
</div>
</main>
);
}
```

FILE: src/app/dashboard\GoalSection.tsx

```
"use client";

import { useState } from "react";
import {
  calculateGoalProgressPercentage,
  type FinancialGoal,
  GOAL_TYPES,
  GOAL_PRIORITIES,
  GOAL_STATUSES,
} from "@/lib/finance/goals";
import { createGoalAction, type DashboardSnapshot } from "../actions";
import { formatCurrency } from "@/lib/utils/formatters";
import { ScrollReveal } from "@/components/ui/scroll-reveal";
import { motion } from "framer-motion";
```


AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
interface GoalSectionProps {
  goals: FinancialGoal[];
  onSnapshot: (snapshot: DashboardSnapshot) => void;
}

export function GoalSection({ goals, onSnapshot }: GoalSectionProps) {
  const [isAdding, setIsAdding] = useState(false);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const [form, setForm] = useState({
    goal_name: "",
    goal_type: "emergency_fund",
    target_amount: "",
    current_progress: "",
    priority: "medium",
    status: "active",
    target_date: "",
    notes: "",
  });

  async function handleSubmit(e: React.FormEvent) {
    e.preventDefault();
    setIsLoading(true);
    setError(null);
    try {
      const snapshot = await createGoalAction({
        goal_name: form.goal_name,
        goal_type: form.goal_type,
        target_amount: Number(form.target_amount),
        current_progress: form.current_progress
          ? Number(form.current_progress)
          : 0,
        priority: form.priority,
        status: form.status,
        target_date: form.target_date || null,
        notes: form.notes || undefined,
      });
    } finally {
      setForm({
        goal_name: "",
        goal_type: "emergency_fund",
        target_amount: "",
        current_progress: "",
        priority: "medium",
        status: "active",
        target_date: "",
        notes: "",
      });
    }
  }
}
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
    setIsAdding(false);
    onSnapshot(snapshot);
  } catch (err: unknown) {
    setError(
      err instanceof Error ? err.message : "Failed to record financial goal",
    );
  } finally {
    setIsLoading(false);
  }
}

return (
  <div className="space-y-12">
    <div className="flex items-center justify-between">
      <h2 className="font-cormorant text-2xl text-white tracking-wide uppercase">
        Wealth Milestones
      </h2>
      <button
        onClick={() => setIsAdding(!isAdding)}
        className="font-mono text-[9px] tracking-[0.4em] uppercase text-white/40 hover:text-white
transition-colors"
        >
        {isAdding ? "? CANCEL" : "+ APPEND MILESTONE"}
      </button>
    </div>

    {isAdding && (
      <div className="bg-white2 border border-white/5 p-8 animate-in fade-in slide-in-from-top-4
duration-500">
        <form onSubmit={handleSubmit} className="space-y-8">
          <div className="grid grid-cols-1 md:grid-cols-2 gap-8">
            <div className="space-y-2">
              <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
                Milestone Name
              </label>
              <input
                type="text"
                required
                placeholder="e.g. Dream House"
                value={form.goal_name}
                onChange={(e) =>
                  setForm({ ...form, goal_name: e.target.value })
                }
                className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light
focus:outline-none focus:border-white transition-colors"
              />
            </div>
            <div className="space-y-2">
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
<label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
  Milestone Type
</label>
<select
  value={form.goal_type}
  onChange={(e) =>
    setForm({ ...form, goal_type: e.target.value })
  }
  className="w-full bg-transparent border-b border-white/10 py-3 text-white/60 font-mono
text-[10px] tracking-widest uppercase focus:outline-none"
>
  {GOAL_TYPES.map((type) => (
    <option
      key={type.value}
      value={type.value}
      className="bg-black"
    >
      {type.label}
    </option>
  ))}
</select>
</div>
</div>

<div className="grid grid-cols-1 md:grid-cols-2 gap-8">
  <div className="space-y-2">
    <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
      Priority
    </label>
    <select
      value={form.priority}
      onChange={(e) =>
        setForm({ ...form, priority: e.target.value })
      }
      className="w-full bg-transparent border-b border-white/10 py-3 text-white/60 font-mono
text-[10px] tracking-widest uppercase focus:outline-none"
    >
      {GOAL_PRIORITIES.map((p) => (
        <option key={p.value} value={p.value} className="bg-black">
          {p.label}
        </option>
      ))}
    </select>
  </div>
  <div className="space-y-2">
    <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
      Status
    </label>
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
<select
  value={form.status}
  onChange={(e) => setForm({ ...form, status: e.target.value })}
  className="w-full bg-transparent border-b border-white/10 py-3 text-white/60 font-mono
text-[10px] tracking-widest uppercase focus:outline-none"
>
  {GOAL_STATUSES.map((s) => (
    <option key={s.value} value={s.value} className="bg-black">
      {s.label}
    </option>
  ))}
</select>
</div>
</div>

<div className="grid grid-cols-1 md:grid-cols-2 gap-8">
  <div className="space-y-2">
    <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
      Target Amount
    </label>
    <input
      type="number"
      required
      placeholder="0.00"
      value={form.target_amount}
      onChange={(e) =>
        setForm({ ...form, target_amount: e.target.value })
      }
      className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light
focus:outline-none focus:border-white transition-colors"
    />
  </div>
  <div className="space-y-2">
    <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
      Current Progress
    </label>
    <input
      type="number"
      placeholder="0.00"
      value={form.current_progress}
      onChange={(e) =>
        setForm({ ...form, current_progress: e.target.value })
      }
      className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light
focus:outline-none focus:border-white transition-colors"
    />
  </div>
</div>
</div>
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
<div className="grid grid-cols-1 md:grid-cols-2 gap-8">
  <div className="space-y-2">
    <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
      Target Date
    </label>
    <input
      type="date"
      value={form.target_date}
      onChange={(e) =>
        setForm({ ...form, target_date: e.target.value })
      }
      className="w-full bg-transparent border-b border-white/10 py-3 text-white/60 font-mono
text-[10px] tracking-widest uppercase focus:outline-none"
    />
  </div>
  <div className="space-y-2">
    <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
      Milestone Notes
    </label>
    <input
      type="text"
      placeholder="Add context..."
      value={form.notes}
      onChange={(e) => setForm({ ...form, notes: e.target.value })}
      className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light
focus:outline-none focus:border-white transition-colors"
    />
  </div>
</div>

{error && (
  <p className="text-red-500 font-mono text-[9px] uppercase tracking-widest">
    {error}
  </p>
)}

<button
  type="submit"
  disabled={isLoading}
  className="w-full bg-white text-black py-4 font-medium tracking-[0.2em] uppercase text-[10px]
hover:bg-white/90 transition-colors disabled:opacity-50"
  >
  {isLoading ? "SAVING..." : "CONFIRM MILESTONE"}
</button>
</form>
</div>
)}
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
<div className="space-y-8">
  {goals.length === 0 ? (
    <div className="py-12 text-center opacity-20">
      <p className="text-[10px] font-mono uppercase tracking-[0.5em]">
        No milestones active
      </p>
    </div>
  ) : (
    goals.map((goal, index) => {
      const progress = calculateGoalProgressPercentage(goal);
      return (
        <ScrollReveal key={goal.id} delay={index * 0.1}>
          <div className="space-y-4 group cursor-default pb-8 border-b border-white/5 last:border-0">
            <div className="flex items-center justify-between mb-2">
              <div className="space-y-1">
                <span
                  className={`text-[8px] font-mono tracking-widest uppercase ${
                    goal.priority === "critical"
                      ? "text-red-500"
                      : "text-white/20"
                  }`}
                >
                  {goal.priority}
                </span>
                <h3 className="font-cormorant text-2xl text-white transition-transform
group-hover:translate-x-2">
                  {goal.goal_name}
                </h3>
              </div>
              <div className="text-right">
                <p className="font-cormorant text-3xl text-white tabular-nums">
                  {Math.round(progress)}%
                </p>
              </div>
            </div>
          </div>

          <div className="h-px w-full bg-white/5 relative overflow-hidden">
            <motion.div
              initial={{ width: 0 }}
              whileInView={{ width: `${Math.min(100, progress)}%` }}
              viewport={{ once: true }}
              transition={{
                duration: 1.5,
                delay: 0.5 + index * 0.1,
                ease: "easeOut",
              }}
            >
              <div className="absolute inset-y-0 left-0 bg-white/60">
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
        />
      </div>

      <div className="flex justify-between items-end pt-2 opacity-30 group-hover:opacity-60
transition-opacity">
        <p className="text-[9px] font-mono uppercase tracking-widest">
          {formatCurrency(goal.current_progress)}
        </p>
        <p className="text-[9px] font-mono uppercase tracking-widest">
          Target: {formatCurrency(goal.target_amount)}
        </p>
      </div>
    </div>
  </ScrollReveal>
);
})
)}
</div>
</div>
);
}
```

FILE: src/app/dashboard\HistoricalAudit.tsx

```
"use client";

import React from "react";
import { Deployment } from "@lib/analytics/types";
import { formatCurrency } from "@lib/utils/formatters";
import { DeploymentMap } from "@lib/utils/taxonomy";

interface HistoricalAuditProps {
  deployments: Deployment[];
}

export function HistoricalAudit({ deployments }: HistoricalAuditProps) {
  if (!deployments || deployments.length === 0) {
    return (
      <div className="p-8 text-center border border-white/5 rounded-3xl opacity-20">
        <p className="text-[10px] font-mono uppercase tracking-[0.4em]">
          No deployment history found
        </p>
      </div>
    );
  }

  // Sort by date descending
  const sortedDeployments = [...deployments].sort(
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
(a, b) =>
  new Date(b.created_at).getTime() - new Date(a.created_at).getTime(),
);

return (
  <div className="space-y-12">
    <div className="flex items-center gap-6">
      <h3 className="font-cormorant italic text-3xl text-white/40">
        Historical Audit
      </h3>
      <div className="flex-1 h-px bg-white/5" />
    </div>

    <div className="space-y-4 max-h-[600px] overflow-y-auto scrollbar-hide pr-2">
      {sortedDeployments.map((record) => (
        <div
          key={record.id}
          className="group relative bg-white/[0.02] border border-white/5 rounded-2xl p-6 flex items-center
justify-between transition-all hover:bg-white/[0.04] hover:border-white/10"
        >
          <div className="flex items-center gap-6">
            <div className="w-12 h-12 bg-white/5 rounded-xl flex items-center justify-center
group-hover:scale-110 transition-transform">
              <svg
                width="20"
                height="20"
                viewBox="0 0 24 24"
                fill="none"
                stroke="white"
                strokeWidth="1.5"
                className="opacity-40 group-hover:opacity-80 transition-opacity"
              >
                <path d="M12 2v20M17 5H9.5a3.5 3.5 0 0 0 0 0 7h5a3.5 3.5 0 0 1 0 7H6" />
              </svg>
            </div>
            <div className="space-y-1">
              <h4 className="font-cormorant text-xl text-white/80 group-hover:text-white transition-colors">
                {record.title}
              </h4>
              <div className="flex items-center gap-3">
                <span className="text-[9px] font-mono bg-white/5 px-2 py-0.5 rounded-full text-white/40
uppercase tracking-widest">
                  {DeploymentMap[
                    record.category as keyof typeof DeploymentMap
                  ] || record.category}
                </span>
                <span className="text-[8px] font-mono text-white/20 uppercase tracking-widest">
                  {new Date(record.created_at).toLocaleDateString(undefined, {
```


AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
        month: "short",
        day: "numeric",
    }}}
    </span>
  </div>
</div>
<div className="text-right">
  <p className="font-cormorant text-2xl text-white">
    {formatCurrency(record.amount)}
  </p>
  <p className="text-[8px] font-mono text-white/10 uppercase tracking-[0.3em] mt-1">
    Verified Asset
  </p>
</div>
</div>
)}}
</div>

<p className="text-center text-[8px] font-mono text-white/10 uppercase tracking-[0.5em]">
  Forensic layer immutable // Protocol v1.0
</p>
</div>
);
}
```

FILE: src/app/dashboard/IncomeSection.tsx

```
"use client";

import { useState } from "react";
import {
  INCOME_TYPES,
  CADENCES,
  type IncomeStream,
  type IncomeType,
  type Cadence,
} from "@lib/finance/income";
import { createIncomeAction, type DashboardSnapshot } from "../actions";
import { formatCurrency } from "@lib/utils/formatters";
import { IncomeMap } from "@lib/utils/taxonomy";
import { DistributionPieChart } from "@components/dashboard/MiniCharts";
import { ScrollReveal } from "@components/ui/scroll-reveal";

interface IncomeSectionProps {
  incomeStreams: IncomeStream[];
  onSnapshot: (snapshot: DashboardSnapshot) => void;
}
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
interface FormEntry {
  income_name: string;
  income_type: IncomeType;
  amount: string;
  cadence: Cadence;
  execution_day: number;
  is_recurring: boolean;
  source: string;
}

export function IncomeSection({
  incomeStreams,
  onSnapshot,
}: IncomeSectionProps) {
  const [isAdding, setIsAdding] = useState(false);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const chartData = incomeStreams.map((s) => ({
    name: s.income_name,
    value: Number(s.amount),
  }));

  const [entries, setEntries] = useState<FormEntry[]>([
    {
      income_name: "",
      income_type: "salary",
      amount: "",
      cadence: "monthly",
      execution_day: 1,
      is_recurring: true,
      source: "",
    },
  ]);

  const addEntry = () => {
    setEntries([
      ...entries,
      {
        income_name: "",
        income_type: "salary",
        amount: "",
        cadence: "monthly",
        execution_day: 1,
        is_recurring: true,
        source: "",
      },
    ],
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
    ]);
};

const updateEntry = <K extends keyof FormEntry>(
  index: number,
  field: K,
  value: FormEntry[K],
) => {
  const newEntries = [...entries];
  newEntries[index][field] = value;
  setEntries(newEntries);
};

async function handleSubmit(e: React.FormEvent) {
  e.preventDefault();
  setIsLoading(true);
  setError(null);
  try {
    const snapshot = await createIncomeAction(
      entries.map((entry) => ({
        ...entry,
        amount: Number(entry.amount),
        cadence: entry.is_recurring ? entry.cadence : "irregular",
        execution_day:
          entry.is_recurring &&
          (entry.cadence === "monthly" ||
            entry.cadence === "weekly" ||
            entry.cadence === "biweekly")
            ? Number(entry.execution_day)
            : null,
        source: entry.source || undefined,
      })),
    );
    setEntries([
      {
        income_name: "",
        income_type: "salary",
        amount: "",
        cadence: "monthly",
        execution_day: 1,
        is_recurring: true,
        source: "",
      },
    ]);
    setIsAdding(false);
    onSnapshot(snapshot);
  } catch (err: unknown) {
    setError(
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
    err instanceof Error ? err.message : "Failed to record income streams",
  );
} finally {
  setIsLoading(false);
}
}

return (
  <div className="space-y-12">
    <div className="flex items-center justify-between">
      <h2 className="font-cormorant text-2xl text-white tracking-wide uppercase">
        Income Velocity
      </h2>
      <button
        onClick={() => setIsAdding(!isAdding)}
        className="font-mono text-[9px] tracking-[0.4em] uppercase text-white/40 hover:text-white
transition-colors"
      >
        {isAdding ? "? CANCEL" : "+ APPEND INCOME"}
      </button>
    </div>

    {isAdding && (
      <div className="bg-white/2 border border-white/5 p-8 animate-in fade-in slide-in-from-top-4
duration-500">
        <form onSubmit={handleSubmit} className="space-y-12">
          {entries.map((entry, index) => (
            <div key={index} className="space-y-8 relative">
              {index > 0 && <div className="border-t border-white/5 pt-8" />}

              <div className="grid grid-cols-1 md:grid-cols-2 gap-8">
                <div className="space-y-2">
                  <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
                    Source Name
                  </label>
                  <input
                    type="text"
                    required
                    placeholder="e.g. Salary"
                    value={entry.income_name}
                    onChange={(e) =>
                      updateEntry(index, "income_name", e.target.value)
                    }
                    className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light
focus:outline-none focus:border-white transition-colors"
                  />
                </div>
                <div className="space-y-2">
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
<label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
  Origin / Payer
</label>
<input
  type="text"
  placeholder="e.g. Acme Corp"
  value={entry.source}
  onChange={(e) =>
    updateEntry(index, "source", e.target.value)
  }
  className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light
focus:outline-none focus:border-white transition-colors"
/>
</div>
</div>

<div className="grid grid-cols-1 md:grid-cols-2 gap-8">
  <div className="space-y-2">
    <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
      Type
    </label>
    <select
      value={entry.income_type}
      onChange={(e) =>
        updateEntry(
          index,
          "income_type",
          e.target.value as IncomeType,
        )
      }
      className="w-full bg-transparent border-b border-white/10 py-3 text-white/60 font-mono
text-[10px] tracking-widest uppercase focus:outline-none"
    >
      {INCOME_TYPES.map((t) => (
        <option
          key={t.value}
          value={t.value}
          className="bg-black"
        >
          {t.label}
        </option>
      ))}
    </select>
  </div>
  <div className="space-y-2">
    <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
      Amount
    </label>
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
<input
  type="number"
  required
  placeholder="0.00"
  value={entry.amount}
  onChange={(e) =>
    updateEntry(index, "amount", e.target.value)
  }
  className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light
focus:outline-none focus:border-white transition-colors"
/>
</div>
</div>

<div className="space-y-4">
  <label className="flex items-center space-x-3 cursor-pointer group">
    <input
      type="checkbox"
      checked={entry.is_recurring}
      onChange={(e) =>
        updateEntry(index, "is_recurring", e.target.checked)
      }
      className="w-4 h-4 rounded border-white/10 bg-transparent checked:bg-white
transition-colors cursor-pointer"
    />
    <span className="text-[10px] font-mono text-white/40 uppercase tracking-widest
group-hover:text-white/60 transition-colors">
      Recurring Inflow
    </span>
  </label>

  {entry.is_recurring && (
    <div className="space-y-8 pt-4 animate-in fade-in slide-in-from-top-2 border-l
border-white/5 pl-6 ml-2">
      <div className="grid grid-cols-1 md:grid-cols-2 gap-8">
        <div className="space-y-2">
          <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
            Frequency
          </label>
          <select
            value={entry.cadence}
            onChange={(e) =>
              updateEntry(
                index,
                "cadence",
                e.target.value as Cadence,
              )
            }
          >
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
        className="w-full bg-transparent border-b border-white/10 py-3 text-white/60
font-mono text-[10px] tracking-widest uppercase focus:outline-none"
    >
        {CADENCES.map((c) => (
            <option
                key={c.value}
                value={c.value}
                className="bg-black"
            >
                {c.label}
            </option>
        ))}
    </select>
</div>
{entry.cadence === "monthly" ||
entry.cadence === "weekly" ||
entry.cadence === "biweekly") && (
<div className="space-y-2">
    <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
        {entry.cadence === "monthly"
            ? "Day of Month (1-31)"
            : "Day of Week"}
    </label>
    {entry.cadence === "monthly" ? (
        <input
            type="number"
            min="1"
            max="31"
            required
            value={entry.execution_day || ""}
            onChange={(e) =>
                updateEntry(
                    index,
                    "execution_day",
                    Number(e.target.value),
                )
            }
            className="w-full bg-transparent border-b border-white/10 py-3 text-white
font-light focus:outline-none focus:border-white transition-colors"
        />
    ) : (
        <select
            value={entry.execution_day || 1}
            onChange={(e) =>
                updateEntry(
                    index,
                    "execution_day",
                    Number(e.target.value),
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
    )
  }
  className="w-full bg-transparent border-b border-white/10 py-3 text-white/60
font-mono text-[10px] tracking-widest uppercase focus:outline-none"
  >
  {[
    { label: "Monday", value: 1 },
    { label: "Tuesday", value: 2 },
    { label: "Wednesday", value: 3 },
    { label: "Thursday", value: 4 },
    { label: "Friday", value: 5 },
    { label: "Saturday", value: 6 },
    { label: "Sunday", value: 7 },
  ]}.map((day) => (
    <option
      key={day.value}
      value={day.value}
      className="bg-black"
    >
      {day.label}
    </option>
  )))
</select>
  })
</div>
  })
</div>
</div>
  })
</div>
  })
  <div className="space-y-6">
    <button
      type="button"
      onClick={addEntry}
      className="text-[9px] font-mono tracking-widest uppercase text-white/20 hover:text-white
transition-colors"
    >
      + APPEND ANOTHER SOURCE
    </button>

    {error && (
      <p className="text-red-500 font-mono text-[9px] uppercase tracking-widest">
        {error}
      </p>
    )}
  )}
```


AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
<button
  type="submit"
  disabled={isLoading}
  className="w-full bg-white text-black py-4 font-medium tracking-[0.2em] uppercase text-[10px]
hover:bg-white/90 transition-colors disabled:opacity-50"
>
  {isLoading ? "SAVING..." : "CONFIRM INCOMES"}
</button>
</div>
</form>
</div>
)}

{incomeStreams.length > 0 && (
  <div className="py-6 border-b border-white/5 mb-6">
    <DistributionPieChart data={chartData} />
  </div>
)}

<div className="space-y-2">
  {incomeStreams.length === 0 ? (
    <div className="py-12 text-center opacity-20">
      <p className="text-[10px] font-mono uppercase tracking-[0.5em]">
        No income streams active
      </p>
    </div>
  ) : (
    incomeStreams.map((stream, index) => (
      <ScrollReveal key={stream.id} delay={index * 0.05} distance={10}>
        <div className="flex items-center justify-between py-6 border-b border-white/5 group
hover:bg-white/1 transition-all px-2">
          <div className="space-y-1">
            <span className="text-[8px] font-mono tracking-widest uppercase text-white/20">
              {IncomeMap[stream.income_type] || stream.income_type}
              {stream.source && ` ? ${stream.source}`}
            </span>
            <h3 className="font-cormorant text-xl text-white transition-transform
group-hover:translate-x-2">
              {stream.income_name}
            </h3>
          </div>
          <div className="text-right">
            <p className="font-cormorant text-xl text-white">
              {formatCurrency(stream.amount)}
            <span className="text-[10px] ml-2 text-white/20 font-sans tracking-widest uppercase">
              / {stream.cadence}
            </span>
          </div>
        </div>
      </ScrollReveal>
    )
  )
}
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
        </p>
        {stream.execution_day && (
          <p className="text-[8px] font-mono text-white/10 uppercase tracking-widest mt-1">
            Executes on day {stream.execution_day}
          </p>
        )}
      </div>
    </div>
  </ScrollReveal>
))
)}
</div>
</div>
);
}
```

FILE: src/app/dashboard/layout.tsx

```
"use client";

import { supabase } from "@/lib/supabase";
import { useEffect, useState } from "react";
import { User } from "@supabase/supabase-js";
import Link from "next/link";
import { SmoothScrollProvider } from "@/components/ui/smooth-scroll-provider";
import { checkOnboardingStatusAction } from "@/actions";
import { BrandMark } from "@/components/ui/brand-mark";

export const dynamic = "force-dynamic";

export default function DashboardLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  const [user, setUser] = useState<User | null>(null);
  const [isProfileOpen, setIsProfileOpen] = useState(false);
  const [activeSection, setActiveSection] = useState("overview");
  const [isOnboarded, setIsOnboarded] = useState<boolean | null>(null);

  useEffect(() => {
    async function init() {
      const {
        data: { user },
      } = await supabase.auth.getUser();
      setUser(user);

      try {
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
const { isOnboarded } = await checkOnboardingStatusAction();
setIsOnboarded(isOnboarded);
} catch {
  setIsOnboarded(false);
}
}
init();

const handleScroll = () => {
  const sections = ["overview", "deploy", "intelligence", "ledger"];
  const scrollTop = window.scrollY + 200;

  for (const section of sections) {
    const element = document.getElementById(section);
    if (element) {
      const { offsetTop, offsetHeight } = element;
      if (
        scrollTop >= offsetTop &&
        scrollTop < offsetTop + offsetHeight
      ) {
        setActiveSection(section);
        break;
      }
    }
  }
};

window.addEventListener("scroll", handleScroll);
return () => window.removeEventListener("scroll", handleScroll);
}, []);

async function handleLogout() {
  await supabase.auth.signOut();
  window.location.href = "/";
}

const navItems = [
  { id: "overview", label: "Overview", roman: "I" },
  { id: "deploy", label: "Architecture", roman: "II" },
  { id: "intelligence", label: "Intelligence", roman: "III" },
  { id: "ledger", label: "Ledger", roman: "IV" },
];

const scrollToSection = (id: string) => {
  const element = document.getElementById(id);
  if (element) {
    const offset = 100;
    const bodyRect = document.body.getBoundingClientRect().top;
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
const elementRect = element.getBoundingClientRect().top;
const elementPosition = elementRect - bodyRect;
const offsetPosition = elementPosition - offset;

window.scrollTo({
  top: offsetPosition,
  behavior: "smooth",
});
}
};

return (
  <div className="min-h-screen bg-black text-white flex flex-col md:flex-row relative selection:bg-white selection:text-black">
    {/ * Background Atmosphere */}
    <div className="fixed inset-0 z-0">
      <div className="absolute top-0 right-0 w-[40%] h-[40%] bg-white/2 rounded-full blur-[120px]" />
      <div className="absolute bottom-0 left-0 w-[40%] h-[40%] bg-white/2 rounded-full blur-[120px]" />
      <div className="absolute inset-0 bg-[url('https://www.transparenttextures.com/patterns/carbon-fibre.png')] opacity-10 pointer-events-none" />
    </div>

    {isOnboarded === true && (
      <>
        {/ * SIDEBAR */}
        <aside className="hidden md:flex flex-col fixed left-0 top-0 bottom-0 w-80 bg-black/40 backdrop-blur-3xl border-r border-white/5 z-50 p-12 animate-in fade-in slide-in-from-left-4 duration-1000 overflow-hidden">
          {/ * Ambient Sidebar Glow */}
          <div className="absolute top-0 left-0 w-full h-full bg-[radial-gradient(ellipse_at_top_left,rgba(255,255,255,0.03),transparent_70%)] pointer-events-none" />

          <div className="relative z-10 flex flex-col h-full justify-between">
            <div className="space-y-24">
              {/ * Logo */}
              <Link href="/" className="group mb-8 block">
                <div className="flex items-center gap-4">
                  <BrandMark className="w-10 h-10 group-hover:scale-105 transition-transform duration-700" />
                  <span className="font-cormorant text-2xl tracking-widest uppercase text-white group-hover:text-white/80 transition-colors">
                    AXIOM
                  </span>
                </div>
              </div>
              </Link>

              {/ * Navigation */}
              <nav className="space-y-6">
                {navItems.map((item) => (
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
<button
  key={item.id}
  onClick={() => scrollToSection(item.id)}
  className={`w-full group text-left transition-all duration-500 py-3 ${
    activeSection === item.id
      ? "opacity-100"
      : "opacity-30 hover:opacity-100"
  }}
>
  <div className="flex items-center gap-6 relative">
    {/* Active Line Indicator */}
    <div
      className={`absolute -left-6 top-1/2 -translate-y-1/2 w-1 h-0 bg-white transition-all
duration-500 ${
        activeSection === item.id
          ? "h-full"
          : "group-hover:h-1/2 opacity-30"
        }}
    />
    <span className="font-cormorant italic text-2xl text-white/60 w-8">
      {item.roman}.
    </span>
    <span
      className={`font-cormorant text-3xl tracking-wide transition-all duration-500 ${
        activeSection === item.id
          ? "translate-x-2 text-white"
          : "group-hover:translate-x-2 text-white/80"
        }}
    >
      {item.label}
    </span>
  </div>
</button>
  )})
</nav>
</div>

{/* User Profile */}
<div className="pt-12 border-t border-white/5 relative">
  <button
    onClick={() => setIsProfileOpen(!isProfileOpen)}
    className="flex items-center gap-4 w-full group"
  >
    <div className="w-10 h-10 border border-white/10 bg-white/5 flex items-center justify-center
font-mono text-xs text-white group-hover:bg-white group-hover:text-black transition-all duration-500
shadow-[0_0_15px_rgba(255,255,255,0.05)]">
      {user?.email?.[0].toUpperCase()}
    </div>
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
<div className="flex flex-col text-left">
  <span className="text-[10px] font-mono tracking-widest text-white/80 uppercase truncate
max-w-30 group-hover:text-white transition-colors">
    {user?.email?.split("@")[0]}
  </span>
  <span className="text-[8px] font-mono tracking-widest text-white/20 uppercase">
    Verified Session
  </span>
</div>
</button>

{isProfileOpen && (
  <div className="absolute bottom-full left-0 mb-4 w-full bg-black/90 backdrop-blur-xl border
border-white/10 p-2 shadow-2xl animate-in fade-in slide-in-from-bottom-2 duration-300">
    <button
      onClick={handleLogout}
      className="w-full text-left font-mono text-[9px] tracking-widest text-red-500
hover:text-red-400 uppercase p-4 hover:bg-white/5 transition-colors"
    >
      Terminate Session
    </button>
  </div>
)}
</div>
</div>
</aside>

{/* MOBILE HEADER */}
<header className="md:hidden sticky top-0 bg-black/80 backdrop-blur-xl border-b border-white/5 z-50
px-6 py-6 flex justify-between items-center">
  <div className="flex items-center gap-3">
    <BrandMark className="w-8 h-8" />
    <span className="font-mono text-[10px] tracking-widest uppercase text-white/40">
      Axiom // Portal
    </span>
  </div>
  <button onClick={() => setIsProfileOpen(!isProfileOpen)}>
    <div className="w-8 h-8 border border-white/10 flex items-center justify-center font-mono
text-[10px]">
      {user?.email?.[0].toUpperCase()}
    </div>
  </button>
</header>
</>
)}

{/* MAIN CONTENT AREA */}
<main
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
        className={`flex-1 min-h-screen relative z-10 transition-all duration-1000 ${isOnboarded === true ?
"md:ml-80" : "md:ml-0"}`}
    >
    <SmoothScrollProvider>
    <div
        className={`max-w-5xl mx-auto px-6 md:px-16 py-12 md:py-24 ${isOnboarded === false ? "h-full flex
items-center justify-center" : ""}`}
    >
        {children}
    </div>
    </SmoothScrollProvider>
</main>

{/* MOBILE BOTTOM NAV */}
{isOnboarded === true && (
    <nav className="md:hidden fixed bottom-0 left-0 right-0 bg-black/80 backdrop-blur-xl border-t
border-white/5 z-50 px-6 py-4 flex justify-between items-center">
        {navItems.map((item) => (
            <button
                key={item.id}
                onClick={() => scrollToSection(item.id)}
                className={`font-mono text-[8px] tracking-widest uppercase transition-all ${
                    activeSection === item.id ? "text-white" : "text-white/20"
                }`}
            >
                {item.label}
            </button>
        ))}
    </nav>
)}
</div>
);
}
```

FILE: src/app/dashboard/LiabilitySection.tsx

```
"use client";

import { useState } from "react";
import { LIABILITY_TYPES, type Liability } from "@lib/finance/liabilities";
import { createLiabilityAction, type DashboardSnapshot } from "../actions";
import { formatCurrency } from "@lib/utils/formatters";
import { LiabilityMap } from "@lib/utils/taxonomy";
import { DistributionPieChart } from "@components/dashboard/MiniCharts";
import { ScrollReveal } from "@components/ui/scroll-reveal";

interface LiabilitySectionProps {
    liabilities: Liability[];
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
onSnapshot: (snapshot: DashboardSnapshot) => void;
}

export function LiabilitySection({
  liabilities,
  onSnapshot,
}: LiabilitySectionProps) {
  const [isAdding, setIsAdding] = useState(false);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const chartData = liabilities.map((l) => ({
    name: l.liability_name,
    value: Number(l.outstanding_balance),
  }));

  const [form, setForm] = useState({
    liability_name: "",
    liability_type: "credit_card",
    outstanding_balance: "",
    interest_rate: "",
    is_paid_in_cadences: false,
    cadence: "monthly",
    cadence_day_date: "",
    cadence_amount: "",
    institution: "",
  });

  async function handleSubmit(e: React.FormEvent) {
    e.preventDefault();
    setIsLoading(true);
    setError(null);
    try {
      const snapshot = await createLiabilityAction({
        liability_name: form.liability_name,
        liability_type: form.liability_type,
        outstanding_balance: Number(form.outstanding_balance),
        interest_rate: form.interest_rate ? Number(form.interest_rate) : 0,
        is_paid_in_cadences: form.is_paid_in_cadences,
        cadence: form.is_paid_in_cadences ? form.cadence : null,
        cadence_day_date: form.is_paid_in_cadences
          ? form.cadence_day_date
          : null,
        cadence_amount: form.is_paid_in_cadences
          ? Number(form.cadence_amount)
          : null,
        institution: form.institution || undefined,
      });
    };
```


AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
setForm({
  liability_name: "",
  liability_type: "credit_card",
  outstanding_balance: "",
  interest_rate: "",
  is_paid_in_cadences: false,
  cadence: "monthly",
  cadence_day_date: "",
  cadence_amount: "",
  institution: "",
});
setIsAdding(false);
onSnapshot(snapshot);
} catch (err: unknown) {
  setError(
    err instanceof Error ? err.message : "Failed to record obligation",
  );
} finally {
  setIsLoading(false);
}
}

return (
  <div className="space-y-12">
    <div className="flex items-center justify-between">
      <h2 className="font-cormorant text-2xl text-white tracking-wide uppercase">
        Financial Commitments
      </h2>
      <button
        onClick={() => setIsAdding(!isAdding)}
        className="font-mono text-[9px] tracking-[0.4em] uppercase text-white/40 hover:text-white
transition-colors"
      >
        {isAdding ? "? CANCEL" : "+ APPEND COMMITMENT"}
      </button>
    </div>

    {isAdding && (
      <div className="bg-white/2 border border-white/5 p-8 animate-in fade-in slide-in-from-top-4
duration-500">
        <form onSubmit={handleSubmit} className="space-y-8">
          <div className="grid grid-cols-1 md:grid-cols-3 gap-8">
            <div className="space-y-2">
              <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
                Commitment Name
              </label>
              <input
                type="text"

```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
        required
        placeholder="e.g. Equity Bank Loan"
        value={form.liability_name}
        onChange={(e) =>
            setForm({ ...form, liability_name: e.target.value })
        }
        className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light
focus:outline-none focus:border-white transition-colors"
    />
</div>
<div className="space-y-2">
    <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
        Financial Institution
    </label>
    <input
        type="text"
        placeholder="Optional"
        value={form.institution}
        onChange={(e) =>
            setForm({ ...form, institution: e.target.value })
        }
        className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light
focus:outline-none focus:border-white transition-colors"
    />
</div>
<div className="space-y-2">
    <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
        Outstanding Balance
    </label>
    <input
        type="number"
        required
        placeholder="0.00"
        value={form.outstanding_balance}
        onChange={(e) =>
            setForm({ ...form, outstanding_balance: e.target.value })
        }
        className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light
focus:outline-none focus:border-white transition-colors"
    />
</div>
</div>

<div className="grid grid-cols-1 md:grid-cols-2 gap-8">
    <div className="space-y-2">
        <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
            Type
        </label>
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
<select
  value={form.liability_type}
  onChange={(e) =>
    setForm({ ...form, liability_type: e.target.value })
  }
  className="w-full bg-transparent border-b border-white/10 py-3 text-white/60 font-mono
text-[10px] tracking-widest uppercase focus:outline-none"
>
  {LIABILITY_TYPES.map((t) => (
    <option key={t.value} value={t.value} className="bg-black">
      {t.label}
    </option>
  ))}
</select>
</div>
<div className="space-y-2">
  <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
    Interest (% per cadence)
  </label>
  <input
    type="number"
    step="0.01"
    placeholder="0.00"
    value={form.interest_rate}
    onChange={(e) =>
      setForm({ ...form, interest_rate: e.target.value })
    }
    className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light
focus:outline-none focus:border-white transition-colors"
  />
</div>
</div>

<div className="space-y-4">
  <label className="flex items-center space-x-3 cursor-pointer group">
    <input
      type="checkbox"
      checked={form.is_paid_in_cadences}
      onChange={(e) =>
        setForm({ ...form, is_paid_in_cadences: e.target.checked })
      }
      className="w-4 h-4 rounded border-white/10 bg-transparent checked:bg-white transition-colors
cursor-pointer"
    />
    <span className="text-[10px] font-mono text-white/40 uppercase tracking-widest
group-hover:text-white/60 transition-colors">
      Paid in Cadences
    </span>
  </label>
</div>
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

</label>

```
{form.is_paid_in_cadences && (
  <div className="space-y-8 pt-4 animate-in fade-in slide-in-from-top-2">
    <div className="grid grid-cols-1 md:grid-cols-2 gap-8">
      <div className="space-y-2">
        <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
          Cadence
        </label>
        <select
          value={form.cadence}
          onChange={(e) =>
            setForm({ ...form, cadence: e.target.value })
          }
          className="w-full bg-transparent border-b border-white/10 py-3 text-white/60 font-mono
text-[10px] tracking-widest uppercase focus:outline-none"
        >
          <option value="weekly" className="bg-black">
            Weekly
          </option>
          <option value="monthly" className="bg-black">
            Monthly
          </option>
        </select>
      </div>
      <div className="space-y-2">
        <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
          {form.cadence === "monthly"
            ? "Day of Month (1-31)"
            : "Day of Week"}
        </label>
        {form.cadence === "monthly" ? (
          <input
            type="number"
            min="1"
            max="31"
            placeholder="e.g. 15"
            value={form.cadence_day_date}
            onChange={(e) =>
              setForm({
                ...form,
                cadence_day_date: e.target.value,
              })
            }
            className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light
focus:outline-none focus:border-white transition-colors"
          />
        ) : (
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
<select
  value={form.cadence_day_date}
  onChange={(e) =>
    setForm({
      ...form,
      cadence_day_date: e.target.value,
    })
  }
  className="w-full bg-transparent border-b border-white/10 py-3 text-white/60
font-mono text-[10px] tracking-widest uppercase focus:outline-none"
>
  <option value="" className="bg-black">
    Select Day
  </option>
  {[
    "Monday",
    "Tuesday",
    "Wednesday",
    "Thursday",
    "Friday",
    "Saturday",
    "Sunday",
  ].map((day) => (
    <option key={day} value={day} className="bg-black">
      {day}
    </option>
  )))
</select>
)}
</div>
</div>

<div className="space-y-2">
  <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
    Payment Amount per Cadence
  </label>
  <input
    type="number"
    required={form.is_paid_in_cadences}
    placeholder="0.00"
    value={form.cadence_amount}
    onChange={(e) =>
      setForm({ ...form, cadence_amount: e.target.value })
    }
    className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light
focus:outline-none focus:border-white transition-colors"
  />
</div>
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```

    </div>
  })
</div>

{error && (
  <p className="text-red-500 font-mono text-[9px] uppercase tracking-widest">
    {error}
  </p>
)}

<button
  type="submit"
  disabled={isLoading}
  className="w-full bg-white text-black py-4 font-medium tracking-[0.2em] uppercase text-[10px]
  hover:bg-white/90 transition-colors disabled:opacity-50"
  >
  {isLoading ? "INITIALIZING..." : "CONFIRM COMMITMENT"}
</button>
</form>
</div>
)}

{liabilities.length > 0 && (
  <div className="py-6 border-b border-white/5 mb-6">
    <DistributionPieChart data={chartData} />
  </div>
)}

<div className="space-y-2">
  {liabilities.length === 0 ? (
    <div className="py-12 text-center opacity-20">
      <p className="text-[10px] font-mono uppercase tracking-[0.5em]">
        No active commitments
      </p>
    </div>
  ) : (
    liabilities.map((liability, index) => (
      <ScrollReveal key={liability.id} delay={index * 0.05} distance={10}>
        <div className="flex items-center justify-between py-6 border-b border-white/5 group
        hover:bg-white/1 transition-all px-2">
          <div className="space-y-1">
            <span className="text-[8px] font-mono tracking-widest uppercase text-white/20">
              {LiabilityMap[liability.liability_type] ||
                liability.liability_type}
            {liability.is_paid_in_cadences && (
              <span className="ml-2 text-white/10">
                ? {liability.cadence} ({liability.cadence_day_date})
              </span>
            )}
          </div>
        </div>
      </ScrollReveal>
    )
  )
)}
</div>
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
        })
      </span>
      <h3 className="font-cormorant text-xl text-white transition-transform
group-hover:translate-x-2">
        {liability.liability_name}
      </h3>
    </div>
    <div className="text-right">
      <p className="font-cormorant text-xl text-white">
        {formatCurrency(liability.outstanding_balance)}
      </p>
      {liability.interest_rate > 0 && (
        <p className="text-[8px] font-mono text-red-500/40 uppercase tracking-widest mt-1">
          {liability.interest_rate}% Per Cadence
        </p>
      )}
    </div>
  </div>
</ScrollReveal>
))
})
</div>
</div>
);
}
```

FILE: src/app/dashboard/loading.tsx

```
export default function DashboardLoading() {
  return (
    <div className="fixed inset-0 bg-black z-100 flex items-center justify-center px-8 md:px-12 animate-pulse">
      <div className="max-w-7xl w-full h-150 grid lg:grid-cols-[1.1fr_1.4fr] gap-16 md:gap-24 items-stretch">
        /* Left Side Skeleton - Roman Numeral & Text */
        <div className="flex flex-col justify-center h-full border-r border-white/5 pr-12 space-y-6">
          <div className="h-32 w-48 bg-white/5 rounded-2xl" />{" "}
          /* Roman Numeral */
          <div className="h-16 w-64 bg-white/10 rounded-xl" /> { /* Title */}
          <div className="space-y-3">
            <div className="h-4 w-full bg-white/5 rounded-full" />
            <div className="h-4 w-5/6 bg-white/5 rounded-full" />
            <div className="h-4 w-4/6 bg-white/5 rounded-full" />
          </div>
        </div>

        /* Right Side Skeleton - Luxury Card */
        <div className="h-full bg-white/2 border border-white/10 rounded-3xl p-10 md:p-14 flex flex-col
justify-between">
          <div className="space-y-12">
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
<div className="h-8 w-48 bg-white/5 rounded-lg" />{" "}
{ /* Form Header */ }
<div className="space-y-10">
  <div className="grid grid-cols-2 gap-8">
    <div className="h-12 bg-white/5 rounded-lg border-b border-white/10" />
    <div className="h-12 bg-white/5 rounded-lg border-b border-white/10" />
  </div>
  <div className="grid grid-cols-2 gap-8">
    <div className="h-12 bg-white/5 rounded-lg border-b border-white/10" />
    <div className="h-12 bg-white/5 rounded-lg border-b border-white/10" />
  </div>
</div>

<div className="flex justify-center pt-12">
  <div className="h-14 w-64 bg-white/5 rounded-full border border-white/10" />
</div>
</div>
</div>
);
}
```

FILE: src/app/dashboard/NewEntryForm.tsx

```
"use client";

import { useState } from "react";
import { TAXONOMY_CATEGORIES } from "@lib/finance/taxonomy";
import { DeploymentMap } from "@lib/utils/taxonomy";
import { formatCurrency } from "@lib/utils/formatters";
import { Account } from "@lib/analytics/types";

interface NewEntryFormProps {
  accounts: Account[];
  liquidity: number;
  isActionLoading: boolean;
  onSubmit: (data: {
    title: string;
    amount: number;
    category: string;
    accountId?: string;
  }) => Promise<void>;
  onCancel?: () => void;
}

export function NewEntryForm({
  accounts,
```


AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
liquidity,
isActionLoading,
onSubmit,
onCancel,
}: NewEntryFormProps) {
  const [title, setTitle] = useState("");
  const [amount, setAmount] = useState("");
  const [category, setCategory] = useState("Unclassified");
  const [accountId, setAccountId] = useState("");
  const [error, setError] = useState<string | null>(null);

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setError(null);

    if (category === "Unclassified") {
      setError("Strategic intent required.");
      return;
    }

    try {
      await onSubmit({
        title,
        amount: Number(amount),
        category,
        accountId: accountId || undefined,
      });
      setTitle("");
      setAmount("");
      setCategory("Unclassified");
      setAccountId("");
    } catch (err: unknown) {
      setError(err instanceof Error ? err.message : "Execution failed");
    }
  };

  return (
    <form onSubmit={handleSubmit} className="space-y-12">
      <div className="space-y-12">
        <div className="space-y-4">
          <label className="text-[10px] font-mono text-white/20 uppercase tracking-[0.4em] ml-1">
            Intent
          </label>
          <input
            type="text"
            disabled={isActionLoading}
            placeholder="Objective of this deployment?"
            value={title}
          />
        </div>
      </div>
    </form>
  );
}
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
        onChange={(e) => setTitle(e.target.value)}
        className="w-full bg-transparent border-b border-white/10 py-6 font-cormorant text-4xl text-white
placeholder:text-white/20 focus:outline-none focus:border-white transition-all"
        required
    />
</div>

<div className="grid grid-cols-1 md:grid-cols-2 gap-16">
  <div className="space-y-8">
    <div className="space-y-4">
      <div className="flex justify-between items-center">
        <label className="text-[10px] font-mono text-white/20 uppercase tracking-[0.4em] ml-1">
          Amount (KES)
        </label>
        <span className="text-[8px] font-mono text-white/10 uppercase tracking-widest">
          Available: {formatCurrency(liquidity)}
        </span>
      </div>
      <input
        type="number"
        disabled={isActionLoading}
        placeholder="0.00"
        value={amount}
        onChange={(e) => setAmount(e.target.value)}
        className="w-full bg-transparent border-b border-white/10 py-4 font-cormorant text-3xl
text-white placeholder:text-white/20 focus:outline-none focus:border-white transition-all tabular-nums"
        required
      />
    </div>
    <div className="space-y-4">
      <label className="text-[10px] font-mono text-white/20 uppercase tracking-[0.4em] ml-1">
        Funding Source
      </label>
      <select
        disabled={isActionLoading}
        value={accountId}
        onChange={(e) => setAccountId(e.target.value)}
        className="w-full bg-transparent border-b border-white/10 py-3 font-mono text-[10px]
tracking-widest uppercase text-white/60 focus:outline-none"
      >
        <option value="" className="bg-black">
          External / Manual
        </option>
        {accounts.map((acc) => (
          <option key={acc.id} value={acc.id} className="bg-black">
            {acc.account_name} ({formatCurrency(acc.current_balance)})
          </option>
        ))}
      </select>
    </div>
  </div>
</div>
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```

    </select>
  </div>
</div>

<div className="space-y-6">
  <label className="text-[10px] font-mono text-white/20 uppercase tracking-[0.4em] ml-1">
    Strategic Classification
  </label>
  <div className="grid grid-cols-2 gap-4">
    {TAXONOMY_CATEGORIES.map((cat) => (
      <button
        key={cat.value}
        type="button"
        onClick={() => setCategory(cat.value)}
        className={`p-4 border transition-all text-left space-y-2 ${
          category === cat.value
            ? "border-white bg-white text-black"
            : "border-white/5 bg-white/2 text-white/40 hover:border-white/20 hover:text-white"
        }`}
      >
        <span className="block text-[8px] font-mono tracking-widest uppercase">
          {DeploymentMap[cat.value] || cat.label}
        </span>
        <span
          className={`block text-[7px] font-light leading-tight ${category === cat.value ?
"text-black/60" : "text-white/20"}`}
        >
          {cat.definition}
        </span>
      </button>
    ))}
  </div>
</div>
</div>

{error && (
  <div className="p-4 border border-white/10 font-mono text-[9px] text-white/60 uppercase
tracking-widest">
    ERROR: {error}
  </div>
)}

<div className="pt-8 flex gap-4">
  {onCancel && (
    <button
      type="button"
      onClick={onCancel}

```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
        disabled={isActionLoading}
        className="flex-1 border border-white/5 text-white/40 py-6 font-medium tracking-[0.3em] uppercase
text-xs hover:bg-white/5 transition-all"
      >
        Cancel
      </button>
    )}
  <button
    type="submit"
    disabled={isActionLoading}
    className="flex-2 bg-white text-black py-6 font-medium tracking-[0.3em] uppercase text-xs
hover:bg-white/90 transition-all hover:scale-[1.02] active:scale-[0.98] shadow-2xl shadow-white/5
disabled:opacity-50"
  >
    {isActionLoading ? "EXECUTING..." : "AUTHORIZE DEPLOYMENT"}
  </button>
</div>
</form>
);
}
```

FILE: src/app/dashboard/page.tsx

```
"use client";

import { useEffect, useState, useCallback } from "react";
import {
  getDashboardSnapshotAction,
  createDeploymentAction,
  type DashboardSnapshot,
} from "../actions";
import { AccountSection } from "../AccountSection";
import { LiabilitySection } from "../LiabilitySection";
import { IncomeSection } from "../IncomeSection";
import { GoalSection } from "../GoalSection";
import { StrategicObjectiveSection } from "../StrategicObjectiveSection";
import { BaselineSection } from "../BaselineSection";
import { motion } from "framer-motion";
import { PendingInflows } from "../PendingInflows";
import { PendingLiabilities } from "../PendingLiabilities";
import { PendingBaselines } from "../PendingBaselines";
import { HistoricalAudit } from "../HistoricalAudit";
import { GrandTrajectoryChart } from "@components/dashboard/GrandTrajectoryChart";
import {
  MiniSparkline,
  MiniBarChart,
  MiniDonut,
} from "@components/dashboard/MiniCharts";
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
import { AnimatedNumber } from "@components/ui/animated-number";
import { LuxuryCard } from "@components/ui/luxury-card";
import { TelemetryDashboard } from "@components/dashboard/TelemetryDashboard";
import { NewEntryForm } from "./NewEntryForm";
import type {
  AnalyticsSummary,
  Deployment,
  Account,
  Liability,
  IncomeStream,
  FinancialGoal,
  StrategicObjective,
  OperationalBaseline,
} from "@lib/analytics/types";
import type { KairosInsight } from "@lib/ai/kairos";
import { formatCurrency } from "@lib/utils/formatters";

interface LedgerState {
  deployments: Deployment[];
  accounts: Account[];
  liabilities: Liability[];
  incomeStreams: IncomeStream[];
  goals: FinancialGoal[];
  objectives: StrategicObjective[];
  baseline: OperationalBaseline[];
  analytics: AnalyticsSummary | null;
}

export default function DashboardPage() {
  const [ledger, setLedger] = useState<LedgerState>({
    deployments: [],
    accounts: [],
    liabilities: [],
    incomeStreams: [],
    goals: [],
    objectives: [],
    baseline: [],
    analytics: null,
  });

  const [isInitialLoading, setIsInitialLoading] = useState(true);
  const [kairosInsight, setKairosInsight] = useState<KairosInsight | null>(
    null,
  );
  const [isExecuting, setIsExecuting] = useState(false);

  const applyDashboardSnapshot = useCallback((snapshot: DashboardSnapshot) => {
    setLedger({
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
    deployments: snapshot.deployments,
    accounts: snapshot.accounts,
    liabilities: snapshot.liabilities,
    incomeStreams: snapshot.incomeStreams,
    goals: snapshot.goals,
    objectives: snapshot.objectives,
    baseline: snapshot.baseline,
    analytics: snapshot.analytics,
  });
  setKairosInsight(snapshot.kairosInsight);
}, []);

const fetchDashboardData = useCallback(async () => {
  try {
    const snapshot = await getDashboardSnapshotAction();
    applyDashboardSnapshot(snapshot);
  } catch (error) {
    console.error("Failed to fetch dashboard data:", error);
  } finally {
    setIsInitialLoading(false);
  }
}, [applyDashboardSnapshot]);

useEffect(() => {
  // eslint-disable-next-line react-hooks/set-state-in-effect
  fetchDashboardData();
}, [fetchDashboardData]);

if (isInitialLoading) {
  return (
    <div className="fixed inset-0 bg-black z-[100] flex items-center justify-center px-8 md:px-12 animate-pulse">
      <div className="max-w-7xl w-full h-[600px] grid lg:grid-cols-[1.1fr_1.4fr] gap-16 md:gap-24 items-stretch">
        <div className="flex flex-col justify-center h-full border-r border-white/5 pr-12 space-y-6">
          <div className="h-32 w-48 bg-white/5 rounded-2xl" />
          <div className="h-16 w-64 bg-white/10 rounded-xl" />
          <div className="space-y-3">
            <div className="h-4 w-full bg-white/5 rounded-full" />
            <div className="h-4 w-5/6 bg-white/5 rounded-full" />
            <div className="h-4 w-4/6 bg-white/5 rounded-full" />
          </div>
        </div>
        <div className="h-full bg-white/[0.02] border border-white/10 rounded-3xl p-10 md:p-14 flex flex-col justify-between">
          <div className="space-y-12">
            <div className="h-8 w-48 bg-white/5 rounded-lg" />
            <div className="space-y-10">
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```

    <div className="grid grid-cols-2 gap-8">
      <div className="h-12 bg-white/5 rounded-lg border-b border-white/10" />
      <div className="h-12 bg-white/5 rounded-lg border-b border-white/10" />
    </div>
    <div className="grid grid-cols-2 gap-8">
      <div className="h-12 bg-white/5 rounded-lg border-b border-white/10" />
      <div className="h-12 bg-white/5 rounded-lg border-b border-white/10" />
    </div>
  </div>
</div>
<div className="flex justify-center pt-12">
  <div className="h-14 w-64 bg-white/5 rounded-full border border-white/10" />
</div>
</div>
</div>
);
}
```

// 1. HUD / MACRO KPI ROW

```
const hudMetrics = [
  {
    label: "Sovereign Wealth",
    value: ledger.analytics?.totalAssets || 0,
    prefix: "KES ",
    desc: "Total Capitalized Architecture",
    chartType: "sparkline",
    chartColor: "#8b5cf6", // Purple
    chartData: [100, 110, 105, 120, 115, 130, 140],
  },
  {
    label: "Liquid Capital",
    value: ledger.analytics?.liquidity || 0,
    prefix: "KES ",
    desc: "Immediate Deployment Capacity",
    chartType: "sparkline",
    chartColor: "#3b82f6", // Blue
    chartData: [50, 45, 60, 55, 70, 65, 80],
  },
  {
    label: "Structural Burn",
    value: ledger.analytics?.totalStructuralMonthlyBurn || 0,
    prefix: "KES ",
    suffix: " /mo",
    desc: "Operational Maintenance Cost",
    chartType: "bar",
    chartColor: "#ef4444", // Red
    chartData: [20, 22, 18, 25, 20, 24, 21],
  },
];
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
    },
    {
      label: "Operational Runway",
      value: ledger.analytics?.runwayDays || 0,
      suffix: " DAYS",
      desc: "Time to Critical Depletion",
      chartType: "donut",
      chartColor: "#10b981", // Emerald
      chartData: [],
    },
  ],

  // 2. DETERMINISTIC TRAJECTORY PROJECTION (12-Month)
  const generateTrajectory = () => {
    const assets = ledger.analytics?.totalAssets || 0;
    const monthlyIncome = ledger.analytics?.totalMonthlyIncome || 0;
    const monthlyBurn = ledger.analytics?.totalStructuralMonthlyBurn || 0;
    const monthlyNet = monthlyIncome - monthlyBurn;

    const months = [
      "JAN",
      "FEB",
      "MAR",
      "APR",
      "MAY",
      "JUN",
      "JUL",
      "AUG",
      "SEP",
      "OCT",
      "NOV",
      "DEC",
    ];

    const currentMonthIndex = new Date().getMonth();

    return months.map((name, i) => {
      // Simple linear projection based on current monthly net flow
      // We start from current month and project forward
      const monthDiff =
        i >= currentMonthIndex
          ? i - currentMonthIndex
          : 12 - (currentMonthIndex - i);
      const projectedValue = Math.max(0, assets + monthlyNet * monthDiff);

      return {
        name,
        value: projectedValue,
      };
    });
  };
}
```


AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
    });
  };

  const trajectoryData = generateTrajectory();

  const containerVariants = {
    hidden: { opacity: 0 },
    show: {
      opacity: 1,
      transition: { staggerChildren: 0.15, delayChildren: 0.2 },
    },
  };

  const itemVariants = {
    hidden: { opacity: 0, y: 20 },
    show: {
      opacity: 1,
      y: 0,
      transition: { duration: 0.8, ease: "easeOut" as const },
    },
  };

  return (
    <motion.div
      variants={containerVariants}
      initial="hidden"
      animate="show"
      className="max-w-7xl mx-auto p-6 md:p-12 pb-32 space-y-24"
    >
      { /* HEADER SECTION */ }
      <motion.header
        variants={itemVariants}
        className="flex flex-col md:flex-row md:items-end justify-between gap-8 border-b border-white/5 pb-12"
      >
        <div className="space-y-4">
          <p className="font-mono text-[10px] tracking-[0.6em] text-white/20 uppercase">
            Strategic Command Center // v1.0
          </p>
          <h1 className="font-cormorant text-6xl text-white tracking-tight leading-none">
            Welcome, <span className="italic">Architect.</span>
          </h1>
        </div>
        <div className="flex gap-4">
          <button
            onClick={fetchDashboardData}
            className="px-6 py-3 border border-white/10 rounded-full font-mono text-[9px] tracking-widest uppercase hover:bg-white/5 transition-all active:scale-95"
          >
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```

        Refresh Intel
      </button>
    </div>
  </motion.header>

  { /* ZONE 1: THE HUD (Macro KPIs) */ }
  <motion.section
    id="overview"
    variants={itemVariants}
    className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-8"
  >
    {hudMetrics.map((metric, i) => (
      <LuxuryCard
        key={metric.label}
        className="p-8 group hover:border-white/20 transition-all duration-500 flex flex-col
justify-between"
      >
        <div>
          <p className="font-mono text-[9px] tracking-[0.4em] text-white/20 uppercase
group-hover:text-white/40 transition-colors">
            {metric.label}
          </p>
          <div className="space-y-1 mt-4">
            <div className="flex items-center justify-between">
              <h3 className="font-cormorant text-4xl text-white">
                <AnimatedNumber
                  value={metric.value}
                  prefix={metric.prefix}
                  suffix={metric.suffix}
                />
              </h3>
              <div>
                <p className="text-[9px] font-mono text-white/10 uppercase tracking-widest
group-hover:text-white/20 transition-colors">
                  {metric.desc}
                </p>
              </div>
            </div>
          </div>
        </div>

        { /* Chart Area */ }
        {metric.chartType === "sparkline" && (
          <div className="mt-4 -mx-2">
            <MiniSparkline
              data={metric.chartData}
              color={metric.chartColor}
            />
          </div>
        )}
      )}
    )}
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
{metric.chartType === "bar" && (
  <div className="mt-4 -mx-2">
    <MiniBarChart
      data={metric.chartData}
      color={metric.chartColor}
    />
  </div>
)}
{metric.chartType === "donut" && (
  <div className="mt-4 w-full -mx-2">
    <MiniDonut
      value={metric.value}
      max={365}
      color={metric.chartColor}
    />
  </div>
)}
</LuxuryCard>
)}}
</motion.section>

{/* ZONE 2: ARCHITECTURE DEPLOYMENT (NewEntryForm) */}
<motion.section id="deploy" variants={itemVariants} className="w-full">
  <LuxuryCard className="p-8 md:p-16 border-white/10">
    <div className="flex flex-col md:flex-row gap-12 items-start">
      <div className="space-y-4 md:w-1/3">
        <div className="w-8 h-8 rounded-full bg-white/5 flex items-center justify-center mb-6">
          <div className="w-2 h-2 rounded-full bg-white/40" />
        </div>
        <h2 className="font-cormorant text-4xl text-white">
          Architecture Deployment
        </h2>
        <p className="font-mono text-[10px] tracking-widest text-white/30 uppercase leading-relaxed">
          Log deployments, allocate capital, and dictate the flow of
          resources through your system.
        </p>
      </div>
      <div className="md:w-2/3 w-full">
        <NewEntryForm
          accounts={ledger.accounts}
          liquidity={ledger.analytics?.liquidity || 0}
          isActionLoading={isExecuting}
          onSubmit={async (data) => {
            if (isExecuting) return;
            setIsExecuting(true);
            try {
              const snapshot = await createDeploymentAction({
                title: data.title,
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
        amount: data.amount,
        category: data.category,
        accountId: data.accountId,
    });
    applyDashboardSnapshot(snapshot);
  } finally {
    setIsExecuting(false);
  }
}
}
/>
</div>
</div>
</LuxuryCard>
</motion.section>

{/* ZONE 3: GRAND TRAJECTORY */}
<motion.section
  id="intelligence"
  variants={itemVariants}
  className="space-y-12"
>
  <div className="flex items-center gap-6">
    <span className="font-cormorant italic text-3xl text-white/20">
      I.
    </span>
    <h2 className="font-cormorant text-4xl text-white tracking-wide">
      Trajectory Modeling
    </h2>
    <div className="flex-1 h-px bg-white/5" />
  </div>
  <GrandTrajectoryChart data={trajectoryData} />
</motion.section>

{/* ZONE 4: TACTICAL & STRATEGIC GRID */}
<motion.div
  id="ledger"
  variants={itemVariants}
  className="grid grid-cols-1 lg:grid-cols-[1.5fr_1fr] gap-12 lg:gap-24"
>
  {/* LEFT COLUMN: TACTICAL ENGINE */}
  <section className="space-y-12 lg:space-y-24">
    {/* PENDING ACTIONS */}
    <div className="space-y-12">
      <div className="flex items-center gap-6">
        <span className="font-cormorant italic text-3xl text-white/20">
          II.
        </span>
        <h2 className="font-cormorant text-4xl text-white tracking-wide">
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
Tactical Response
</h2>
<div className="flex-1 h-px bg-white/5" />
</div>
<div className="space-y-6">
  <PendingInflows
    incomeStreams={ledger.incomeStreams}
    accounts={ledger.accounts}
    onSnapshot={applyDashboardSnapshot}
  />
  <PendingLiabilities
    liabilities={ledger.liabilities}
    accounts={ledger.accounts}
    onSnapshot={applyDashboardSnapshot}
  />
  <PendingBaselines
    baseline={ledger.baseline}
    accounts={ledger.accounts}
    onSnapshot={applyDashboardSnapshot}
  />
</div>
</div>

<div className="space-y-8 lg:space-y-12">
  <LuxuryCard className="p-6 md:p-10">
    <IncomeSection
      incomeStreams={ledger.incomeStreams}
      onSnapshot={applyDashboardSnapshot}
    />
  </LuxuryCard>
  <LuxuryCard className="p-6 md:p-10">
    <BaselineSection
      baseline={ledger.baseline}
      onSnapshot={applyDashboardSnapshot}
    />
  </LuxuryCard>
</div>

<LuxuryCard className="p-6 md:p-10">
  <LiabilitySection
    liabilities={ledger.liabilities}
    onSnapshot={applyDashboardSnapshot}
  />
</LuxuryCard>

<LuxuryCard className="p-6 md:p-10">
  <AccountSection
    accounts={ledger.accounts}
  />
</LuxuryCard>
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
        onSnapshot={applyDashboardSnapshot}
      />
    </LuxuryCard>
  </section>

  { /* RIGHT COLUMN: STRATEGIC ENGINE */ }
  <section className="space-y-12 lg:space-y-24">
    <div className="space-y-12">
      <div className="flex items-center gap-6">
        <span className="font-cormorant italic text-3xl text-white/20">
          III.
        </span>
        <h2 className="font-cormorant text-4xl text-white tracking-wide">
          Strategic Alignment
        </h2>
        <div className="flex-1 h-px bg-white/5" />
      </div>
      <LuxuryCard className="p-6 md:p-10">
        <StrategicObjectiveSection
          objectives={ledger.objectives}
          onSnapshot={applyDashboardSnapshot}
        />
      </LuxuryCard>
    </div>

    <LuxuryCard className="p-6 md:p-10">
      <GoalSection
        goals={ledger.goals}
        onSnapshot={applyDashboardSnapshot}
      />
    </LuxuryCard>

    <LuxuryCard className="p-6 md:p-10">
      <HistoricalAudit deployments={ledger.deployments} />
    </LuxuryCard>

  { /* OBSERVABILITY */ }
  <section className="opacity-20 hover:opacity-100 transition-opacity duration-1000">
    <div className="flex items-center gap-6 mb-12">
      <span className="font-cormorant italic text-3xl text-white/20">
        IV.
      </span>
      <h2 className="font-cormorant text-4xl text-white tracking-wide">
        System Telemetry
      </h2>
      <div className="flex-1 h-px bg-white/5" />
    </div>
    <TelemetryDashboard />
  </section>
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
        </section>
      </section>
    </motion.div>
  </motion.div>
);
}
```

FILE: src/app/dashboard\PendingBaselines.tsx

```
"use client";

import { useState } from "react";
import type { OperationalBaseline, Account } from "@lib/analytics/types";
import {
  resolvePendingBaselineAction,
  type DashboardSnapshot,
} from "../actions";
import { formatCurrency } from "@lib/utils/formatters";

interface PendingBaselinesProps {
  baseline: OperationalBaseline[];
  accounts: Account[];
  onSnapshot: (snapshot: DashboardSnapshot) => void;
}

export function PendingBaselines({
  baseline,
  accounts,
  onSnapshot,
}: PendingBaselinesProps) {
  const [loadingId, setLoadingId] = useState<string | null>(null);
  const [selectedAccounts, setSelectedAccounts] = useState<
    Record<string, string>
  >({});

  const today = new Date();
  const currentDayOfWeek = today.getDay() === 0 ? 7 : today.getDay();
  const currentDayOfMonth = today.getDate();

  const pendingBaselines = baseline.filter((b) => {
    if (!b.is_recurring) return false;

    // Check if already executed today
    if (b.last_executed_at) {
      const lastExecuted = new Date(b.last_executed_at);
      if (
        lastExecuted.getDate() === today.getDate() &&
        lastExecuted.getMonth() === today.getMonth() &&

```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
        lastExecuted.getFullYear() === today.getFullYear()
    ) {
        return false;
    }
}

// Daily baselines are always pending if not executed today
if (b.cadence === "daily") return true;

if (b.execution_day) {
    if (b.cadence === "weekly") {
        return currentDayOfWeek === b.execution_day;
    }
    if (b.cadence === "monthly" || b.cadence === "biweekly") {
        return currentDayOfMonth === b.execution_day;
    }
}

return false;
});

if (pendingBaselines.length === 0) return null;

async function handleVerify(item: OperationalBaseline) {
    const accountId = selectedAccounts[item.id];
    if (!accountId) {
        alert("Please select a source account.");
        return;
    }

    setLoadingId(item.id);
    try {
        const snapshot = await resolvePendingBaselineAction({
            baselineId: item.id,
            accountId,
            amount: item.amount,
            title: item.title,
        });
        onSnapshot(snapshot);
    } catch (err: unknown) {
        alert(err instanceof Error ? err.message : "Verification failed");
    } finally {
        setLoadingId(null);
    }
}

return (
    <div className="space-y-6 animate-in fade-in duration-700">
```


AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
<div className="flex items-center gap-4">
  <div className="w-2 h-2 rounded-full bg-red-500 animate-pulse"></div>
  <h2 className="text-[10px] font-mono text-white/60 uppercase tracking-[0.4em]">
    Baseline Verification Required
  </h2>
</div>

{pendingBaselines.map((item) => (
  <div
    key={item.id}
    className="bg-red-500/10 border border-red-500/20 p-8 md:p-12 flex flex-col md:flex-row
md:items-center justify-between gap-8 text-white"
  >
    <div className="space-y-2">
      <p className="text-[10px] font-mono tracking-widest uppercase text-red-500/60">
        Structural Maintenance Prompt ({item.cadence})
      </p>
      <h3 className="font-cormorant text-4xl">
        {formatCurrency(item.amount)}
      </h3>
      <p className="text-xs font-light tracking-wide opacity-80">
        {item.title}
      </p>
    </div>

    <div className="flex flex-col sm:flex-row items-stretch sm:items-center gap-4">
      <select
        disabled={loadingId === item.id}
        value={selectedAccounts[item.id] || ""}
        onChange={(e) =>
          setSelectedAccounts({
            ...selectedAccounts,
            [item.id]: e.target.value,
          })
        }
        className="bg-transparent border-b border-white/40 py-2 font-mono text-[10px] tracking-widest
uppercase focus:outline-none focus:border-white transition-colors min-w-50"
      >
        <option value="" className="text-black">
          Source Account
        </option>
        {accounts.map((acc) => (
          <option key={acc.id} value={acc.id} className="text-black">
            {acc.account_name}
          </option>
        ))}
      </select>
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
        <button
          disabled={loadingId === item.id}
          onClick={() => handleVerify(item)}
          className="bg-white text-black px-10 py-4 font-medium tracking-widest uppercase text-[10px]
hover:bg-white/90 transition-all disabled:opacity-50"
        >
          {loadingId === item.id ? "VERIFYING..." : "CONFIRM SETTLEMENT"}
        </button>
      </div>
    </div>
  )}
</div>
);
}
```

FILE: src/app/dashboard\PendingInflows.tsx

```
"use client";

import { useState } from "react";
import type { IncomeStream, Account } from "@lib/analytics/types";
import { resolvePendingInflowAction, type DashboardSnapshot } from "../actions";
import { formatCurrency } from "@lib/utils/formatters";
import { IncomeMap } from "@lib/utils/taxonomy";

interface PendingInflowsProps {
  incomeStreams: IncomeStream[];
  accounts: Account[];
  onSnapshot: (snapshot: DashboardSnapshot) => void;
}

export function PendingInflows({
  incomeStreams,
  accounts,
  onSnapshot,
}: PendingInflowsProps) {
  const [loadingId, setLoadingId] = useState<string | null>(null);
  const [selectedAccounts, setSelectedAccounts] = useState<
    Record<string, string>
  >({});

  const today = new Date();
  const currentDayOfWeek = today.getDay();
  const currentDayOfMonth = today.getDate();
  const mappedDayOfWeek = currentDayOfWeek === 0 ? 7 : currentDayOfWeek;

  const pendingInflows = incomeStreams.filter((stream) => {
    if (!stream.is_recurring || !stream.execution_day) return false;
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
if (stream.last_executed_at) {
  const lastExecuted = new Date(stream.last_executed_at);
  if (
    lastExecuted.getDate() === today.getDate() &&
    lastExecuted.getMonth() === today.getMonth() &&
    lastExecuted.getFullYear() === today.getFullYear()
  ) {
    return false;
  }
}

if (stream.cadence === "weekly") {
  return mappedDayOfWeek === stream.execution_day;
}
if (stream.cadence === "monthly" || stream.cadence === "biweekly") {
  return currentDayOfMonth === stream.execution_day;
}

return false;
});

if (pendingInflows.length === 0) return null;

async function handleVerify(stream: IncomeStream) {
  const accountId = selectedAccounts[stream.id];
  if (!accountId) {
    alert("Please select a destination account.");
    return;
  }

  setLoadingId(stream.id);
  try {
    const snapshot = await resolvePendingInflowAction({
      incomeId: stream.id,
      accountId,
      amount: stream.amount,
    });
    onSnapshot(snapshot);
  } catch (err: unknown) {
    alert(err instanceof Error ? err.message : "Verification failed");
  } finally {
    setLoadingId(null);
  }
}

return (
  <div className="space-y-6 animate-in fade-in duration-700">
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
<div className="flex items-center gap-4">
  <div className="w-2 h-2 rounded-full bg-white animate-pulse"></div>
  <h2 className="text-[10px] font-mono text-white/60 uppercase tracking-[0.4em]">
    Verification Required
  </h2>
</div>

{pendingInflows.map((stream) => (
  <div
    key={stream.id}
    className="bg-white p-8 md:p-12 flex flex-col md:flex-row md:items-center justify-between gap-8
text-black"
  >
    <div className="space-y-2">
      <p className="text-[10px] font-mono tracking-widest uppercase opacity-40">
        Expected Income Detected
      </p>
      <h3 className="font-cormorant text-4xl">
        {formatCurrency(stream.amount)}
      </h3>
      <p className="text-xs font-light tracking-wide opacity-60">
        {stream.income_name} (
        {IncomeMap[stream.income_type] || stream.income_type})
      </p>
    </div>

    <div className="flex flex-col sm:flex-row items-stretch sm:items-center gap-4">
      <select
        disabled={loadingId === stream.id}
        value={selectedAccounts[stream.id] || ""}
        onChange={(e) =>
          setSelectedAccounts({
            ...selectedAccounts,
            [stream.id]: e.target.value,
          })
        }
        className="bg-transparent border-b border-black/20 py-2 font-mono text-[10px] tracking-widest
uppercase focus:outline-none focus:border-black transition-colors min-w-50"
      >
        <option value="">Destination Account</option>
        {accounts.map((acc) => (
          <option key={acc.id} value={acc.id}>
            {acc.account_name}
          </option>
        ))}
      </select>

      <button
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```

disabled={loadingId === stream.id}
onClick={() => handleVerify(stream)}
      className="bg-black text-white px-10 py-4 font-medium tracking-widest uppercase text-[10px]
hover:bg-black/90 transition-all disabled:opacity-50"
    >
      {loadingId === stream.id ? "VERIFYING..." : "CONFIRM RECEIPT"}
    </button>
  </div>
</div>
  )}}
</div>
);
}

```

FILE: src/app/dashboard\PendingLiabilities.tsx

```
"use client";

import { useState } from "react";
import type { Liability, Account } from "@lib/analytics/types";
import {
  resolvePendingLiabilityAction,
  type DashboardSnapshot,
} from "../actions";
import { formatCurrency } from "@lib/utils/formatters";
import { LiabilityMap } from "@lib/utils/taxonomy";

interface PendingLiabilitiesProps {
  liabilities: Liability[];
  accounts: Account[];
  onSnapshot: (snapshot: DashboardSnapshot) => void;
}

export function PendingLiabilities({
  liabilities,
  accounts,
  onSnapshot,
}: PendingLiabilitiesProps) {
  const [loadingId, setLoadingId] = useState<string | null>(null);
  const [selectedAccounts, setSelectedAccounts] = useState<
    Record<string, string>
  >({});

  const today = new Date();
  const currentDayOfWeek = today
    .toLocaleString("en-US", { weekday: "long" })
    .toLowerCase();

  const currentDayOfMonth = today.getDate();
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
const pendingLiabilities = liabilities.filter((liab) => {
  if (!liab.is_paid_in_cadences || !liab.cadence_day_date) return false;

  // Check if already executed today
  if (liab.last_executed_at) {
    const lastExecuted = new Date(liab.last_executed_at);
    if (
      lastExecuted.getDate() === today.getDate() &&
      lastExecuted.getMonth() === today.getMonth() &&
      lastExecuted.getFullYear() === today.getFullYear()
    ) {
      return false;
    }
  }

  if (liab.cadence === "weekly") {
    return currentDayOfWeek === liab.cadence_day_date.toLowerCase();
  }
  if (liab.cadence === "monthly") {
    return currentDayOfMonth === parseInt(liab.cadence_day_date, 10);
  }

  return false;
});

if (pendingLiabilities.length === 0) return null;

async function handleVerify(liab: Liability) {
  const accountId = selectedAccounts[liab.id];
  if (!accountId) {
    alert("Please select a source account.");
    return;
  }

  setLoadingId(liab.id);
  try {
    const snapshot = await resolvePendingLiabilityAction({
      liabilityId: liab.id,
      accountId,
      amount: liab.cadence_amount || 0,
    });
    onSnapshot(snapshot);
  } catch (err: unknown) {
    alert(err instanceof Error ? err.message : "Verification failed");
  } finally {
    setLoadingId(null);
  }
}
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
}

return (
  <div className="space-y-6 animate-in fade-in duration-700">
    <div className="flex items-center gap-4">
      <div className="w-2 h-2 rounded-full bg-red-500 animate-pulse"></div>
      <h2 className="text-[10px] font-mono text-white/60 uppercase tracking-[0.4em]">
        Payment Verification Required
      </h2>
    </div>

    {pendingLiabilities.map((liab) => (
      <div
        key={liab.id}
        className="bg-red-500 p-8 md:p-12 flex flex-col md:flex-row md:items-center justify-between gap-8
text-white"
      >
        <div className="space-y-2">
          <p className="text-[10px] font-mono tracking-widest uppercase opacity-60">
            Scheduled Obligation Detected
          </p>
          <h3 className="font-cormorant text-4xl">
            {formatCurrency(liab.cadence_amount || 0)}
          </h3>
          <p className="text-xs font-light tracking-wide opacity-80">
            {liab.liability_name} (
            {LiabilityMap[liab.liability_type] || liab.liability_type})
          </p>
        </div>

        <div className="flex flex-col sm:flex-row items-stretch sm:items-center gap-4">
          <select
            disabled={loadingId === liab.id}
            value={selectedAccounts[liab.id] || ""}
            onChange={(e) =>
              setSelectedAccounts({
                ...selectedAccounts,
                [liab.id]: e.target.value,
              })
            }
            className="bg-transparent border-b border-white/40 py-2 font-mono text-[10px] tracking-widest
uppercase focus:outline-none focus:border-white transition-colors min-w-50"
          >
            <option value="" className="text-black">
              Source Account
            </option>
            {accounts.map((acc) => (
              <option key={acc.id} value={acc.id} className="text-black">
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
        {acc.account_name}
      </option>
    )}}
  </select>

  <button
    disabled={loadingId === liab.id}
    onClick={() => handleVerify(liab)}
    className="bg-white text-red-500 px-10 py-4 font-medium tracking-widest uppercase text-[10px]
hover:bg-white/90 transition-all disabled:opacity-50"
  >
    {loadingId === liab.id ? "VERIFYING..." : "CONFIRM SETTLEMENT"}
  </button>
</div>
</div>
)}}
</div>
);
}
```

FILE: src/app/dashboard\RunwayCard.tsx

```
"use client";

import React from "react";

interface RunwayCardProps {
  runwayDays: number | null;
  netWorth?: number;
}

export function RunwayCard({ runwayDays, netWorth = 0 }: RunwayCardProps) {
  // 1. Logic for determining visual state
  const isInfiniteRunway = runwayDays === null;
  const isCritical = !isInfiniteRunway && runwayDays! < 30;
  const isInsolvent = netWorth < 0;

  // 2. Formatting
  const displayValue = isInfiniteRunway
    ? isInsolvent
      ? "INSOLVENT STABLE"
      : "SYSTEM STABLE"
    : `${(runwayDays! / 30).toFixed(1)} MONTHS`;

  // 3. Conditional Styles
  const containerClasses = `bg-foreground/5 border rounded-2xl p-6 md:p-8 flex flex-col justify-between
transition-all duration-500 ${
    isInfiniteRunway
```


AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
? isInsolvent
  ? "border-rose-500/20"
  : "border-emerald-500/10"
: isCritical
  ? "border-rose-500/30 bg-rose-500/5 shadow-inner"
  : "border-foreground/10"
} `;

const labelClasses = `text-[11px] font-black uppercase tracking-[0.2em] mb-4 ${
  isInfiniteRunway
  ? isInsolvent
    ? "text-rose-500/50"
    : "text-emerald-500/50"
  : isCritical
    ? "text-rose-500/60"
    : "text-foreground/40"
} `;

const valueClasses = `text-4xl font-black tabular-nums transition-colors duration-500 ${
  isInfiniteRunway
  ? isInsolvent
    ? "text-rose-500/80 font-mono tracking-tighter"
    : "text-emerald-500/80 font-mono tracking-tighter"
  : isCritical
    ? "text-rose-600"
    : "text-foreground"
} `;

const description = isInfiniteRunway
  ? isInsolvent
    ? "Burn is absorbed by incomes, but total commitments exceed assets."
    : "Net inbound capital completely absorbs current structural burn."
  : isCritical
    ? "A recalibration of The Foundation is recommended."
    : "Duration of sustained operation based on current structural burn and liquidity.";

return (
  <div className={containerClasses}>
    <div className="flex items-center justify-between mb-4">
      <span className={labelClasses}>
        {isInfiniteRunway
          ? isInsolvent
            ? "Insolvent Stability"
            : "Infinite Stability"
          : "Operating Horizon"}
      </span>
      {isInfiniteRunway && !isInsolvent && (
        <div className="flex items-center gap-1.5">
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
        <span className="w-1.5 h-1.5 bg-emerald-500/40 rounded-full animate-pulse"></span>
        <span className="text-[8px] font-black text-emerald-500/40 uppercase tracking-widest">
          Active Protection
        </span>
      </div>
    )}
    {isInfiniteRunway && isInsolvent && (
      <div className="flex items-center gap-1.5">
        <span className="w-1.5 h-1.5 bg-rose-500/40 rounded-full animate-pulse"></span>
        <span className="text-[8px] font-black text-rose-500/40 uppercase tracking-widest">
          Solvency Risk
        </span>
      </div>
    )}
  </div>

  <div className="flex flex-col">
    <span className={valueClasses}>{displayValue}</span>
    <p
      className={`text-[10px] font-bold uppercase tracking-widest mt-4 ${
        isInfiniteRunway ? "text-emerald-600/40" : "text-foreground/40"
      }`}
    >
      {description}
    </p>
  </div>
</div>
);
}
```

FILE: src/app/dashboard\StrategicObjectiveSection.tsx

```
"use client";

import { useState } from "react";
import {
  calculateObjectiveFundingRatio,
  type StrategicObjective,
  OBJECTIVE_TYPES,
  OBJECTIVE_PRIORITIES,
  OBJECTIVE_STATUSES,
} from "@lib/finance/objectives";
import {
  createStrategicObjectiveAction,
  updateStrategicObjectiveAction,
  type DashboardSnapshot,
} from "../actions";
import { formatCurrency } from "@lib/utils/formatters";
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
import { ScrollReveal } from "@components/ui/scroll-reveal";
import { motion } from "framer-motion";

interface StrategicObjectiveSectionProps {
  objectives: StrategicObjective[];
  onSnapshot: (snapshot: DashboardSnapshot) => void;
}

export function StrategicObjectiveSection({
  objectives,
  onSnapshot,
}: StrategicObjectiveSectionProps) {
  const [isAdding, setIsAdding] = useState(false);
  const [editingId, setEditingId] = useState<string | null>(null);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const [form, setForm] = useState({
    objective_name: "",
    objective_type: "emergency_reserve",
    target_amount: "",
    current_amount: "",
    priority_level: "moderate",
    status: "active",
    target_date: "",
    notes: "",
  });

  async function handleSubmit(e: React.FormEvent) {
    e.preventDefault();
    setIsLoading(true);
    setError(null);
    try {
      let snapshot;
      if (editingId) {
        snapshot = await updateStrategicObjectiveAction(editingId, {
          objective_name: form.objective_name,
          objective_type: form.objective_type,
          target_amount: Number(form.target_amount),
          current_amount: form.current_amount ? Number(form.current_amount) : 0,
          priority_level: form.priority_level,
          status: form.status,
          target_date: form.target_date || null,
          notes: form.notes || undefined,
        });
      } else {
        snapshot = await createStrategicObjectiveAction({
          objective_name: form.objective_name,
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
        objective_type: form.objective_type,
        target_amount: Number(form.target_amount),
        current_amount: form.current_amount ? Number(form.current_amount) : 0,
        priority_level: form.priority_level,
        status: form.status,
        target_date: form.target_date || null,
        notes: form.notes || undefined,
    });
}
setForm({
    objective_name: "",
    objective_type: "emergency_reserve",
    target_amount: "",
    current_amount: "",
    priority_level: "moderate",
    status: "active",
    target_date: "",
    notes: "",
});
setIsAdding(false);
setEditingId(null);
onSnapshot(snapshot);
} catch (err: unknown) {
    setError(
        err instanceof Error
        ? err.message
        : "Failed to establish strategic intent",
    );
} finally {
    setIsLoading(false);
}
}

return (
    <div className="space-y-12">
        <div className="flex items-center justify-between">
            <h2 className="font-cormorant text-2xl text-white tracking-wide uppercase">
                Strategic Intent
            </h2>
            <button
                onClick={() => setIsAdding(!isAdding)}
                className="font-mono text-[9px] tracking-[0.4em] uppercase text-white/40 hover:text-white
transition-colors"
            >
                {isAdding ? "? CANCEL" : "+ APPEND OBJECTIVE"}
            </button>
        </div>
    </div>
);
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
{isAdding && (
  <div className="bg-white/2 border border-white/5 p-8 animate-in fade-in slide-in-from-top-4
duration-500">
    <form onSubmit={handleSubmit} className="space-y-8">
      <div className="grid grid-cols-1 md:grid-cols-2 gap-8">
        <div className="space-y-2">
          <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
            Objective Name
          </label>
          <input
            type="text"
            required
            placeholder="e.g. Emergency Fund"
            value={form.objective_name}
            onChange={(e) =>
              setForm({ ...form, objective_name: e.target.value })
            }
            className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light
focus:outline-none focus:border-white transition-colors"
          />
        </div>
        <div className="space-y-2">
          <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
            Objective Type
          </label>
          <select
            value={form.objective_type}
            onChange={(e) =>
              setForm({ ...form, objective_type: e.target.value })
            }
            className="w-full bg-transparent border-b border-white/10 py-3 text-white/60 font-mono
text-[10px] tracking-widest uppercase focus:outline-none"
          >
            {OBJECTIVE_TYPES.map((type) => (
              <option
                key={type.value}
                value={type.value}
                className="bg-black"
              >
                {type.label}
              </option>
            ))}
          </select>
        </div>
      </div>

      <div className="grid grid-cols-1 md:grid-cols-2 gap-8">
        <div className="space-y-2">
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
<label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
  Priority Level
</label>
<select
  value={form.priority_level}
  onChange={(e) =>
    setForm({ ...form, priority_level: e.target.value })
  }
  className="w-full bg-transparent border-b border-white/10 py-3 text-white/60 font-mono
text-[10px] tracking-widest uppercase focus:outline-none"
>
  {OBJECTIVE_PRIORITIES.map((p) => (
    <option key={p.value} value={p.value} className="bg-black">
      {p.label}
    </option>
  ))}
</select>
</div>
<div className="space-y-2">
  <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
    Status
  </label>
  <select
    value={form.status}
    onChange={(e) => setForm({ ...form, status: e.target.value })}
    className="w-full bg-transparent border-b border-white/10 py-3 text-white/60 font-mono
text-[10px] tracking-widest uppercase focus:outline-none"
  >
    {OBJECTIVE_STATUSES.map((s) => (
      <option key={s.value} value={s.value} className="bg-black">
        {s.label}
      </option>
    ))}
  </select>
</div>
</div>

<div className="grid grid-cols-1 md:grid-cols-2 gap-8">
  <div className="space-y-2">
    <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
      Target Amount
    </label>
    <input
      type="number"
      required
      placeholder="0.00"
      value={form.target_amount}
      onChange={(e) =>
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
        setForm({ ...form, target_amount: e.target.value })
    }

    className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light
focus:outline-none focus:border-white transition-colors"
  />
</div>
<div className="space-y-2">
  <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
    Current Position
  </label>
  <input
    type="number"
    placeholder="0.00"
    value={form.current_amount}
    onChange={(e) =>
      setForm({ ...form, current_amount: e.target.value })
    }
    className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light
focus:outline-none focus:border-white transition-colors"
  />
</div>
</div>

<div className="grid grid-cols-1 md:grid-cols-2 gap-8">
  <div className="space-y-2">
    <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
      Target Date
    </label>
    <input
      type="date"
      value={form.target_date}
      onChange={(e) =>
        setForm({ ...form, target_date: e.target.value })
      }
      className="w-full bg-transparent border-b border-white/10 py-3 text-white/60 font-mono
text-[10px] tracking-widest uppercase focus:outline-none"
    />
  </div>
  <div className="space-y-2">
    <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
      Strategic Notes
    </label>
    <input
      type="text"
      placeholder="Add context..."
      value={form.notes}
      onChange={(e) => setForm({ ...form, notes: e.target.value })}
      className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
focus:outline-none focus:border-white transition-colors"
    />
  </div>
</div>

{error && (
  <p className="text-red-500 font-mono text-[9px] uppercase tracking-widest">
    {error}
  </p>
)}

<button
  type="submit"
  disabled={isLoading}
  className="w-full bg-white text-black py-4 font-medium tracking-[0.2em] uppercase text-[10px]
hover:bg-white/90 transition-colors disabled:opacity-50"
  >
  {isLoading ? "ESTABLISHING..." : "SET STRATEGIC INTENT"}
</button>
</form>
</div>
)}

<div className="space-y-8">
  {objectives.length === 0 ? (
    <div className="py-12 text-center opacity-20">
      <p className="text-[10px] font-mono uppercase tracking-[0.5em]">
        No objectives active
      </p>
    </div>
  ) : (
    objectives.map((obj, index) => {
      const ratio = calculateObjectiveFundingRatio(obj);
      return (
        <ScrollReveal key={obj.id} delay={index * 0.1}>
          <div className="space-y-4 group cursor-default pb-8 border-b border-white/5 last:border-0">
            <div className="flex items-center justify-between mb-2">
              <div className="space-y-1">
                <span
                  className={`text-[8px] font-mono tracking-widest uppercase ${
                    obj.priority_level === "critical"
                      ? "text-red-500"
                      : "text-white/20"
                  }`>
                </span>
              </div>
              <div>
                {obj.priority_level}
              </div>
            </div>
          </div>
        </ScrollReveal>
      )
    })
  )
}

<h3 className="font-cormorant text-2xl text-white transition-transform
```


AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
group-hover:translate-x-2">
    {obj.objective_name}
  </h3>
</div>
<div className="text-right">
  <p className="font-cormorant text-3xl text-white tabular-nums">
    {Math.round(ratio)}%
  </p>
</div>
</div>

<div className="h-px w-full bg-white/5 relative overflow-hidden">
  <motion.div
    initial={{ width: 0 }}
    whileInView={{ width: `${Math.min(100, ratio)}%` }}
    viewport={{ once: true }}
    transition={{
      duration: 1.5,
      delay: 0.5 + index * 0.1,
      ease: "easeOut",
    }}
    className="absolute inset-y-0 left-0 bg-white/60"
  />
</div>

    <div className="flex justify-between items-end pt-2 opacity-30 group-hover:opacity-60
transition-opacity">
      <p className="text-[9px] font-mono uppercase tracking-widest">
        {formatCurrency(obj.current_amount)}
      </p>
      <p className="text-[9px] font-mono uppercase tracking-widest">
        Target: {formatCurrency(obj.target_amount)}
      </p>
    </div>
  </div>
</ScrollReveal>
  );
})
})
</div>
</div>
);
}
```

FILE: src/lib/db/accounts.ts

```
import type { SupabaseClient } from "@supabase/supabase-js";
import { validateAccount, type AccountType } from "../finance/accounts";
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
export async function getAccounts(supabase: SupabaseClient) {
  const { data, error } = await supabase
    .from("accounts")
    .select("*")
    .is("deleted_at", null)
    .order("account_name", { ascending: true });

  if (error) throw error;
  return data;
}
```

```
export async function createAccount(
  supabase: SupabaseClient,
  userId: string,
  data: {
    account_name: string;
    account_type: string;
    current_balance: number;
    institution?: string;
    currency?: string;
    deleted_at?: string | null;
  },
) {
  const validated = validateAccount({
    account_name: data.account_name,
    account_type: data.account_type,
    current_balance: data.current_balance,
  });

  const { data: account, error } = await supabase
    .from("accounts")
    .insert({
      user_id: userId,
      account_name: validated.account_name,
      account_type: validated.account_type,
      current_balance: validated.current_balance,
      institution: data.institution,
      currency: data.currency || "KSh",
    })
    .select()
    .single();

  if (error) throw error;
  return account;
}
```

```
export async function updateAccount(
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
supabase: SupabaseClient,
id: string,
userId: string,
updates: {
  account_name?: string;
  account_type?: string;
  current_balance?: number;
  institution?: string;
  deleted_at?: string | null;
},
) {
  const dbUpdates: Record<string, unknown> = { ...updates };

  if (
    updates.account_name !== undefined ||
    updates.account_type !== undefined ||
    updates.current_balance !== undefined
  ) {
    // Basic validation for what's provided
    if (
      updates.account_name !== undefined &&
      updates.account_name.trim().length === 0
    ) {
      throw new Error("Account name is required.");
    }
    // Note: We don't use full validateAccount here to allow partial updates
  }

  const { data, error } = await supabase
    .from("accounts")
    .update(dbUpdates)
    .eq("id", id)
    .eq("user_id", userId)
    .select()
    .single();

  if (error) throw error;
  return data;
}

export async function deleteAccount(
  supabase: SupabaseClient,
  id: string,
  userId: string,
) {
  const { error } = await supabase
    .from("accounts")
    .update({ deleted_at: new Date().toISOString() })
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
.eq("id", id)
.eq("user_id", userId);

if (error) throw error;
return true;
}
```

FILE: src/lib/db\audit.ts

```
import type { SupabaseClient } from "@supabase/supabase-js";

export type AuditRecordType = "deployment" | "income" | "liability";

export interface AuditRecord {
  id: string;
  type: AuditRecordType;
  name: string;
  amount: number;
  cadence?: string;
  deleted_at: string;
  original_created_at: string;
}

/**
 * Retrieves all soft-deleted records from the ledger tables (deployments, income_streams, liabilities).
 * Returns a unified, chronologically sorted timeline of deletions.
 */
export async function getDeletedLedgerRecords(
  supabase: SupabaseClient,
  userId: string,
): Promise<AuditRecord[]> {
  const [deployments, incomeStreams, liabilities] = await Promise.all([
    supabase
      .from("deployments")
      .select("id, title, amount, deleted_at, created_at")
      .eq("user_id", userId)
      .not("deleted_at", "is", null),
    supabase
      .from("income_streams")
      .select("id, income_name, amount, cadence, deleted_at, created_at")
      .eq("user_id", userId)
      .not("deleted_at", "is", null),
    supabase
      .from("liabilities")
      .select("id, liability_name, outstanding_balance, deleted_at, created_at")
      .eq("user_id", userId)
      .not("deleted_at", "is", null),
  ]);
}
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
if (deployments.error) throw deployments.error;
if (incomeStreams.error) throw incomeStreams.error;
if (liabilities.error) throw liabilities.error;

const normalizedDeployments: AuditRecord[] = (deployments.data || []).map(
  (d) => ({
    id: d.id,
    type: "deployment",
    name: d.title,
    amount: d.amount,
    deleted_at: d.deleted_at!,
    original_created_at: d.created_at,
  })),
);

const normalizedIncome: AuditRecord[] = (incomeStreams.data || []).map(
  (i) => ({
    id: i.id,
    type: "income",
    name: i.income_name,
    amount: i.amount,
    cadence: i.cadence,
    deleted_at: i.deleted_at!,
    original_created_at: i.created_at,
  })),
);

const normalizedLiabilities: AuditRecord[] = (liabilities.data || []).map(
  (l) => ({
    id: l.id,
    type: "liability",
    name: l.liability_name,
    amount: l.outstanding_balance,
    deleted_at: l.deleted_at!,
    original_created_at: l.created_at,
  })),
);

return [
  ...normalizedDeployments,
  ...normalizedIncome,
  ...normalizedLiabilities,
].sort(
  (a, b) =>
    new Date(b.deleted_at).getTime() - new Date(a.deleted_at).getTime(),
);
}
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

FILE: src/lib/db\baseline.ts

```
import type { SupabaseClient } from "@supabase/supabase-js";
import { OperationalBaseline } from "../analytics/types";

export async function getOperationalBaseline(supabase: SupabaseClient) {
  const { data, error } = await supabase
    .from("operational_baseline")
    .select("*")
    .is("deleted_at", null)
    .order("created_at", { ascending: true });

  if (error) throw error;
  return data as OperationalBaseline[];
}

export async function createOperationalBaseline(
  supabase: SupabaseClient,
  userId: string,
  data: Omit<
    OperationalBaseline,
    "id" | "user_id" | "created_at" | "updated_at" | "deleted_at"
  >,
) {
  const { data: baseline, error } = await supabase
    .from("operational_baseline")
    .insert({
      user_id: userId,
      ...data,
    })
    .select()
    .single();

  if (error) throw error;
  return baseline as OperationalBaseline;
}

export async function updateOperationalBaseline(
  supabase: SupabaseClient,
  id: string,
  userId: string,
  updates: Partial<
    Omit<
      OperationalBaseline,
      "id" | "user_id" | "created_at" | "updated_at" | "deleted_at"
    >
  >,
) {
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
const { data: baseline, error } = await supabase
  .from("operational_baseline")
  .update(updates)
  .eq("id", id)
  .eq("user_id", userId)
  .select()
  .single();

if (error) throw error;
return baseline as OperationalBaseline;
}

export async function deleteOperationalBaseline(
  supabase: SupabaseClient,
  id: string,
  userId: string,
) {
  const { error } = await supabase
    .from("operational_baseline")
    .update({ deleted_at: new Date().toISOString() })
    .eq("id", id)
    .eq("user_id", userId);

  if (error) throw error;
  return true;
}
```

FILE: src/lib/db/deployments.ts

```
import type { SupabaseClient } from "@supabase/supabase-js";
import { validateDeployment } from "../finance/validateDeployment";
import {
  hasDeploymentAdvancedContext,
  type DeploymentAdvancedContextInput,
} from "../finance/deploymentContext";

function isMissingAdvancedContextColumn(error: { message?: string }) {
  return Boolean(error.message?.toLowerCase().includes("advanced_context"));
}

export async function getDeployments(supabase: SupabaseClient) {
  const { data, error } = await supabase
    .from("deployments")
    .select("*")
    .is("deleted_at", null)
    .order("created_at", { ascending: false });

  if (error) throw error;
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
    return data;
}

/**
 * Creates a new deployment record.
 * Guaranteed to pass through the Taxonomy Enforcement Gate.
 */
export async function createDeployment(
  supabase: SupabaseClient,
  title: string,
  amount: number,
  userId: string,
  category: string,
  impactScore: number = 0,
  advancedContext: DeploymentAdvancedContextInput = {},
  accountId?: string,
) {
  // 1. Pass through Taxonomy Enforcement Gate
  const validated = validateDeployment({
    title,
    amount,
    category,
    impactScore,
    advancedContext,
  });

  // 2. Database Write (Verified Truth)
  const deploymentRecord = {
    title: validated.title,
    amount: validated.amount,
    user_id: userId,
    category: validated.category,
    impact_score: validated.impactScore,
    account_id: accountId || null,
    ...(hasDeploymentAdvancedContext(validated.advancedContext)
      ? { advanced_context: validated.advancedContext }
      : {}),
  };

  const { data, error } = await supabase
    .from("deployments")
    .insert(deploymentRecord);

  if (error) {
    if (
      hasDeploymentAdvancedContext(validated.advancedContext) &&
      isMissingAdvancedContextColumn(error)
    ) {

```


AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
        throw new Error(
            "Advanced Context schema unavailable. Apply supabase-setup.sql before saving advanced deployment
metadata.",
        );
    }

    throw error;
}
return data;
}

/**
 * Updates an existing deployment record.
 * Enforces partial taxonomy validation for updated fields.
 */
export async function updateDeployment(
    supabase: SupabaseClient,
    id: string,
    userId: string,
    updates: {
        title?: string;
        amount?: number;
        category?: string;
        impact_score?: number;
        advancedContext?: DeploymentAdvancedContextInput;
        accountId?: string;
    },
) {
    // 1. Enforcement for provided fields
    const dbUpdates: Record<string, unknown> = {};

    if (updates.accountId !== undefined) {
        dbUpdates.account_id = updates.accountId || null;
    }

    const shouldValidate =
        updates.title !== undefined ||
        updates.amount !== undefined ||
        updates.category !== undefined ||
        updates.advancedContext !== undefined ||
        updates.impact_score !== undefined;

    if (shouldValidate) {
        // We simulate a full input to reuse the Gate logic,
        // but only use the validated fields for the update.
        const validated = validateDeployment({
            title: updates.title ?? "Temporary Title",
            amount: updates.amount ?? 1,
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
    category: updates.category ?? "Leakage", // Explicit default for gate reuse
    impactScore: updates.impact_score,
    advancedContext: updates.advancedContext,
  });

  if (updates.title !== undefined) dbUpdates.title = validated.title;
  if (updates.amount !== undefined) dbUpdates.amount = validated.amount;
  if (updates.category !== undefined) dbUpdates.category = validated.category;
  if (updates.impact_score !== undefined)
    dbUpdates.impact_score = validated.impactScore;
  if (updates.advancedContext !== undefined)
    dbUpdates.advanced_context = validated.advancedContext;
}

// 2. Database Update
const { data, error } = await supabase
  .from("deployments")
  .update(dbUpdates)
  .eq("id", id)
  .eq("user_id", userId);

if (error) {
  if (
    updates.advancedContext !== undefined &&
    isMissingAdvancedContextColumn(error)
  ) {
    throw new Error(
      "Advanced Context schema unavailable. Apply supabase-setup.sql before saving advanced deployment metadata.",
    );
  }

  throw error;
}
return data;
}

export async function deleteDeployment(
  supabase: SupabaseClient,
  id: string,
  userId: string,
) {
  const { error } = await supabase
    .from("deployments")
    .update({ deleted_at: new Date().toISOString() })
    .eq("id", id)
    .eq("user_id", userId);
}
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
    if (error) throw error;
    return true;
}
```

FILE: src/lib/db\goals.ts

```
import type { SupabaseClient } from "@supabase/supabase-js";
import { validateGoal } from "../finance/goals";
```

```
export async function getGoals(supabase: SupabaseClient) {
  const { data, error } = await supabase
    .from("financial_goals")
    .select("*")
    .is("deleted_at", null)
    .order("priority", { ascending: false });

  if (error) throw error;
  return data;
}
```

```
export async function createGoal(
  supabase: SupabaseClient,
  userId: string,
  data: {
    goal_name: string;
    goal_type: string;
    target_amount: number;
    current_progress?: number;
    target_date?: string | null;
    priority: string;
    status: string;
    notes?: string;
    deleted_at?: string | null;
  },
) {
  const validated = validateGoal({
    goal_name: data.goal_name,
    goal_type: data.goal_type,
    target_amount: data.target_amount,
    current_progress: data.current_progress,
    priority: data.priority,
    status: data.status,
    target_date: data.target_date,
    notes: data.notes,
  });

  const { data: goal, error } = await supabase
    .from("financial_goals")
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
.insert({
  user_id: userId,
  goal_name: validated.goal_name,
  goal_type: validated.goal_type,
  target_amount: validated.target_amount,
  current_progress: validated.current_progress,
  target_date: validated.target_date,
  priority: validated.priority,
  status: validated.status,
  notes: validated.notes,
})
.select()
.single();

if (error) throw error;
return goal;
}

export async function updateGoal(
  supabase: SupabaseClient,
  id: string,
  userId: string,
  updates: {
    goal_name?: string;
    goal_type?: string;
    target_amount?: number;
    current_progress?: number;
    target_date?: string | null;
    priority?: string;
    status?: string;
    notes?: string | null;
    deleted_at?: string | null;
  },
) {
  const dbUpdates: Record<string, unknown> = { ...updates };

  if (
    updates.goal_name !== undefined &&
    updates.goal_name.trim().length === 0
  ) {
    throw new Error("Goal name is required.");
  }

  const { data, error } = await supabase
    .from("financial_goals")
    .update(dbUpdates)
    .eq("id", id)
    .eq("user_id", userId)
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
        .select()
        .single();

    if (error) throw error;
    return data;
}

export async function deleteGoal(
    supabase: SupabaseClient,
    id: string,
    userId: string,
) {
    const { error } = await supabase
        .from("financial_goals")
        .update({ deleted_at: new Date().toISOString() })
        .eq("id", id)
        .eq("user_id", userId);

    if (error) throw error;
    return true;
}
```

FILE: src/lib/db/income.ts

```
import type { SupabaseClient } from "@supabase/supabase-js";
import { validateIncomeStream } from "../finance/income";

export async function getIncomeStreams(supabase: SupabaseClient) {
    const { data, error } = await supabase
        .from("income_streams")
        .select("*")
        .is("deleted_at", null)
        .order("income_name", { ascending: true });

    if (error) throw error;
    return data;
}

export async function createIncomeStream(
    supabase: SupabaseClient,
    userId: string,
    data: {
        income_name: string;
        income_type: string;
        amount: number;
        cadence: string;
        execution_day?: number | null;
        is_recurring?: boolean;
    }
) {
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
    source?: string;
    currency?: string;
    start_date?: string;
    end_date?: string | null;
    deleted_at?: string | null;
  },
) {
  const validated = validateIncomeStream({
    income_name: data.income_name,
    income_type: data.income_type,
    amount: data.amount,
    cadence: data.cadence,
    execution_day: data.execution_day,
    is_recurring: data.is_recurring,
  });

  const { data: stream, error } = await supabase
    .from("income_streams")
    .insert({
      user_id: userId,
      income_name: validated.income_name,
      income_type: validated.income_type,
      amount: validated.amount,
      cadence: validated.cadence,
      execution_day: validated.execution_day,
      is_recurring: validated.is_recurring,
      source: data.source,
      currency: data.currency || "KSh",
      start_date: data.start_date || new Date().toISOString().split("T")[0],
      end_date: data.end_date,
    })
    .select()
    .single();

  if (error) throw error;
  return stream;
}

export async function createIncomeStreams(
  supabase: SupabaseClient,
  userId: string,
  inputs: {
    income_name: string;
    income_type: string;
    amount: number;
    cadence: string;
    execution_day?: number | null;
    is_recurring?: boolean;
  }
)
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
    source?: string;
    currency?: string;
    start_date?: string;
    end_date?: string | null;
    deleted_at?: string | null;
  }[],
) {
  const validatedInputs = inputs.map((data) => {
    const validated = validateIncomeStream({
      income_name: data.income_name,
      income_type: data.income_type,
      amount: data.amount,
      cadence: data.cadence,
      execution_day: data.execution_day,
      is_recurring: data.is_recurring,
    });

    return {
      user_id: userId,
      income_name: validated.income_name,
      income_type: validated.income_type,
      amount: validated.amount,
      cadence: validated.cadence,
      execution_day: validated.execution_day,
      is_recurring: validated.is_recurring,
      source: data.source,
      currency: data.currency || "KSh",
      start_date: data.start_date || new Date().toISOString().split("T")[0],
      end_date: data.end_date,
    };
  });

  const { data: streams, error } = await supabase
    .from("income_streams")
    .insert(validatedInputs)
    .select();

  if (error) throw error;
  return streams;
}

export async function updateIncomeStream(
  supabase: SupabaseClient,
  id: string,
  userId: string,
  updates: {
    income_name?: string;
    income_type?: string;
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
    amount?: number;
    cadence?: string;
    execution_day?: number | null;
    last_executed_at?: string | null;
    is_recurring?: boolean;
    source?: string;
    start_date?: string;
    end_date?: string | null;
    deleted_at?: string | null;
  },
) {
  const dbUpdates: Record<string, unknown> = { ...updates };

  if (
    updates.income_name !== undefined &&
    updates.income_name.trim().length === 0
  ) {
    throw new Error("Income name is required.");
  }

  const { data, error } = await supabase
    .from("income_streams")
    .update(dbUpdates)
    .eq("id", id)
    .eq("user_id", userId)
    .select()
    .single();

  if (error) throw error;
  return data;
}

export async function deleteIncomeStream(
  supabase: SupabaseClient,
  id: string,
  userId: string,
) {
  const { error } = await supabase
    .from("income_streams")
    .update({ deleted_at: new Date().toISOString() })
    .eq("id", id)
    .eq("user_id", userId);

  if (error) throw error;
  return true;
}
```


AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

FILE: src/lib/db\insights.ts

```
import type { SupabaseClient } from "@supabase/supabase-js";
import type { KairosInsight } from "@lib/ai/kairos";

export async function saveInsight(
  supabase: SupabaseClient,
  insight: KairosInsight,
  userId: string,
) {
  const { data, error } = await supabase.from("kairos_insights").insert({
    user_id: userId,
    type: insight.type,
    category: insight.category,
    confidence: 1.0, // Deterministic logic has absolute confidence
    message: insight.message,
    metadata: {
      related_ids: [],
      metadata_quality: insight.metadataQuality || null,
      severity: insight.severity,
      supporting_signals: insight.supportingSignals,
      runway: insight.runway,
      capital_efficiency: insight.capitalEfficiency,
      is_silent: insight.isSilent,
      timestamp: insight.timestamp,
      source: "server_orchestrator_v5e",
    },
  },
  {
  });

  if (error) {
    console.error("Error saving insight:", error);
    throw error;
  }
  return data;
}

export async function getInsights(supabase: SupabaseClient, limit: number = 5) {
  const { data, error } = await supabase
    .from("kairos_insights")
    .select("*")
    .order("created_at", { ascending: false })
    .limit(limit);

  if (error) {
    console.error("Error fetching insights:", error);
    throw error;
  }
  return data;
}
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
}
```

FILE: src/lib/db\liabilities.ts

```
import type { SupabaseClient } from "@supabase/supabase-js";
import { validateLiability } from "../finance/liabilities";

export async function getLiabilities(supabase: SupabaseClient) {
  const { data, error } = await supabase
    .from("liabilities")
    .select("*")
    .is("deleted_at", null)
    .order("liability_name", { ascending: true });

  if (error) throw error;
  return data;
}

export async function createLiability(
  supabase: SupabaseClient,
  userId: string,
  data: {
    liability_name: string;
    liability_type: string;
    outstanding_balance: number;
    interest_rate?: number;
    is_paid_in_cadences?: boolean;
    cadence?: string | null;
    cadence_day_date?: string | null;
    cadence_amount?: number | null;
    due_date?: string | null;
    institution?: string;
    currency?: string;
    deleted_at?: string | null;
  },
) {
  const validated = validateLiability({
    liability_name: data.liability_name,
    liability_type: data.liability_type,
    outstanding_balance: data.outstanding_balance,
    interest_rate: data.interest_rate,
    is_paid_in_cadences: data.is_paid_in_cadences,
    cadence: data.cadence,
    cadence_day_date: data.cadence_day_date,
    cadence_amount: data.cadence_amount,
  });

  const { data: liability, error } = await supabase
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
.from("liabilities")
.insert({
  user_id: userId,
  liability_name: validated.liability_name,
  liability_type: validated.liability_type,
  outstanding_balance: validated.outstanding_balance,
  interest_rate: validated.interest_rate,
  is_paid_in_cadences: validated.is_paid_in_cadences,
  cadence: validated.cadence,
  cadence_day_date: validated.cadence_day_date,
  cadence_amount: validated.cadence_amount,
  due_date: data.due_date,
  institution: data.institution,
  currency: data.currency || "KSh",
})
.select()
.single();

if (error) throw error;
return liability;
}

export async function updateLiability(
  supabase: SupabaseClient,
  id: string,
  userId: string,
  updates: {
    liability_name?: string;
    liability_type?: string;
    outstanding_balance?: number;
    interest_rate?: number;
    is_paid_in_cadences?: boolean;
    cadence?: string | null;
    cadence_day_date?: string | null;
    cadence_amount?: number | null;
    last_executed_at?: string | null;
    due_date?: string | null;
    institution?: string;
    deleted_at?: string | null;
  },
) {
  const dbUpdates: Record<string, unknown> = { ...updates };

  if (
    updates.liability_name !== undefined &&
    updates.liability_name.trim().length === 0
  ) {
    throw new Error("Liability name is required.");
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
}

const { data, error } = await supabase
  .from("liabilities")
  .update(dbUpdates)
  .eq("id", id)
  .eq("user_id", userId)
  .select()
  .single();

if (error) throw error;
return data;
}

export async function deleteLiability(
  supabase: SupabaseClient,
  id: string,
  userId: string,
) {
  const { error } = await supabase
    .from("liabilities")
    .update({ deleted_at: new Date().toISOString() })
    .eq("id", id)
    .eq("user_id", userId);

  if (error) throw error;
  return true;
}
```

FILE: src/lib/dblobjectives.ts

```
import type { SupabaseClient } from "@supabase/supabase-js";
import { validateObjective } from "../finance/objectives";

export async function getStrategicObjectives(supabase: SupabaseClient) {
  const { data, error } = await supabase
    .from("strategic_objectives")
    .select("*")
    .is("deleted_at", null)
    .order("priority_level", { ascending: false })
    .order("created_at", { ascending: false });

  if (error) throw error;
  return data;
}

export async function createStrategicObjective(
  supabase: SupabaseClient,
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
    userId: string,
    data: {
      objective_name: string;
      objective_type: string;
      target_amount: number;
      current_amount?: number;
      target_date?: string | null;
      priority_level: string;
      status: string;
      notes?: string;
      deleted_at?: string | null;
    },
  ) {
    const validated = validateObjective({
      objective_name: data.objective_name,
      objective_type: data.objective_type,
      target_amount: data.target_amount,
      current_amount: data.current_amount,
      priority_level: data.priority_level,
      status: data.status,
      target_date: data.target_date,
      notes: data.notes,
    });

    const { data: objective, error } = await supabase
      .from("strategic_objectives")
      .insert({
        user_id: userId,
        objective_name: validated.objective_name,
        objective_type: validated.objective_type,
        target_amount: validated.target_amount,
        current_amount: validated.current_amount,
        target_date: validated.target_date,
        priority_level: validated.priority_level,
        status: validated.status,
        notes: validated.notes,
      })
      .select()
      .single();

    if (error) throw error;
    return objective;
  }

export async function updateStrategicObjective(
  supabase: SupabaseClient,
  id: string,
  userId: string,
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
updates: {
  objective_name?: string;
  objective_type?: string;
  target_amount?: number;
  current_amount?: number;
  target_date?: string | null;
  priority_level?: string;
  status?: string;
  notes?: string | null;
  deleted_at?: string | null;
},
) {
  // If objective_name is provided, validate it's not empty
  if (
    updates.objective_name !== undefined &&
    updates.objective_name.trim().length === 0
  ) {
    throw new Error("Objective name is required.");
  }

  const { data, error } = await supabase
    .from("strategic_objectives")
    .update(updates)
    .eq("id", id)
    .eq("user_id", userId)
    .select()
    .single();

  if (error) throw error;
  return data;
}

export async function deleteStrategicObjective(
  supabase: SupabaseClient,
  id: string,
  userId: string,
) {
  const { error } = await supabase
    .from("strategic_objectives")
    .update({ deleted_at: new Date().toISOString() })
    .eq("id", id)
    .eq("user_id", userId);

  if (error) throw error;
  return true;
}
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

FILE: src/lib/db/settings.ts

```
import type { SupabaseClient } from "@supabase/supabase-js";

export async function getUserSettings(
  supabase: SupabaseClient,
  userId: string,
) {
  const { data, error } = await supabase
    .from("user_settings")
    .select("*")
    .eq("user_id", userId)
    .maybeSingle();

  if (error) throw error;

  // If no settings exist, create default
  if (!data) {
    const { data: newData, error: createError } = await supabase
      .from("user_settings")
      .insert({ user_id: userId, total_liquidity: 0 })
      .select()
      .single();

    if (createError) throw createError;
    return newData;
  }

  return data;
}

export async function updateLiquidity(
  supabase: SupabaseClient,
  userId: string,
  amount: number,
) {
  const { data, error } = await supabase
    .from("user_settings")
    .upsert({
      user_id: userId,
      total_liquidity: amount,
      updated_at: new Date().toISOString(),
    })
    .select()
    .single();

  if (error) throw error;
  return data;
}
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
}
```

FILE: src/lib/db\telemetry.ts

```
import { createClient } from "@lib/supabase/server";

/**
 * TELEMETRY SERVICE (Phase 1)
 * Records insight evaluation events for observability.
 */
export async function logKairosEvaluation(data: {
  ruleId: string;
  severity: string;
  durationMs: number;
}) {
  try {
    let supabase;
    try {
      supabase = await createClient();
    } catch (_e) {
      // If we're in a test environment or outside a request scope, cookies() will fail.
      // We silently exit as telemetry is non-critical for these contexts.
      return;
    }
  }

  const {
    data: { user },
  } = await supabase.auth.getUser();
  if (!user) return;

  // Record evaluation event to kairos_insights table.
  // This phase uses the rule registry ID and performance metadata.
  const { error } = await supabase.from("kairos_insights").insert({
    user_id: user.id,
    rule_id: data.ruleId,
    severity: data.severity,
    evaluation_time_ms: data.durationMs,
  });

  if (error) {
    console.error("[Telemetry] Error recording evaluation:", error.message);
  }
} catch (err) {
  // Observability must be non-blocking; we swallow errors to protect primary flows.
  console.error("[Telemetry] Critical failure in logging:", err);
}
```


AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

FILE: src/lib/finance/accounts.ts

```
export const ACCOUNT_TYPES = [
  { value: "checking", label: "Checking" },
  { value: "savings", label: "Savings" },
  { value: "mobile_money", label: "Mobile Money" },
  { value: "brokerage", label: "Brokerage" },
  { value: "crypto", label: "Crypto" },
  { value: "cash", label: "Cash" },
] as const;

export type AccountType = (typeof ACCOUNT_TYPES)[number]["value"];

export interface Account {
  id: string;
  user_id: string;
  account_name: string;
  account_type: AccountType;
  current_balance: number;
  currency: string;
  institution?: string | null;
  created_at: string;
  updated_at: string;
  deleted_at?: string | null;
}

export function isValidAccountType(type: string): type is AccountType {
  return ACCOUNT_TYPES.some((t) => t.value === type);
}

export function validateAccount(data: {
  account_name: string;
  account_type: string;
  current_balance: number;
}) {
  if (!data.account_name || data.account_name.trim().length === 0) {
    throw new Error("Account name is required.");
  }

  if (!isValidAccountType(data.account_type)) {
    throw new Error(`Invalid account type: ${data.account_type}`);
  }

  if (isNaN(data.current_balance)) {
    throw new Error("Initial balance must be a number.");
  }

  return {
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
    account_name: data.account_name.trim(),
    account_type: data.account_type as AccountType,
    current_balance: data.current_balance,
  };
}
```

FILE: src/lib/finance\deploymentContext.ts

```
export const EXPECTED_RETURN_HORIZONS = [
  { value: "short-term", label: "Short-term" },
  { value: "medium-term", label: "Medium-term" },
  { value: "long-term", label: "Long-term" },
] as const;

export type ExpectedReturnHorizon =
  (typeof EXPECTED_RETURN_HORIZONS)[number]["value"];

export interface DeploymentAdvancedContext {
  associatedAccount?: string;
  expectedReturnHorizon?: ExpectedReturnHorizon;
  tags?: string[];
}

export interface DeploymentAdvancedContextInput {
  associatedAccount?: string | null;
  expectedReturnHorizon?: string | null;
  tags?: string | string[] | null;
}

const EXPECTED_RETURN_HORIZON_VALUES = new Set(
  EXPECTED_RETURN_HORIZONS.map((horizon) => horizon.value),
);

export function isExpectedReturnHorizon(
  value: string,
): value is ExpectedReturnHorizon {
  return EXPECTED_RETURN_HORIZON_VALUES.has(value as ExpectedReturnHorizon);
}

function normalizeTags(tags: DeploymentAdvancedContextInput["tags"]) {
  const source = Array.isArray(tags) ? tags.join(",") : tags || "";
  const normalizedTags = source
    .split(",")
    .map((tag) => tag.trim().toLowerCase())
    .filter(Boolean);

  return Array.from(new Set(normalizedTags));
}
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
export function normalizeDeploymentAdvancedContext(
  input: DeploymentAdvancedContextInput | null = {},
): DeploymentAdvancedContext {
  const safeInput = input || {};
  const associatedAccount = (safeInput.associatedAccount || "").trim();
  const expectedReturnHorizon = (
    safeInput.expectedReturnHorizon || ""
  ).trim();
  const tags = normalizeTags(safeInput.tags);

  if (
    expectedReturnHorizon &&
    !isExpectedReturnHorizon(expectedReturnHorizon)
  ) {
    throw new Error(
      "Context Gate: Unknown expected return horizon. Use short-term, medium-term, or long-term.",
    );
  }

  return {
    ...(associatedAccount ? { associatedAccount } : {}),
    ...(expectedReturnHorizon
      ? { expectedReturnHorizon: expectedReturnHorizon as ExpectedReturnHorizon }
      : {}),
    ...(tags.length > 0 ? { tags } : {}),
  };
}

export function hasDeploymentAdvancedContext(
  context: DeploymentAdvancedContext,
) {
  return Boolean(
    context.associatedAccount ||
    context.expectedReturnHorizon ||
    (context.tags && context.tags.length > 0),
  );
}
```

FILE: src/lib/finance/financeRules.test.ts

```
import assert from "node:assert/strict";
import test from "node:test";

import {
  evaluateMetadataQuality,
  summarizeMetadataQuality,
  weightConfidenceForMetadataQuality,
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
} from "./metadataQuality";
import {
  normalizeDeploymentAdvancedContext,
} from "./deploymentContext";
import { isValidCategory } from "./taxonomy";
import { validateDeployment } from "./validateDeployment";

test("taxonomy validation accepts only canonical return categories", () => {
  assert.equal(isValidCategory("Asset"), true);
  assert.equal(isValidCategory("Leakage"), true);
  assert.equal(isValidCategory("Unclassified"), false);
  assert.equal(isValidCategory("Unknown"), false);
});

test("metadata quality scoring is deterministic for generic labels", () => {
  assert.deepEqual(evaluateMetadataQuality(" misc ").reasons, [
    "generic_label",
  ]);
  assert.equal(evaluateMetadataQuality("stuff").isLowQuality, true);
  assert.equal(evaluateMetadataQuality("Revenue engine setup").isLowQuality, false);
});

test("metadata quality summary produces stable confidence weighting", () => {
  const summary = summarizeMetadataQuality([
    { title: "misc" },
    { title: "Brokerage allocation" },
  ]);

  assert.equal(summary.lowQualityCount, 1);
  assert.equal(summary.lowQualityRatio, 0.5);
  assert.equal(summary.confidenceMultiplier, 0.93);
  assert.equal(weightConfidenceForMetadataQuality(0.9, summary), 0.84);
});

test("advanced context normalization preserves optional structured metadata", () => {
  assert.deepEqual(
    normalizeDeploymentAdvancedContext({
      associatedAccount: " Brokerage ",
      expectedReturnHorizon: "long-term",
      tags: "Ops, recurring, ops",
    }),
    {
      associatedAccount: "Brokerage",
      expectedReturnHorizon: "long-term",
      tags: ["ops", "recurring"],
    },
  );
});
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
test("advanced context rejects unknown return horizons", () => {
  assert.throws(
    () =>
      normalizeDeploymentAdvancedContext({
        expectedReturnHorizon: "eventually",
      }),
    /Unknown expected return horizon/,
  );
});
```

```
test("deployment validation remains the write gate", () => {
  assert.throws(
    () =>
      validateDeployment({
        title: "Capital",
        amount: 100,
        category: "Unclassified",
      }),
    /Strategic classification is mandatory/,
  );
});
```

```
assert.deepEqual(
  validateDeployment({
    title: " Cloud infra ",
    amount: 500,
    category: "Leverage",
    advancedContext: {
      expectedReturnHorizon: "medium-term",
      tags: "ops, infra",
    },
  }).advancedContext,
  {
    expectedReturnHorizon: "medium-term",
    tags: ["ops", "infra"],
  },
);
});
```

FILE: src/lib/finance\goals.test.ts

```
import { test } from "node:test";
import assert from "node:assert";
import { validateGoal, calculateGoalProgressPercentage } from "../goals";
import { generateSummary } from "../../analytics/engine";
import type { FinancialGoal } from "../../analytics/types";
```

```
test("goal validation enforces structural integrity", () => {
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
const data = {
  goal_name: "Emergency Fund",
  goal_type: "emergency_fund",
  target_amount: 1000000,
  current_progress: 100000,
  priority: "critical",
  status: "active",
};

const validated = validateGoal(data);
assert.strictEqual(validated.goal_name, "Emergency Fund");
assert.strictEqual(validated.goal_type, "emergency_fund");
assert.strictEqual(validated.target_amount, 1000000);
assert.strictEqual(validated.current_progress, 100000);
});

test("goal progress calculation is deterministic", () => {
  assert.strictEqual(calculateGoalProgressPercentage({ target_amount: 1000, current_progress: 500 }), 50);
  assert.strictEqual(calculateGoalProgressPercentage({ target_amount: 1000, current_progress: 1000 }), 100);
  assert.strictEqual(calculateGoalProgressPercentage({ target_amount: 1000, current_progress: 1500 }), 100);
  assert.strictEqual(calculateGoalProgressPercentage({ target_amount: 0, current_progress: 500 }), 0);
});

test("strategic metrics are accurately aggregated", () => {
  const goals: FinancialGoal[] = [
    {
      id: "g1",
      user_id: "u1",
      goal_name: "G1",
      goal_type: "emergency_fund",
      target_amount: 1000,
      current_progress: 200,
      priority: "critical",
      status: "active",
      linked_account_ids: [],
      created_at: "",
      updated_at: "",
      target_date: null,
    },
    {
      id: "g2",
      user_id: "u1",
      goal_name: "G2",
      goal_type: "retirement",
      target_amount: 1000,
      current_progress: 800,
      priority: "medium",
      status: "active",
    }
  ]
}
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
        linked_account_ids: [],
        created_at: "",
        updated_at: "",
        target_date: null,
    },
];

const summary = generateSummary([], 0, [], [], [], goals);

assert.strictEqual(summary.totalStrategicTargets, 2000);
assert.strictEqual(summary.totalCurrentProgress, 1000);
assert.strictEqual(summary.averageGoalProgress, 50); // (20% + 80%) / 2
assert.strictEqual(summary.criticalGoalCount, 1);
assert.strictEqual(summary.fundingGap, 1000);
});
```

FILE: src/lib/finance\goals.ts

```
export const GOAL_TYPES = [
    { value: "emergency_fund", label: "Emergency Fund" },
    { value: "retirement", label: "Retirement" },
    { value: "home_purchase", label: "Home Purchase" },
    { value: "investment_target", label: "Investment Target" },
    { value: "business_runway", label: "Business Runway" },
    { value: "education", label: "Education" },
    { value: "debt_payoff", label: "Debt Payoff" },
    { value: "wealth_preservation", label: "Wealth Preservation" },
    { value: "other", label: "Other" },
] as const;

export type GoalType = (typeof GOAL_TYPES)[number]["value"];

export const GOAL_PRIORITIES = [
    { value: "critical", label: "Critical" },
    { value: "high", label: "High" },
    { value: "medium", label: "Medium" },
    { value: "low", label: "Low" },
] as const;

export type GoalPriority = (typeof GOAL_PRIORITIES)[number]["value"];

export const GOAL_STATUSES = [
    { value: "active", label: "Active" },
    { value: "paused", label: "Paused" },
    { value: "achieved", label: "Achieved" },
    { value: "archived", label: "Archived" },
] as const;
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
export type GoalStatus = (typeof GOAL_STATUSES)[number]["value"];

export interface FinancialGoal {
  id: string;
  user_id: string;
  goal_name: string;
  goal_type: GoalType;
  target_amount: number;
  current_progress: number;
  target_date: string | null;
  priority: GoalPriority;
  status: GoalStatus;
  linked_account_ids: string[];
  notes?: string | null;
  created_at: string;
  updated_at: string;
  deleted_at?: string | null;
}

export function isValidGoalType(type: string): type is GoalType {
  return GOAL_TYPES.some((t) => t.value === type);
}

export function isValidPriority(priority: string): priority is GoalPriority {
  return GOAL_PRIORITIES.some((p) => p.value === priority);
}

export function isValidStatus(status: string): status is GoalStatus {
  return GOAL_STATUSES.some((s) => s.value === status);
}

export function validateGoal(data: {
  goal_name: string;
  goal_type: string;
  target_amount: number;
  current_progress?: number;
  priority: string;
  status: string;
  target_date?: string | null;
  notes?: string | null;
}) {
  if (!data.goal_name || data.goal_name.trim().length === 0) {
    throw new Error("Goal name is required.");
  }

  if (!isValidGoalType(data.goal_type)) {
    throw new Error(`Invalid goal type: ${data.goal_type}`);
  }
}
```


AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
const targetAmount = Number(data.target_amount);
const currentProgress = Number(data.current_progress || 0);

if (isNaN(targetAmount) || targetAmount <= 0) {
  throw new Error("Target amount must be a positive number.");
}

if (isNaN(currentProgress) || currentProgress < 0) {
  throw new Error("Current progress must be a non-negative number.");
}

if (!isValidPriority(data.priority)) {
  throw new Error(`Invalid priority: ${data.priority}`);
}

if (!isValidStatus(data.status)) {
  throw new Error(`Invalid status: ${data.status}`);
}

// Date Validation
let normalizedDate: string | null = null;
if (data.target_date) {
  const d = new Date(data.target_date);
  if (isNaN(d.getTime())) {
    throw new Error("Invalid target date format.");
  }
  normalizedDate = d.toISOString().split("T")[0];
}

const finalNotes = data.notes?.trim() || null;

return {
  goal_name: data.goal_name.trim(),
  goal_type: data.goal_type as GoalType,
  target_amount: targetAmount,
  current_progress: currentProgress,
  priority: data.priority as GoalPriority,
  status: data.status as GoalStatus,
  target_date: normalizedDate,
  notes: finalNotes === "" ? null : finalNotes,
};
}

/**
 * Calculates deterministic goal progress.
 */
export function calculateGoalProgressPercentage(
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
goal: Pick<FinancialGoal, "current_progress" | "target_amount">,
): number {
  if (goal.target_amount <= 0) return 0;
  return Math.min(
    100,
    Math.max(0, (goal.current_progress / goal.target_amount) * 100),
  );
}
```

FILE: src/lib/finance/income.test.ts

```
import { test } from "node:test";
import assert from "node:assert";
import { validateIncomeStream, calculateMonthlyInflow } from "../income";
import { generateSummary } from "../analytics/engine";
import type { Deployment, IncomeStream } from "../analytics/types";
```

```
test("income validation enforces structural integrity", () => {
  const data = {
    income_name: "Salary",
    income_type: "salary",
    amount: 100000,
    cadence: "monthly",
  };
});
```

```
const validated = validateIncomeStream(data);
assert.strictEqual(validated.income_name, "Salary");
assert.strictEqual(validated.income_type, "salary");
assert.strictEqual(validated.amount, 100000);
assert.strictEqual(validated.cadence, "monthly");
});
```

```
test("income normalization is accurate", () => {
  assert.strictEqual(
    calculateMonthlyInflow({ amount: 1000, cadence: "weekly" }),
    1000 * (52 / 12),
  );
  assert.strictEqual(
    calculateMonthlyInflow({ amount: 120000, cadence: "annually" }),
    10000,
  );
  assert.strictEqual(
    calculateMonthlyInflow({ amount: 5000, cadence: "monthly" }),
    5000,
  );
});
```

```
test("runway logic incorporates income offset correctly", () => {
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
const deployments: Deployment[] = [
  {
    id: "1",
    amount: 3000,
    created_at: new Date().toISOString(),
    title: "Rent",
    category: "Fixed",
  },
]; // 100/day burn over 30 days

const liquidity = 1000;
const incomeStreams: IncomeStream[] = [
  {
    id: "s1",
    user_id: "u1",
    income_name: "Salary",
    amount: 1500,
    cadence: "monthly",
    is_recurring: true,
    income_type: "salary",
    currency: "KSh",
    start_date: "",
    created_at: "",
    updated_at: "",
  },
]; // 50/day offset

// Correct Formula: Liquidity / (Daily Burn - (Monthly Income / 30))
// 1000 / (100 - (1500 / 30)) = 1000 / 50 = 20 days
const summary = generateSummary(
  deployments,
  liquidity,
  [],
  [],
  incomeStreams,
);

assert.strictEqual(summary.dailyBurnRate, 100);
assert.strictEqual(summary.totalMonthlyIncome, 1500);
assert.strictEqual(summary.runwayDays, 20);
assert.strictEqual(summary.adjustedDailyBurn, 50);
});
```

FILE: src/lib/finance/income.ts

```
export const INCOME_TYPES = [
  { value: "salary", label: "Salary" },
  { value: "freelance", label: "Freelance" },
];
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
{ value: "business", label: "Business" },
{ value: "dividends", label: "Dividends" },
{ value: "rental", label: "Rental" },
{ value: "contract", label: "Contract" },
{ value: "other", label: "Other" },
] as const;

export type IncomeType = (typeof INCOME_TYPES)[number]["value"];

export const CADENCES = [
  { value: "weekly", label: "Weekly", factor: 52 / 12 },
  { value: "biweekly", label: "Biweekly", factor: 26 / 12 },
  { value: "monthly", label: "Monthly", factor: 1 },
  { value: "quarterly", label: "Quarterly", factor: 1 / 3 },
  { value: "annually", label: "Annually", factor: 1 / 12 },
  { value: "irregular", label: "Irregular", factor: 0 },
] as const;

export type Cadence = (typeof CADENCES)[number]["value"];

export interface IncomeStream {
  id: string;
  user_id: string;
  income_name: string;
  income_type: IncomeType;
  amount: number;
  cadence: Cadence;
  execution_day?: number | null;
  last_executed_at?: string | null;
  is_recurring: boolean;
  currency: string;
  source?: string | null;
  start_date: string;
  end_date?: string | null;
  created_at: string;
  updated_at: string;
  deleted_at?: string | null;
}

export function isValidIncomeType(type: string): type is IncomeType {
  return INCOME_TYPES.some((t) => t.value === type);
}

export function isValidCadence(cadence: string): cadence is Cadence {
  return CADENCES.some((c) => c.value === cadence);
}

export function validateIncomeStream(data: {
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
income_name: string;
income_type: string;
amount: number;
cadence: string;
execution_day?: number | null;
is_recurring?: boolean;
}) {
  if (!data.income_name || data.income_name.trim().length === 0) {
    throw new Error("Income name is required.");
  }

  if (!isValidIncomeType(data.income_type)) {
    throw new Error(`Invalid income type: ${data.income_type}`);
  }

  if (!isValidCadence(data.cadence)) {
    throw new Error(`Invalid cadence: ${data.cadence}`);
  }

  if (isNaN(data.amount) || data.amount < 0) {
    throw new Error("Amount must be a non-negative number.");
  }

  return {
    income_name: data.income_name.trim(),
    income_type: data.income_type as IncomeType,
    amount: data.amount,
    cadence: data.cadence as Cadence,
    execution_day: data.execution_day,
    is_recurring: data.is_recurring ?? true,
  };
}

/**
 * Normalizes an income stream amount to a monthly value.
 */
export function calculateMonthlyInflow(
  stream: Pick<IncomeStream, "amount" | "cadence">,
): number {
  const cadenceConfig = CADENCES.find((c) => c.value === stream.cadence);
  if (!cadenceConfig) return 0;
  return stream.amount * cadenceConfig.factor;
}
```

FILE: src/lib/finance/liabilities.test.ts

```
import { test } from "node:test";
import assert from "node:assert";
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
import { validateLiability } from "../liabilities";
import { generateSummary } from "../analytics/engine";
import type { Account, Liability } from "../analytics/types";

test("liability validation enforces structural integrity", () => {
  const data = {
    liability_name: "Credit Card",
    liability_type: "credit_card",
    outstanding_balance: 50000,
    interest_rate: 2,
    is_paid_in_cadences: true,
    cadence: "monthly",
    cadence_day_date: "15",
    cadence_amount: 5000,
  };

  const validated = validateLiability(data);
  assert.strictEqual(validated.liability_name, "Credit Card");
  assert.strictEqual(validated.liability_type, "credit_card");
  assert.strictEqual(validated.outstanding_balance, 50000);
  assert.strictEqual(validated.is_paid_in_cadences, true);
  assert.strictEqual(validated.cadence, "monthly");
  assert.strictEqual(validated.cadence_day_date, "15");
  assert.strictEqual(validated.cadence_amount, 5000);
});

test("liability validation rejects invalid monthly date", () => {
  assert.throws(() => {
    validateLiability({
      liability_name: "Test",
      liability_type: "credit_card",
      outstanding_balance: 1000,
      is_paid_in_cadences: true,
      cadence: "monthly",
      cadence_day_date: "32",
      cadence_amount: 100,
    });
  }, /Monthly cadence requires a valid date/);
});

test("liability validation rejects invalid weekly day", () => {
  assert.throws(() => {
    validateLiability({
      liability_name: "Test",
      liability_type: "credit_card",
      outstanding_balance: 1000,
      is_paid_in_cadences: true,
      cadence: "weekly",
    });
  });
});
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
        cadence_day_date: "Funday",
        cadence_amount: 100,
    });
}, /Weekly cadence requires a valid day/);
});

test("net worth calculation and interest accrual are deterministic", () => {
    const accounts: Account[] = [
        {
            id: "1",
            user_id: "u1",
            account_name: "Checking",
            account_type: "checking",
            current_balance: 100000,
            currency: "KSh",
            created_at: "",
            updated_at: "",
            institution: null,
        },
    ];

    const liabilities: Liability[] = [
        {
            id: "l1",
            user_id: "u1",
            liability_name: "Loan",
            liability_type: "personal_loan",
            outstanding_balance: 30000,
            interest_rate: 1, // 1% per month
            is_paid_in_cadences: true,
            cadence: "monthly",
            cadence_day_date: "1",
            cadence_amount: 3000,
            currency: "KSh",
            created_at: "",
            updated_at: "",
            institution: null,
            due_date: null,
        },
    ];

    const summary = generateSummary([], 100000, accounts, liabilities);

    assert.strictEqual(summary.totalAssets, 100000);
    assert.strictEqual(summary.totalLiabilities, 30000);
    assert.strictEqual(summary.netWorth, 70000);

    // Monthly interest = 30000 * 0.01 = 300
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
// Daily interest burn = 300 / 30 = 10
assert.strictEqual(summary.dailyBurnRate, 10);
});
```

FILE: src/lib/finance\liabilities.ts

```
export const LIABILITY_TYPES = [
  { value: "credit_card", label: "Credit Card" },
  { value: "mortgage", label: "Mortgage" },
  { value: "personal_loan", label: "Personal Loan" },
  { value: "student_loan", label: "Student Loan" },
  { value: "business_loan", label: "Business Loan" },
  { value: "line_of_credit", label: "Line of Credit" },
  { value: "other", label: "Other" },
] as const;

export type LiabilityType = (typeof LIABILITY_TYPES)[number]["value"];

export interface Liability {
  id: string;
  user_id: string;
  liability_name: string;
  liability_type: LiabilityType;
  outstanding_balance: number;
  interest_rate: number;
  is_paid_in_cadences: boolean;
  cadence: "weekly" | "monthly" | null;
  cadence_day_date: string | null;
  cadence_amount: number | null;
  last_executed_at?: string | null;
  due_date: string | null;
  currency: string;
  institution?: string | null;
  created_at: string;
  updated_at: string;
  deleted_at?: string | null;
}

export function isValidLiabilityType(type: string): type is LiabilityType {
  return LIABILITY_TYPES.some((t) => t.value === type);
}

export function validateLiability(data: {
  liability_name: string;
  liability_type: string;
  outstanding_balance: number;
  interest_rate?: number;
  is_paid_in_cadences?: boolean;
```


AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
cadence?: string | null;
cadence_day_date?: string | null;
cadence_amount?: number | null;
}) {
  if (!data.liability_name || data.liability_name.trim().length === 0) {
    throw new Error("Liability name is required.");
  }

  if (!isValidLiabilityType(data.liability_type)) {
    throw new Error(`Invalid liability type: ${data.liability_type}`);
  }

  if (isNaN(data.outstanding_balance)) {
    throw new Error("Outstanding balance must be a number.");
  }

  const isPaidInCadences = !!data.is_paid_in_cadences;
  let cadence: "weekly" | "monthly" | null = null;
  let cadenceDayDate: string | null = null;
  let cadenceAmount: number | null = null;

  if (isPaidInCadences) {
    if (data.cadence !== "weekly" && data.cadence !== "monthly") {
      throw new Error("A valid cadence (weekly or monthly) is required.");
    }
    cadence = data.cadence;

    if (!data.cadence_day_date || data.cadence_day_date.trim().length === 0) {
      throw new Error("Cadence day/date is required.");
    }

    if (cadence === "monthly") {
      const dateNum = parseInt(data.cadence_day_date, 10);
      if (isNaN(dateNum) || dateNum < 1 || dateNum > 31) {
        throw new Error("Monthly cadence requires a valid date (1-31).");
      }
    } else if (cadence === "weekly") {
      const validDays = [
        "monday",
        "tuesday",
        "wednesday",
        "thursday",
        "friday",
        "saturday",
        "sunday",
      ];
      if (!validDays.includes(data.cadence_day_date.toLowerCase())) {
        throw new Error("Weekly cadence requires a valid day of the week.");
      }
    }
  }
}
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
    }
  }
  cadenceDayDate = data.cadence_day_date;

  if (
    data.cadence_amount === undefined ||
    data.cadence_amount === null ||
    isNaN(data.cadence_amount)
  ) {
    throw new Error("Cadence payment amount is required.");
  }
  cadenceAmount = data.cadence_amount;
}

return {
  liability_name: data.liability_name.trim(),
  liability_type: data.liability_type as LiabilityType,
  outstanding_balance: data.outstanding_balance,
  interest_rate: data.interest_rate || 0,
  is_paid_in_cadences: isPaidInCadences,
  cadence,
  cadence_day_date: cadenceDayDate,
  cadence_amount: cadenceAmount,
};
}
```

FILE: src/lib/finance\metadataQuality.ts

```
export type MetadataQualityLevel = "specific" | "low";

export interface MetadataQualityEvaluation {
  level: MetadataQualityLevel;
  score: number;
  isLowQuality: boolean;
  normalizedLabel: string;
  reasons: string[];
}

export interface MetadataQualitySummary {
  averageScore: number;
  lowQualityCount: number;
  lowQualityRatio: number;
  confidenceMultiplier: number;
}

type LabelBearingInput = {
  title?: string | null;
};
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
const LOW_QUALITY_EXACT_LABELS = new Set([
  "misc",
  "stuff",
  "other",
  "random",
  "generic",
  "unclear",
  "unknown",
  "untitled",
  "no title",
  "none",
  "nil",
  "na",
  "n/a",
  "tbd",
  "test",
]);

const EMPTY_LIKE_PATTERNS = [/^-$/, /^\.+$/, /^?+$/, /^_+$/];

const roundQualityScore = (value: number) => Math.round(value * 100) / 100;

export function normalizeMetadataLabel(label: string) {
  return label.trim().toLowerCase().replace(/\s+/g, " ");
}

export function evaluateMetadataQuality(
  label: string | null | undefined,
): MetadataQualityEvaluation {
  const normalizedLabel = normalizeMetadataLabel(label || "");
  const semanticLabel = normalizedLabel
    .replace(/^[a-z0-9/?._-]+/g, " ")
    .replace(/\s+/g, " ")
    .trim();
  const comparableLabel = semanticLabel.replace(
    /^[^a-z0-9]+|^[a-z0-9]+$/g,
    "",
  );

  const reasons: string[] = [];

  if (
    !semanticLabel ||
    EMPTY_LIKE_PATTERNS.some((pattern) => pattern.test(semanticLabel))
  ) {
    reasons.push("empty_like_label");
  }
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
if (
  LOW_QUALITY_EXACT_LABELS.has(semanticLabel) ||
  LOW_QUALITY_EXACT_LABELS.has(comparableLabel)
) {
  reasons.push("generic_label");
}

const isLowQuality = reasons.length > 0;

return {
  level: isLowQuality ? "low" : "specific",
  score: isLowQuality ? 0.35 : 1,
  isLowQuality,
  normalizedLabel,
  reasons,
};
}

export function summarizeMetadataQuality(
  deployments: LabelBearingInput[],
): MetadataQualitySummary {
  if (deployments.length === 0) {
    return {
      averageScore: 1,
      lowQualityCount: 0,
      lowQualityRatio: 0,
      confidenceMultiplier: 1,
    };
  }

  const evaluations = deployments.map((deployment) =>
    evaluateMetadataQuality(deployment.title),
  );
  const lowQualityCount = evaluations.filter(
    (evaluation) => evaluation.isLowQuality,
  ).length;
  const averageScore =
    evaluations.reduce((sum, evaluation) => sum + evaluation.score, 0) /
    evaluations.length;
  const lowQualityRatio = lowQualityCount / evaluations.length;

  return {
    averageScore: roundQualityScore(averageScore),
    lowQualityCount,
    lowQualityRatio: roundQualityScore(lowQualityRatio),
    confidenceMultiplier: roundQualityScore(
      Math.max(0.85, 1 - lowQualityRatio * 0.15),
    ),
  };
}
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
    ),
  };
}

export function weightConfidenceForMetadataQuality(
  confidence: number,
  metadataQuality: MetadataQualitySummary,
) {
  return roundQualityScore(
    Math.min(1, Math.max(0, confidence * metadataQuality.confidenceMultiplier)),
  );
}
```

FILE: src/lib/finance/objectives.test.ts

```
import { test } from "node:test";
import assert from "node:assert";
import { validateObjective } from "../objectives";

test("strategic objective validation enforces structural integrity", () => {
  const data = {
    objective_name: "Safety Net",
    objective_type: "emergency_reserve",
    target_amount: 500000,
    current_amount: 50000,
    priority_level: "critical",
    status: "active",
  };

  const validated = validateObjective(data);
  assert.strictEqual(validated.objective_name, "Safety Net");
  assert.strictEqual(validated.objective_type, "emergency_reserve");
  assert.strictEqual(validated.target_amount, 500000);
  assert.strictEqual(validated.current_amount, 50000);
  assert.strictEqual(validated.priority_level, "critical");
  assert.strictEqual(validated.status, "active");
});

test("objective validation fails on invalid types", () => {
  assert.throws(() => {
    validateObjective({
      objective_name: "Test",
      objective_type: "invalid_type",
      target_amount: 1000,
      priority_level: "high",
      status: "active",
    });
  }, /Invalid objective type/);
});
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
});
```

```
test("objective validation fails on invalid priority", () => {
  assert.throws(() => {
    validateObjective({
      objective_name: "Test",
      objective_type: "liquidity",
      target_amount: 1000,
      priority_level: "very_high",
      status: "active",
    });
  }, /Invalid priority level/);
});
```

```
test("objective validation fails on invalid status", () => {
  assert.throws(() => {
    validateObjective({
      objective_name: "Test",
      objective_type: "liquidity",
      target_amount: 1000,
      priority_level: "high",
      status: "unknown",
    });
  }, /Invalid status/);
});
```

```
test("objective validation fails on zero or negative target amount", () => {
  assert.throws(() => {
    validateObjective({
      objective_name: "Test",
      objective_type: "liquidity",
      target_amount: 0,
      priority_level: "high",
      status: "active",
    });
  }, /Target amount must be a positive number/);

  assert.throws(() => {
    validateObjective({
      objective_name: "Test",
      objective_type: "liquidity",
      target_amount: -1,
      priority_level: "high",
      status: "active",
    });
  }, /Target amount must be a positive number/);
});
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
test("objective validation fails on overfunded state", () => {
  assert.throws(() => {
    validateObjective({
      objective_name: "Test",
      objective_type: "liquidity",
      target_amount: 1000,
      current_amount: 1001,
      priority_level: "high",
      status: "active",
    });
  }, /Current amount cannot exceed target amount/);
});

test("objective validation handles date normalization", () => {
  const validated = validateObjective({
    objective_name: "Test",
    objective_type: "liquidity",
    target_amount: 1000,
    priority_level: "high",
    status: "active",
    target_date: "2026-12-31",
  });
  assert.strictEqual(validated.target_date, "2026-12-31");
});

test("objective validation fails on malformed dates", () => {
  assert.throws(() => {
    validateObjective({
      objective_name: "Test",
      objective_type: "liquidity",
      target_amount: 1000,
      priority_level: "high",
      status: "active",
      target_date: "not-a-date",
    });
  }, /Invalid target date format/);
});

test("objective normalization trims whitespace and handles empty notes", () => {
  const validated = validateObjective({
    objective_name: "  Safety Net  ",
    objective_type: "emergency_reserve",
    target_amount: 1000,
    priority_level: "critical",
    status: "active",
    notes: "  Need this for safety  ",
  });
  assert.strictEqual(validated.objective_name, "Safety Net");
});
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
assert.strictEqual(validated.notes, "Need this for safety");

const validatedEmptyNotes = validateObjective({
  objective_name: "Test",
  objective_type: "liquidity",
  target_amount: 1000,
  priority_level: "high",
  status: "active",
  notes: "    ",
});
assert.strictEqual(validatedEmptyNotes.notes, null);
});
```

FILE: src/lib/finance/objectives.ts

```
export const OBJECTIVE_TYPES = [
  { value: "emergency_reserve", label: "Emergency Reserve" },
  { value: "liquidity", label: "Liquidity" },
  { value: "debt_elimination", label: "Debt Elimination" },
  { value: "investment", label: "Investment" },
  { value: "retirement", label: "Retirement" },
  { value: "education", label: "Education" },
  { value: "acquisition", label: "Acquisition" },
  { value: "custom", label: "Custom" },
] as const;

export type ObjectiveType = (typeof OBJECTIVE_TYPES)[number]["value"];

export const OBJECTIVE_PRIORITIES = [
  { value: "critical", label: "Critical" },
  { value: "high", label: "High" },
  { value: "moderate", label: "Moderate" },
  { value: "low", label: "Low" },
] as const;

export type ObjectivePriority = (typeof OBJECTIVE_PRIORITIES)[number]["value"];

export const OBJECTIVE_STATUSES = [
  { value: "active", label: "Active" },
  { value: "paused", label: "Paused" },
  { value: "completed", label: "Completed" },
  { value: "abandoned", label: "Abandoned" },
] as const;

export type ObjectiveStatus = (typeof OBJECTIVE_STATUSES)[number]["value"];

export interface StrategicObjective {
  id: string;
```


AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
    user_id: string;
    objective_name: string;
    objective_type: ObjectiveType;
    target_amount: number;
    current_amount: number;
    target_date: string | null;
    priority_level: ObjectivePriority;
    status: ObjectiveStatus;
    notes?: string | null;
    created_at: string;
    updated_at: string;
    deleted_at?: string | null;
}

export function isValidObjectiveType(type: string): type is ObjectiveType {
    return OBJECTIVE_TYPES.some((t) => t.value === type);
}

export function isValidPriority(
    priority: string,
): priority is ObjectivePriority {
    return OBJECTIVE_PRIORITIES.some((p) => p.value === priority);
}

export function isValidStatus(status: string): status is ObjectiveStatus {
    return OBJECTIVE_STATUSES.some((s) => s.value === status);
}

/**
 * Validates and normalizes strategic objective data.
 * This is the canonical authority for objective correctness.
 */
/**
 * Calculates deterministic objective progress percentage.
 */
export function calculateObjectiveProgressPercentage(
    objective: Pick<StrategicObjective, "current_amount" | "target_amount">,
): number {
    if (objective.target_amount <= 0) return 0;
    return Math.min(
        100,
        Math.max(0, (objective.current_amount / objective.target_amount) * 100),
    );
}

/**
 * Calculates the funding ratio (alias for progress for semantic clarity in UI).
 */
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
export function calculateObjectiveFundingRatio(
  objective: Pick<StrategicObjective, "current_amount" | "target_amount">,
): number {
  return calculateObjectiveProgressPercentage(objective);
}

export function validateObjective(data: {
  objective_name: string;
  objective_type: string;
  target_amount: number;
  current_amount?: number;
  priority_level: string;
  status: string;
  target_date?: string | null;
  notes?: string | null;
}) {
  // 1. Required Fields & Basic Types
  if (!data.objective_name || data.objective_name.trim().length === 0) {
    throw new Error("Objective name is required.");
  }

  // 2. Enum Validations
  if (!isValidObjectiveType(data.objective_type)) {
    throw new Error(`Invalid objective type: ${data.objective_type}`);
  }

  if (!isValidPriority(data.priority_level)) {
    throw new Error(`Invalid priority level: ${data.priority_level}`);
  }

  if (!isValidStatus(data.status)) {
    throw new Error(`Invalid status: ${data.status}`);
  }

  // 3. Numeric Validations
  const targetAmount = Number(data.target_amount);
  const currentAmount = Number(data.current_amount || 0);

  if (isNaN(targetAmount) || targetAmount <= 0) {
    throw new Error("Target amount must be a positive number.");
  }

  if (isNaN(currentAmount) || currentAmount < 0) {
    throw new Error("Current amount must be a non-negative number.");
  }

  if (currentAmount > targetAmount) {
    throw new Error(
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
        "Current amount cannot exceed target amount (overfunding requires manual strategic adjustment).",
    );
}

// 4. Date Validation
let normalizedDate: string | null = null;
if (data.target_date) {
    const d = new Date(data.target_date);
    if (isNaN(d.getTime())) {
        throw new Error("Invalid target date format.");
    }
    normalizedDate = d.toISOString().split("T")[0]; // Store as YYYY-MM-DD
}

// 5. Normalization
const normalizedNotes = data.notes?.trim() || null;
const finalNotes = normalizedNotes === "" ? null : normalizedNotes;

return {
    objective_name: data.objective_name.trim(),
    objective_type: data.objective_type as ObjectiveType,
    target_amount: targetAmount,
    current_amount: currentAmount,
    priority_level: data.priority_level as ObjectivePriority,
    status: data.status as ObjectiveStatus,
    target_date: normalizedDate,
    notes: finalNotes,
};
}
```

FILE: src/lib/finance/taxonomy.ts

```
/**
 * AXIOM RETURN-BASED TAXONOMY
 * Centralized definitions for the financial intelligence system.
 */

export const VALID_CATEGORIES = [
    "Asset",
    "Skill",
    "Leverage",
    "Experience",
    "Maintenance",
    "Leakage",
] as const;
export type ValidCategory = (typeof VALID_CATEGORIES)[number];

export interface TaxonomyCategory {
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
value: ValidCategory;
label: string;
definition: string;
interpretation: string;
}

export const TAXONOMY_CATEGORIES: TaxonomyCategory[] = [
  {
    value: "Asset",
    label: "Investments",
    definition: "Future value generation (Stocks, Crypto, Property)",
    interpretation:
      "Strategic capital foundation. These deployments are expected to produce tangible appreciation or yield over time.",
  },
  {
    value: "Skill",
    label: "Growth",
    definition: "Learning and high-income skill development",
    interpretation:
      "Human capital investment. Deployments into skills increase your intrinsic market value and future inflow capacity.",
  },
  {
    value: "Leverage",
    label: "Efficiency",
    definition: "Buying back time or multiplying output",
    interpretation:
      "Operational efficiency. Leverage deployments are designed to buy back time or multiply the results of your effort.",
  },
  {
    value: "Experience",
    label: "Lifestyle",
    definition: "Intentional quality-of-life and memories",
    interpretation:
      "Strategic utility. Non-yielding deployments that provide intentional lived-value and mental sustainability.",
  },
  {
    value: "Maintenance",
    label: "Essentials",
    definition: "Bills, rent, food, and survival core",
    interpretation:
      "Operational core. Survival-layer capital needed to maintain your current system (Rent, utilities, basic nourishment).",
  },
  {
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
        value: "Leakage",
        label: "Waste",
        definition: "Non-strategic drift or impulse spending",
        interpretation:
            "Systemic inefficiency. Tracked separately to identify recurring capital drift. Excess leakage compresses
operational runway.",
    },
];

export const isValidCategory = (value: string): boolean => {
    return VALID_CATEGORIES.includes(value as ValidCategory);
};

export const formatTaxonomyCategoryList = (): string => {
    return VALID_CATEGORIES.join(", ");
};

export const getTaxonomyDefinition = (value: string) => {
    return TAXONOMY_CATEGORIES.find((c) => c.value === value)?.definition || "";
};

export const getTaxonomyInterpretation = (value: string) => {
    return (
        TAXONOMY_CATEGORIES.find((c) => c.value === value)?.interpretation || ""
    );
};
```

FILE: src/lib/finance/validateDeployment.ts

```
/**
 * TAXONOMY ENFORCEMENT GATE
 * Ensures every deployment written to the ledger follows the strict return-based taxonomy.
 */

import {
    formatTaxonomyCategoryList,
    isValidCategory,
} from "../taxonomy";
import {
    DeploymentAdvancedContextInput,
    normalizeDeploymentAdvancedContext,
} from "../deploymentContext";
import { evaluateMetadataQuality } from "../metadataQuality";

export { VALID_CATEGORIES, type ValidCategory } from "../taxonomy";

export interface DeploymentInput {
    title: string;
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
    amount: number;
    category: string;
    impactScore?: number;
    advancedContext?: DeploymentAdvancedContextInput;
  }

/**
 * Pre-write Validation Layer
 * Rejects any input that violates taxonomy integrity or data truth.
 */
export function validateDeployment(input: DeploymentInput) {
  const { title, amount, category, impactScore, advancedContext } = input;

  // 1. Structural Integrity
  if (!title || !title.trim()) {
    throw new Error("Taxonomy Gate: Title is required for deployment verification");
  }

  if (amount === undefined || amount === null || isNaN(amount)) {
    throw new Error("Taxonomy Gate: Numeric amount is required");
  }

  if (amount <= 0) {
    throw new Error("Taxonomy Gate: Capital deployment must be a positive value (> 0)");
  }

  // 2. Taxonomy Enforcement
  if (!isValidCategory(category)) {
    // Specifically reject 'Unclassified' and legacy categories
    if (category === "Unclassified") {
      throw new Error("Taxonomy Gate: Strategic classification is mandatory. Entry rejected.");
    }

    throw new Error(`Taxonomy Gate: Unknown category '${category}'. Use ${formatTaxonomyCategoryList()}.`);
  }

  // 3. Normalization
  return {
    title: title.trim(),
    amount: Number(amount),
    category,
    advancedContext: normalizeDeploymentAdvancedContext(advancedContext),
    metadataQuality: evaluateMetadataQuality(title),
    impactScore: Math.min(Math.max(impactScore || 0, 0), 10) // Scale 0-10
  };
}
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

FILE: src/lib/analytics/engine.test.ts

```
import assert from "node:assert/strict";
import test from "node:test";
import {
  projectRunway,
  calculateBurnRate,
  calculateIncomeMetrics,
  countPendingVerifications,
} from "../engine";
import type { Deployment, IncomeStream, OperationalBaseline } from "../types";

test("projectRunway handles dimensional math correctly", () => {
  // Scenario: 1000 balance, 10 daily burn, 0 income
  // Runway = 1000 / 10 = 100 days
  assert.equal(projectRunway(1000, 10, 0), 100);

  // Scenario: 1000 balance, 10 daily burn, 150 monthly income
  // adjustedDailyBurn = 10 - (150 / 30) = 5
  // Runway = 1000 / 5 = 200 days
  assert.equal(projectRunway(1000, 10, 150), 200);
});

test("projectRunway returns null when state is stable (income >= burn)", () => {
  // Scenario: 10 daily burn, 300 monthly income
  // adjustedDailyBurn = 10 - (300 / 30) = 0
  assert.equal(projectRunway(1000, 10, 300), null);

  // Scenario: 10 daily burn, 450 monthly income
  // adjustedDailyBurn = 10 - (450 / 30) = -5
  assert.equal(projectRunway(1000, 10, 450), null);
});

test("projectRunway returns 0 when balance is 0 or negative (and burn > income)", () => {
  assert.equal(projectRunway(0, 10, 0), 0);
  assert.equal(projectRunway(-100, 10, 0), 0);
});

test("projectRunway returns null when dailyBurnRate is 0 or negative (unstable state but no burn)", () => {
  assert.equal(projectRunway(1000, 0, 0), null);
  assert.equal(projectRunway(1000, -5, 0), null);
});

test("calculateBurnRate remains deterministic", () => {
  const deployments: Deployment[] = [
    {
      id: "1",
      amount: 300,
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
        title: "Rent",
        category: "Leakage",
        created_at: new Date().toISOString(),
    },
    {
        id: "2",
        amount: 150,
        title: "Cloud",
        category: "Leverage",
        created_at: new Date().toISOString(),
    },
];

// 450 total / 30 days = 15/day
assert.equal(calculateBurnRate(deployments, 30), 15);
});

test("calculateMaxIncomeConcentration: 90% dependency", () => {
    const streams: Partial<IncomeStream>[] = [
        {
            income_name: "Source A",
            amount: 900,
            cadence: "monthly",
            income_type: "salary",
            is_recurring: true,
        },
        {
            income_name: "Source B",
            amount: 100,
            cadence: "monthly",
            income_type: "freelance",
            is_recurring: true,
        },
    ];

    // total = 1000
    // max = 900
    // ratio = 0.9
    const metrics = calculateIncomeMetrics(streams as IncomeStream[]);
    assert.equal(metrics.maxRatio, 0.9);
});

test("calculateMaxIncomeConcentration: balanced split", () => {
    const streams: Partial<IncomeStream>[] = [
        {
            income_name: "Source A",
            amount: 100,
            cadence: "monthly",
```


AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
        income_type: "salary",
        is_recurring: true,
    },
    {
        income_name: "Source B",
        amount: 100,
        cadence: "monthly",
        income_type: "salary",
        is_recurring: true,
    },
    {
        income_name: "Source C",
        amount: 100,
        cadence: "monthly",
        income_type: "salary",
        is_recurring: true,
    },
];

// total = 300
// max = 100
// ratio = 0.333...
const metrics = calculateIncomeMetrics(streams as IncomeStream[]);
assert.ok(Math.abs(metrics.maxRatio - 0.3333333333333333) < 0.0001);
});

test("calculateMaxIncomeConcentration: zero income", () => {
    const metrics = calculateIncomeMetrics([]);
    assert.equal(metrics.maxRatio, 0);
});

test("countPendingVerifications includes baseline logic", () => {
    const today = new Date();
    const currentDayOfMonth = today.getDate();
    const currentDayOfWeek = today.getDay() === 0 ? 7 : today.getDay();

    const baseline: Partial<OperationalBaseline>[] = [
        {
            id: "b1",
            title: "Daily Coffee",
            cadence: "daily",
            is_recurring: true,
            last_executed_at: null,
        },
        {
            id: "b2",
            title: "Weekly Server",
            cadence: "weekly",
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
        is_recurring: true,
        execution_day: currentDayOfWeek,
        last_executed_at: null,
    },
    {
        id: "b3",
        title: "Monthly Rent",
        cadence: "monthly",
        is_recurring: true,
        execution_day: currentDayOfMonth,
        last_executed_at: null,
    },
];

assert.equal(
    countPendingVerifications([], [], baseline as OperationalBaseline[]),
    3,
);

// If already executed today, should be 0
const executedTodayBaseline = baseline.map((b) => ({
    ...b,
    last_executed_at: today.toISOString(),
}));
assert.equal(
    countPendingVerifications(
        [],
        [],
        executedTodayBaseline as OperationalBaseline[],
    ),
    0,
);
});
```

FILE: src/lib/analytics\engine.ts

```
import {
    Deployment,
    AnalyticsSummary,
    Account,
    Liability,
    IncomeStream,
    FinancialGoal,
    StrategicObjective,
    StrategicAlignment,
    OperationalBaseline,
} from "../types";
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
/**
 * Normalizes cadences into monthly values.
 */
export const normalizeToMonthly = (amount: number, cadence: string): number => {
  switch (cadence) {
    case "daily":
      return amount * 30.4375;
    case "weekly":
      return amount * 4.345;
    case "biweekly":
      return amount * 2.1725;
    case "monthly":
      return amount;
    case "quarterly":
      return amount / 3;
    case "yearly":
      return amount / 12;
    default:
      return amount;
  }
};

/**
 * Calculates monthly structural burn and systemic allocation metrics.
 */
export const calculateBaselineMetrics = (baseline: OperationalBaseline[]) => {
  return baseline.reduce(
    (acc, item) => {
      if (!item.is_active || !item.is_recurring) return acc;
      const monthlyAmount = normalizeToMonthly(item.amount, item.cadence);

      if (item.baseline_type === "expense") {
        acc.monthlyBurn += monthlyAmount;
      } else {
        acc.monthlyAllocation += monthlyAmount;
      }
      return acc;
    },
    { monthlyBurn: 0, monthlyAllocation: 0 },
  );
};

import { summarizeMetadataQuality } from "../finance/metadataQuality";
import { calculateMonthlyInflow } from "../finance/income";
import { calculateGoalProgressPercentage } from "../finance/goals";
import { calculateObjectiveFundingRatio } from "../finance/objectives";

/**
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
* Pure, deterministic engine for financial intelligence.
* Rule: The database is truth. This engine only interprets.
*/

export const calculateLiabilityMetrics = (liabilities: Liability[]) => {
  return liabilities.reduce(
    (acc, l) => {
      const balance = Number(l.outstanding_balance);
      acc.totalBalance += balance;

      if (l.is_paid_in_cadences && l.interest_rate > 0) {
        // Interest is explicitly "per cadence" as per UI/Requirement
        const periodicInterest = balance * (l.interest_rate / 100);

        let monthlyInterest = 0;
        if (l.cadence === "weekly") {
          monthlyInterest = periodicInterest * (52 / 12);
        } else if (l.cadence === "monthly") {
          monthlyInterest = periodicInterest;
        }
        acc.monthlyInterestAccrual += monthlyInterest;
      }
      return acc;
    },
    { totalBalance: 0, monthlyInterestAccrual: 0 },
  );
};

export const calculateTotal = (deployments: Deployment[]): number => {
  return deployments.reduce((sum, d) => sum + Number(d.amount), 0);
};

export const calculateAccountTotal = (accounts: Account[]): number => {
  return accounts.reduce((sum, a) => sum + Number(a.current_balance), 0);
};

export const calculateLiabilityTotal = (liabilities: Liability[]): number => {
  return calculateLiabilityMetrics(liabilities).totalBalance;
};

export const calculateIncomeMetrics = (streams: IncomeStream[]) => {
  const result = streams.reduce(
    (acc, stream) => {
      const monthly = calculateMonthlyInflow(stream);
      acc.total += monthly;
      if (stream.is_recurring) {
        acc.recurring += monthly;
      } else {

```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
        acc.irregular += monthly;
    }
    acc.concentration[stream.income_type] =
        (acc.concentration[stream.income_type] || 0) + monthly;

    // Group by origin source (income_name) for concentration risk analysis
    acc.sourceTotals[stream.income_name] =
        (acc.sourceTotals[stream.income_name] || 0) + monthly;

    return acc;
},
{
    total: 0,
    recurring: 0,
    irregular: 0,
    concentration: {} as Record<string, number>,
    sourceTotals: {} as Record<string, number>,
},
);

const highestSourceSum = Math.max(0, ...Object.values(result.sourceTotals));
const maxRatio = result.total > 0 ? highestSourceSum / result.total : 0;

return {
    total: result.total,
    recurring: result.recurring,
    irregular: result.irregular,
    concentration: result.concentration,
    maxRatio,
};
};

export const calculateGoalMetrics = (
    goals: FinancialGoal[],
    objectives: StrategicObjective[] = [],
) => {
    const goalStats = goals.reduce(
        (acc, goal) => {
            acc.totalTargets += Number(goal.target_amount);
            acc.totalProgress += Number(goal.current_progress);
            if (goal.priority === "critical" && goal.status === "active") {
                acc.criticalCount += 1;
            }
            acc.progressSum += calculateGoalProgressPercentage(goal);
            return acc;
        },
        {
            totalTargets: 0,
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
        totalProgress: 0,
        criticalCount: 0,
        progressSum: 0,
    },
);

return objectives.reduce((acc, obj) => {
    acc.totalTargets += Number(obj.target_amount);
    acc.totalProgress += Number(obj.current_amount);
    if (obj.priority_level === "critical" && obj.status === "active") {
        acc.criticalCount += 1;
    }
    acc.progressSum += calculateObjectiveFundingRatio(obj);
    return acc;
}, goalStats);
};

export const calculateAverage = (deployments: Deployment[]): number => {
    if (deployments.length === 0) return 0;
    return calculateTotal(deployments) / deployments.length;
};

export const countPendingVerifications = (
    income: IncomeStream[],
    liabilities: Liability[],
    baseline: OperationalBaseline[] = [],
): number => {
    const today = new Date();
    const currentDayOfWeek = today.getDay();
    const currentDayOfWeekName = today
        .toLocaleString("en-US", { weekday: "long" })
        .toLowerCase();
    const currentDayOfMonth = today.getDate();
    const mappedDayOfWeek = currentDayOfWeek === 0 ? 7 : currentDayOfWeek;

    const pendingIncomes = income.filter((stream) => {
        if (!stream.is_recurring || !stream.execution_day) return false;

        if (stream.last_executed_at) {
            const lastExecuted = new Date(stream.last_executed_at);
            if (
                lastExecuted.getDate() === today.getDate() &&
                lastExecuted.getMonth() === today.getMonth() &&
                lastExecuted.getFullYear() === today.getFullYear()
            ) {
                return false;
            }
        }
    });
}
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
if (stream.cadence === "weekly") {
  return mappedDayOfWeek === stream.execution_day;
}
if (stream.cadence === "monthly" || stream.cadence === "biweekly") {
  return currentDayOfMonth === stream.execution_day;
}

return false;
});

const pendingLiabilities = liabilities.filter((liab) => {
  if (!liab.is_paid_in_cadences || !liab.cadence_day_date) return false;

  if (liab.last_executed_at) {
    const lastExecuted = new Date(liab.last_executed_at);
    if (
      lastExecuted.getDate() === today.getDate() &&
      lastExecuted.getMonth() === today.getMonth() &&
      lastExecuted.getFullYear() === today.getFullYear()
    ) {
      return false;
    }
  }

  if (liab.cadence === "weekly") {
    return currentDayOfWeekName === liab.cadence_day_date.toLowerCase();
  }
  if (liab.cadence === "monthly") {
    return currentDayOfMonth === parseInt(liab.cadence_day_date, 10);
  }

  return false;
});

const pendingBaselines = baseline.filter((b) => {
  if (!b.is_recurring) return false;

  if (b.last_executed_at) {
    const lastExecuted = new Date(b.last_executed_at);
    if (
      lastExecuted.getDate() === today.getDate() &&
      lastExecuted.getMonth() === today.getMonth() &&
      lastExecuted.getFullYear() === today.getFullYear()
    ) {
      return false;
    }
  }
});
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
    if (b.cadence === "daily") return true;

    if (b.execution_day) {
      if (b.cadence === "weekly") {
        return mappedDayOfWeek === b.execution_day;
      }
      if (b.cadence === "monthly" || b.cadence === "biweekly") {
        return currentDayOfMonth === b.execution_day;
      }
    }

    return false;
  });
  return (
    pendingIncomes.length + pendingLiabilities.length + pendingBaselines.length
  );
};

/**
 * Calculates burn rate over a specific window (in days).
 * Defaulting to a 30-day window for normalized metrics.
 */
export const calculateBurnRate = (
  deployments: Deployment[],
  windowDays: number = 30,
): number => {
  if (windowDays <= 0) return 0;
  const total = calculateTotal(deployments);
  return total / windowDays;
};

/**
 * Projects runway based on deterministic liquidity and income offset.
 * Correct Formula: Runway = balance / (dailyBurnRate - (monthlyIncome / 30))
 * Note: Returns days. Returns null if adjusted burn is <= 0 (stable state).
 */
export const projectRunway = (
  balance: number,
  dailyBurnRate: number,
  monthlyIncome: number = 0,
  netWorth: number = 0,
): number | null => {
  const adjustedDailyBurn = dailyBurnRate - monthlyIncome / 30;

  // If net worth is negative, the structural foundation is critical regardless of cash flow
  if (netWorth < 0) return 0;
```


AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
    if (adjustedDailyBurn <= 0) return null;
    if (balance <= 0) return 0;

    return balance / adjustedDailyBurn;
};

/**
 * Aggregates deployments into categories.
 */
export const getCategoryBreakdown = (
    deployments: Deployment[],
): Record<string, number> => {
    return deployments.reduce(
        (acc, d) => {
            const cat = d.category || "Unclassified";
            acc[cat] = (acc[cat] || 0) + Number(d.amount);
            return acc;
        },
        {} as Record<string, number>,
    );
};

/**
 * Deterministic Strategic Alignment Logic
 */

export function calculateStrategicAlignment(
    objectives: StrategicObjective[],
    deployments: Deployment[],
    liquidity: number,
    incomeTotal: number,
    burnRate: number,
    liabilities: Liability[],
    baseline: OperationalBaseline[] = [],
): StrategicAlignment {
    const fundingRatios: Record<string, number> = {};
    const velocity: Record<
        string,
        "stable" | "accelerating" | "stagnant" | "regressing"
    > = {};
    const objectiveStarvationSignals: string[] = [];
    const strategicAllocationSignals: string[] = [];
    const now = new Date();

    // 1. Funding Ratios & Starvation
    objectives.forEach((obj) => {
        const ratio = calculateObjectiveFundingRatio(obj);
        fundingRatios[obj.id] = ratio;
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
const ageInDays = Math.max(
  1,
  (now.getTime() - new Date(obj.created_at).getTime()) /
    (1000 * 60 * 60 * 24),
);

// Starvation detection
if (obj.status === "active" && ratio < 10 && ageInDays > 90) {
  objectiveStarvationSignals.push(
    `${obj.objective_name} objective has remained below 10% funded for ${Math.floor(ageInDays)} days.`
  );
}

// Velocity (Simplified deterministic approach based on age)
const dailyRate = obj.current_amount / ageInDays;
if (dailyRate === 0) {
  velocity[obj.id] = "stagnant";
} else if (ratio >= 100) {
  velocity[obj.id] = "stable"; // Completed
} else {
  velocity[obj.id] = "stable"; // Default for now as we don't have historical delta
}
});

// 2. Strategic Allocation Awareness
const categories = getCategoryBreakdown(deployments);
const leakage = categories["Leakage"] || 0;
const assets = categories["Asset"] || 0;
const hasActiveAccumulation = objectives.some(
  (o) =>
    o.status === "active" &&
    (o.objective_type === "liquidity" ||
      o.objective_type === "emergency_reserve"),
);

if (hasActiveAccumulation && leakage > assets) {
  strategicAllocationSignals.push(
    "Leakage deployments exceeded Asset deployments during an active liquidity accumulation objective."
  );
}

// 3. Liquidity Sufficiency
const criticalObjectives = objectives.filter(
  (o) => o.priority_level === "critical" && o.status === "active",
);
const totalCriticalTarget = criticalObjectives.reduce(
  (sum, o) => sum + (o.target_amount - o.current_amount),
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
    0,
  );
  const isSufficient = liquidity >= totalCriticalTarget;
  const shortfall = Math.max(0, totalCriticalTarget - liquidity);

  const liquiditySufficiency = {
    isSufficient,
    message: isSufficient
      ? "Current liquidity structure sufficiently supports short-term obligations."
      : `Emergency reserve target exceeds current liquidity capacity by ${shortfall.toLocaleString()} KSh.`,
    shortfall,
  };

  // 4. Alignment Pressure (Weighted Scoring 0-100)
  let pressureScore = 0;

  // Weight: Leakage vs Assets (up to 25 points)
  if (leakage > assets) pressureScore += 25;
  else if (leakage > 0) pressureScore += 10;

  // Weight: Liquidity Sufficiency (up to 30 points)
  if (!isSufficient) pressureScore += 30;

  // Weight: Starvation (up to 20 points)
  if (objectiveStarvationSignals.length > 0) pressureScore += 20;

  // Weight: High Liabilities (up to 25 points)
  const totalLiabilities = liabilities.reduce(
    (sum, l) => sum + Number(l.outstanding_balance),
    0,
  );
  if (totalLiabilities > liquidity * 2) pressureScore += 25;

  return {
    fundingRatios,
    velocity,
    alignmentPressure: Math.min(100, pressureScore),
    liquiditySufficiency,
    objectiveStarvationSignals,
    strategicAllocationSignals,
  };
}

/**
 * Main entry point for generating a full analytics context.
 */
export const generateSummary = (
  deployments: Deployment[],
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
liquidity: number = 0,
accounts: Account[] = [],
liabilities: Liability[] = [],
incomeStreams: IncomeStream[] = [],
goals: FinancialGoal[] = [],
objectives: StrategicObjective[] = [],
baseline: OperationalBaseline[] = [],
): AnalyticsSummary => {
  const total = calculateTotal(deployments);
  const baselineMetrics = calculateBaselineMetrics(baseline);
  const liabilityMetrics = calculateLiabilityMetrics(liabilities);

  // Total daily burn = (Deployment Burn) + (Baseline Burn / 30) + (Liability Interest Burn / 30)
  const deploymentBurn = calculateBurnRate(deployments);
  const baselineDailyBurn = baselineMetrics.monthlyBurn / 30;
  const liabilityInterestDailyBurn =
    liabilityMetrics.monthlyInterestAccrual / 30;
  const burnRate =
    deploymentBurn + baselineDailyBurn + liabilityInterestDailyBurn;

  const totalAssets = calculateAccountTotal(accounts);
  const totalLiabilities = liabilityMetrics.totalBalance;
  const netWorth = totalAssets - totalLiabilities;

  const income = calculateIncomeMetrics(incomeStreams);
  const goalMetrics = calculateGoalMetrics(goals, objectives);

  const strategicAlignment = calculateStrategicAlignment(
    objectives,
    deployments,
    liquidity,
    income.total,
    burnRate,
    liabilities,
    baseline,
  );

  const totalIntentionsCount = goals.length + objectives.length;

  return {
    totalDeployed: total,
    averageDeployment: calculateAverage(deployments),
    dailyBurnRate: burnRate,
    runwayDays: projectRunway(liquidity, burnRate, income.total, netWorth),
    categoryBreakdown: getCategoryBreakdown(deployments),
    deploymentCount: deployments.length,
    metadataQuality: summarizeMetadataQuality(deployments),
    // Liability System v1
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
totalLiabilities,
totalAssets,
netWorth,
liquidity,
// Income Engine v1
totalMonthlyIncome: income.total,
recurringIncome: income.recurring,
irregularIncome: income.irregular,
incomeConcentration: income.concentration,
maxIncomeConcentrationRatio: income.maxRatio,
adjustedDailyBurn: Math.max(0, burnRate - income.total / 30),
// Goal System v1
totalStrategicTargets: goalMetrics.totalTargets,
totalCurrentProgress: goalMetrics.totalProgress,
averageGoalProgress:
  totalIntentionsCount > 0
    ? goalMetrics.progressSum / totalIntentionsCount
    : 0,
criticalGoalCount: goalMetrics.criticalCount,
fundingGap: Math.max(
  0,
  goalMetrics.totalTargets - goalMetrics.totalProgress,
),
// Strategic Objectives v1
strategicAlignment,
// Operational Baseline v1
totalStructuralMonthlyBurn: baselineMetrics.monthlyBurn,
totalSystemicMonthlyAllocation: baselineMetrics.monthlyAllocation,
pendingVerificationCount: countPendingVerifications(
  incomeStreams,
  liabilities,
  baseline,
),
};
};
```

FILE: src/lib/analytics\index.ts

```
export * from "../types";
export * from "../engine";
```

FILE: src/lib/analytics\strategic.test.ts

```
import { test } from "node:test";
import assert from "node:assert";
import { calculateStrategicAlignment } from "../engine";
import type { Deployment, StrategicObjective, Liability } from "../types";
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
test("strategic alignment funding ratios are deterministic", () => {
  const objectives: StrategicObjective[] = [
    {
      id: "obj1",
      user_id: "u1",
      objective_name: "Emergency Fund",
      objective_type: "emergency_reserve",
      target_amount: 1000,
      current_amount: 500,
      target_date: null,
      priority_level: "critical",
      status: "active",
      created_at: new Date().toISOString(),
      updated_at: new Date().toISOString(),
    },
  ];

  const alignment = calculateStrategicAlignment(objectives, [], 0, 0, 0, []);
  assert.strictEqual(alignment.fundingRatios["obj1"], 50);
});

test("alignment pressure scoring reflects structural conflict", () => {
  const objectives: StrategicObjective[] = [
    {
      id: "obj1",
      user_id: "u1",
      objective_name: "Liquidity",
      objective_type: "liquidity",
      target_amount: 1000,
      current_amount: 0,
      target_date: null,
      priority_level: "critical",
      status: "active",
      created_at: new Date().toISOString(),
      updated_at: new Date().toISOString(),
    },
  ];

  const deployments: Deployment[] = [
    {
      id: "d1",
      amount: 500,
      title: "Bad Spending",
      category: "Leakage",
      created_at: new Date().toISOString(),
    },
  ];
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
const liabilities: Liability[] = [
  {
    id: "l1",
    user_id: "u1",
    liability_name: "Debt",
    liability_type: "personal_loan",
    outstanding_balance: 10000,
    created_at: "",
    updated_at: "",
    interest_rate: 0,
    due_date: null,
    currency: "KSh",
    is_paid_in_cadences: false,
    cadence: null,
    cadence_day_date: null,
    cadence_amount: null,
  },
];

// Scenario: High Leakage, No Assets, Insufficient Liquidity, High Liabilities
const alignment = calculateStrategicAlignment(
  objectives,
  deployments,
  1000,
  0,
  0,
  liabilities,
);

// Leakage > Assets (+25)
// !isSufficient (+30) - totalCriticalTarget is 1000, liquidity is 1000. Wait, 1000 >= 1000 is true.
// totalLiabilities (10000) > liquidity * 2 (2000) (+25)
// Score should be at least 50.

assert.ok(alignment.alignmentPressure >= 50);
});

test("liquidity sufficiency detects shortfalls", () => {
  const objectives: StrategicObjective[] = [
    {
      id: "obj1",
      user_id: "u1",
      objective_name: "Reserve",
      objective_type: "emergency_reserve",
      target_amount: 5000,
      current_amount: 1000,
      target_date: null,
      priority_level: "critical",
    },
  ],
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
        status: "active",
        created_at: new Date().toISOString(),
        updated_at: new Date().toISOString(),
    },
];

const alignment = calculateStrategicAlignment(objectives, [], 2000, 0, 0, []);

assert.strictEqual(alignment.liquiditySufficiency.isSufficient, false);
assert.strictEqual(alignment.liquiditySufficiency.shortfall, 2000); // 4000 remaining target - 2000 liquidity
});

test("objective starvation detection works for old stagnant goals", () => {
    const sixMonthsAgo = new Date();
    sixMonthsAgo.setMonth(sixMonthsAgo.getMonth() - 6);

    const objectives: StrategicObjective[] = [
        {
            id: "obj1",
            user_id: "u1",
            objective_name: "Starved Goal",
            objective_type: "investment",
            target_amount: 10000,
            current_amount: 100, // 1%
            target_date: null,
            priority_level: "high",
            status: "active",
            created_at: sixMonthsAgo.toISOString(),
            updated_at: sixMonthsAgo.toISOString(),
        },
    ];

    const alignment = calculateStrategicAlignment(objectives, [], 0, 0, 0, []);

    assert.ok(alignment.objectiveStarvationSignals.length > 0);
    assert.ok(alignment.objectiveStarvationSignals[0].includes("Starved Goal"));
});

test("strategic allocation awareness detects leakage conflict", () => {
    const objectives: StrategicObjective[] = [
        {
            id: "obj1",
            user_id: "u1",
            objective_name: "Build Cash",
            objective_type: "liquidity",
            target_amount: 1000,
            current_amount: 0,
            target_date: null,
```


AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
        priority_level: "high",
        status: "active",
        created_at: new Date().toISOString(),
        updated_at: new Date().toISOString(),
    },
];

const deployments: Deployment[] = [
    {
        id: "d1",
        amount: 1000,
        title: "Leakage",
        category: "Leakage",
        created_at: new Date().toISOString(),
    },
    {
        id: "d2",
        amount: 500,
        title: "Asset",
        category: "Asset",
        created_at: new Date().toISOString(),
    },
];

const alignment = calculateStrategicAlignment(
    objectives,
    deployments,
    0,
    0,
    0,
    [],
);

assert.ok(alignment.strategicAllocationSignals.length > 0);
assert.ok(
    alignment.strategicAllocationSignals[0].includes(
        "Leakage deployments exceeded Asset deployments",
    ),
);
});
```

FILE: src/lib/analytics/types.ts

```
import { MetadataQualitySummary } from "../../finance/metadataQuality";
import { DeploymentAdvancedContext } from "../../finance/deploymentContext";
import { Account } from "../../finance/accounts";
import { Liability } from "../../finance/liabilities";
import { IncomeStream } from "../../finance/income";
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
import { FinancialGoal } from "../finance/goals";
import { StrategicObjective } from "../finance/objectives";

export type { Account, AccountType } from "../finance/accounts";
export type { Liability, LiabilityType } from "../finance/liabilities";
export type { IncomeStream, IncomeType } from "../finance/income";
export type {
  FinancialGoal,
  GoalType,
  GoalPriority,
  GoalStatus,
} from "../finance/goals";
export type {
  StrategicObjective,
  ObjectiveType,
  ObjectivePriority,
  ObjectiveStatus,
} from "../finance/objectives";

export type BaselineCadence =
  | "daily"
  | "weekly"
  | "biweekly"
  | "monthly"
  | "quarterly"
  | "yearly";

export type BaselineType = "expense" | "allocation";

export interface OperationalBaseline {
  id: string;
  user_id: string;
  title: string;
  amount: number;
  category: string;
  cadence: BaselineCadence;
  execution_day?: number | null;
  last_executed_at?: string | null;
  is_recurring: boolean;
  baseline_type: BaselineType;
  is_active: boolean;
  notes?: string | null;
  created_at: string;
  updated_at: string;
  deleted_at?: string | null;
}

export interface Deployment {
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
id: string;
amount: number;
created_at: string;
category?: string | null;
title: string;
advanced_context?: DeploymentAdvancedContext | null;
account_id?: string | null;
deleted_at?: string | null;
}

export interface StrategicAlignment {
  fundingRatios: Record<string, number>; // objective_id -> ratio (0-100)
  alignmentPressure: number; // 0-100 deterministic score
  liquiditySufficiency: {
    isSufficient: boolean;
    message: string;
    shortfall: number;
  };
  objectiveStarvationSignals: string[];
  strategicAllocationSignals: string[];
  velocity: Record<
    string,
    "stable" | "accelerating" | "stagnant" | "regressing"
  >;
}

export interface AnalyticsSummary {
  totalDeployed: number;
  averageDeployment: number;
  dailyBurnRate: number;
  runwayDays: number | null;
  categoryBreakdown: Record<string, number>;
  deploymentCount: number;
  metadataQuality: MetadataQualitySummary;
  // Liability System v1
  totalLiabilities: number;
  netWorth: number;
  totalAssets: number;
  liquidity: number;
  // Income Engine v1
  totalMonthlyIncome: number;
  recurringIncome: number;
  irregularIncome: number;
  incomeConcentration: Record<string, number>;
  maxIncomeConcentrationRatio: number;
  adjustedDailyBurn: number;
  // Goal System v1
  totalStrategicTargets: number;
```

AXIOM REPOSITORY AUDIT

Generated on: 2026-06-09 08:52:31

```
totalCurrentProgress: number;
averageGoalProgress: number;
criticalGoalCount: number;
fundingGap: number;
// Strategic Objectives v1
strategicAlignment: StrategicAlignment;
// Operational Baseline v1
totalStructuralMonthlyBurn: number;
totalSystemicMonthlyAllocation: number;
pendingVerificationCount: number;
}
```